

Componentes/Payment/STBPayMultiNeg.PRW

266 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "Protheus.ch"
2 #INCLUDE "FWEVENTVIEWCONSTS.CH"
3 #INCLUDE "FWADAPTEREAI.CH"
4 #INCLUDE "FWMVCDEF.CH"
5 #INCLUDE "POSCSS.CH"
6 #INCLUDE "STPOS.CH"
7 #INCLUDE "STBPAYMULTINEG.CH"
8
9 Static cPicDefault      := "@E 9,999,999.99"    //Picture padrao para valores
10
11 #DEFINE POS_TIPO        1                      // Tipo
12 #DEFINE POS_QTDPARC     2                      // Quantidade de Parcelas
13 #DEFINE POS_VLRPARC     3                      // Valor Parcela
14 #DEFINE POS_VLRTOT      4                      // Valor Total
15 #DEFINE POS_VLRVEN      5                      // Valor até Vencimento
16 #DEFINE POS_ACRESC      6                      // Acrescimo
17 #DEFINE POS_VENCIM      7                      // Data Vencimento
18
19 //-----
20 /*{Protheus.doc} STBSimulMultiNeg
21 Acao para simular a Multi-Negociação
22
23 @param      oListPayForms    - ListBox da Forma de Pagamento
24 @param      oListDownPay     - Objeto ListBox de Entrada > Multi Negociacao
25 @param      aTypeDown       - Conteudo dos tipos de Entrada
26 @param      oDownPay        - Objeto Get correspondente a Entrada
27 @param      nDownPay        - Valor correspondente a Entrada
28 @param      aLblPayForms     - Representa os Labels da Forma de Pagamento
29 @param      oBtnSimul        - Botão de Simulação
30 @param      nLblTotEntr      - Valor correspondente ao Total de Entrada
31 @param      nLblRestante     - Valor correspondente ao Valor Restante da Venda
32 @param      aTpDownPay       - Array com conteudo de 8 labels referentes a entrada
33 @param      aResume          - Representa label: Entrada,Qtd.Parc,Vencimento,Valor,Acrescimo,Total,Forma,AdminFin,Desconto
34 @param      nSpace           - Tamanho de espaco entre uma informacao e outra
35 @param      nSaleVal         - Valor da Venda
36 @param      aItens           - Array com os itens da venda
37 @param      oMultNegoc       - Objeto de integração da Multinegociação
38 @param      aBrwContent      - Conteudo do Browse da Direita
39 @param      oBrwContent      - Objeto do Browse da Direita
40 @param      lEvenDesc        - Define se acao tem origem do evento de desconto
41 @return      NIL
42 @obs
43 @sample
44 */
45 //-----
46 Function STBSimulMultiNeg( oListPayForms, oListDownPay, aTypeDown, oDownPay, ;
47                          nDownPay, aLblPayForms, oBtnSimul, nLblTotEntr, ;
48                          nLblRestante, aTpDownPay, aResume, nSpace, ;
49                          nSaleVal, aItens, oMultNegoc, aBrwContent, ;
50                          oBrwContent, lEvenDesc)
51
52 Local nFor      := 0    //Variavel do laço

```

```

53 Local oRetItem      := NIL //Objeto de itens da multi-negociação
54 Local cCodigo       := "" //Codigo da Forma de Pagamento da Regra
55 Local nValParcel    := 0 //Valor de cada Parcela recalculado
56
57 Default lEvenDesc   := .F. //Define se acao tem origem do evento de desconto
58
59 If oListPayForms:nAt == 0
60     Return NIL
61 EndIF
62
63 If !lEvenDesc //Se a acao nao tem origem do evento de desconto, atualiza as formas de pagamentos
64     STDLblPayForms(oListPayForms,@aLblPayForms,@aTypeDown,nSpace,nSaleVal)
65     STIAtuObjEntr(oListDownPay,aTypeDown,oDownPay,@nDownPay,aLblPayForms,oBtnSimul,@nLblTotEntr,@nLblRestante,@aTpDownPay,@aResume)
66 EndIf
67
68 // Caso Necessitar de Entrada e o Valor de Entrada digitado estiver MENOR que o Necessário, critica a operacao
69 If VAL(aLblPayForms[05]) <> 0 .AND. nLblTotEntr < VAL(aLblPayForms[05])
70     STFMessage(ProcName(),"STOP",STR0001 + aLblPayForms[05] + ".") //"É necessário inserir Entrada Maior ou Igual a "
71     STFShowMessage(ProcName())
72     STFCleanMessage(ProcName())
73
74     STIMnClrRight(@aResume,aLblPayForms)
75     Return NIL
76 EndIF
77
78 // Caso Necessitar de Entrada e o Valor de Entrada digitado estiver maior que a venda, critica a operacao
79 If VAL(aLblPayForms[05]) <> 0 .AND. nLblTotEntr > nSaleVal
80     STFMessage(ProcName(),"STOP",STR0002) //"O valor de Entrada é maior que o valor da venda."
81     STFShowMessage(ProcName())
82     STFCleanMessage(ProcName())
83
84     STIMnClrRight(@aResume,aLblPayForms)
85     Return NIL
86 EndIF
87
88 STFMessage(ProcName(),"STOP",STR0003) //"Selecione ao lado uma Negociação e Confirme."
89 STFShowMessage(ProcName())
90 STFCleanMessage(ProcName())
91
92 cCodigo := SubStr(oListPayForms:aItems[oListPayForms:nAt],(00000000)+1 ,TamSx3("MBT_CODIGO")[1]) //
93
94 aParcels := {}
95 aBrwContent := {}
96
97 If !EMPTY(aItens)
98
99     If oMultNegoc:oDadosCab:Count() > 0
100         oMultNegoc:SetValor(nSaleVal)
101
102         If FindFunction("Frt060Ret")
103             nNCCUsada := Frt060Ret("NCC_USADA")
104             oMultNegoc:SetValNCC(nNCCUsada)

```

[illegible]

[illegible]

```

209     cTipoEntr := oListDownPay:aItems[oListDownPay:nAt]
210 EndIf
211
212 If EMPTY(cTipoEntr)
213     Return NIL
214 EndIF
215
216 cTipoEntr := AllTrim( SubStr(cTipoEntr,At("-",cTipoEntr)+1,LEN(cTipoEntr)) )
217 nTotalEntr := 0
218
219 // Se achar um label com o mesmo Tipo de Entrada, altera o valor na mesma casa
220 For nFor := 1 To Len(aTpDownPay)
221     If !EMPTY(aTpDownPay[nFor][01])
222         nPosAt := At("-",aTpDownPay[nFor][01])
223         If nPosAt > 0
224             If AllTrim(SubStr(aTpDownPay[nFor][01],1,nPosAt-1)) == cTipoEntr
225                 aTpDownPay[nFor][01] := cTipoEntr + " - " + Alltrim(Transform(nDownPay, cPicDefault))
226                 If EMPTY(aTpDownPay[nFor][01])
227                     aTpDownPay[nFor][02] := ALLTRIM( SUBSTR( oListDownPay:aItems[oListDownPay:nAt],1,At("-",oListDownPay:aItems[oListDownPay:nAt])-1 ) )
228                 EndIf
229                 // Soma ao total de Entradas
230                 nTotalEntr += nDownPay
231                 lAchou := .T.
232             Else
233                 // Apenas soma ao total de Entradas
234                 nTotalEntr += Val( SubStr(aTpDownPay[nFor][01],nPosAt+1,LEN(aTpDownPay[nFor][01])) )
235             EndIf
236         EndIf
237     EndIf
238 Next nFor
239
240 // Se não achou um label com o mesmo Tipo de Entrada, adiciono um novo na proxima casa
241 If !lAchou
242     // Caso Necessitar de Entrada e o Valor de Entrada digitado estiver MENOR que o Necessário, critica a operacao
243     If nDownPay == 0
244         STFMessage(ProcName(),"STOP",STR0004) //"É necessário inserir a Entrada."
245         STFShowMessage(ProcName())
246         STFCleanMessage(ProcName())
247         oDownPay:SetFocus()
248         Return NIL
249     EndIF
250
251     For nFor := 1 To Len(aTpDownPay)
252         If EMPTY(aTpDownPay[nFor][01])
253             aTpDownPay[nFor][01] := cTipoEntr + " - " + Alltrim(Transform(nDownPay, cPicDefault))
254             aTpDownPay[nFor][02] := ALLTRIM( SUBSTR( oListDownPay:aItems[oListDownPay:nAt],1,At("-",oListDownPay:aItems[oListDownPay:nAt])-1 ) )
255             // Soma ao total de Entradas
256             nTotalEntr += nDownPay
257         Exit
258     EndIF
259 Next nFor
260

```

```
261 EndIf
262
263 nLblTotEntr      := nTotalEntr
264 nLblRestante     := nSaleVal - nTotalEntr
265
266 Return NIL
```

Componentes/Payment/STBPayment.prw

2804 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPAYMENT.CH"
4
5 Static cMvEntExce := IIF(!Empty(SuperGetMV("MV_ENTEXCE",.F.,'')), "|"+SuperGetMV("MV_ENTEXCE",.F.,'')+"|", '|CC|CD|CH|FI|VA|VP|CO|FID|' ) // Formas de pagamento nao
    consideradas como entrada (Não será gravado L1_ENTRADA)
6 Static nEntrada := 0 // Valor de entrada
7 Static nArredondar := 0 // Valor de doação para o Instituto Arredondar
8 Static nDArredondar := 0 // Valor de doação para o Instituto Arredondar com desconto da taxa adm. se houver
9 Static nVLBF := 0 // Valor da bonificacao
10 Static aTotal := {} // Array contendo o resumo de pagamento
11 Static nContCheque := 0 // Contador que controla informação do cheque
12 Static aFormasImp := {} // Formas de pagamentos do orçamento importado
13 Static lMFE := IIF( ExistFunc("LjUsaMfe"), LjUsaMfe(), .F. ) //Se utiliza MFE
14 Static lLjRspFisc := ExistFunc("LjRspFisc")
15 Static cSiglaSat := IIF( ExistFunc("LjSiglaSat"),LjSiglaSat(), "SAT" ) //Retorna sigla do equipamento que esta sendo utilizado
16 Static lClientHtml := IIF( ExistFunc("GetCliHTML"), GetCliHTML(), .F. )
17
18 //-----
19 /*{Protoheus.doc} STBRetCup
20 Retorna o numero do cupom
21
22 @return cCupom - Numero do cupom
23 @obs
24 @sample
25 */
26 //-----
27 Function STBRetCup()
28
29 Local aDados := {Space(6), Nil}
30 Local cCupom := ""
31 Local cSerie := ""
32 Local aAreaSL1 := {}
33 Local cDoc := 0
34 Local cFilSL1 := ""
35 Local nTamDoc := 0
36
37 If STFGetCfg("lUseECF")
38
39 aRet := STFFireEvent(ProcName(0), "STGetReceipt", aDados)
40
41 If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) < 1
42 STFMessage("STIPayment","STOP",STR0014) //"Falha na obtenção do cupom!"
43 STFShowMessage("STIPayment")
44 Else
45 cCupom := aDados[1]
46 EndIf
47 Else
48 cCupom := STDGPBasket( "SL1" , "L1_DOC" )
49
50
51 If AllTrim(cCupom) == "NFCE"

```

```

52
53      //Busca o último documento emitido
54      aAreaSL1 := SL1->(GetArea())
55
56      cFilSL1 := xFilial("SL1")
57      nTamDoc := SL1->(TamSx3("L1_DOC"))[1]
58      cSerie := STDGPBasket("SL1", "L1_SERIE")
59      cDoc := Replicate("Z",nTamDoc)
60      cCupom := StrZero(0, nTamDoc)
61
62      DbSelectArea("SL1")
63      SL1->(DbSetOrder(2)) //L1_FILIAL + L1_SERIE + L1_DOC + L1_PDV
64      SL1->(DbSeek(cFilSL1 + cSerie + cDoc, .T.))
65
66      If !SL1->(Eof())
67          SL1->(DbSkip(-1))
68          Do While !SL1->(Eof()) .AND. SL1->(L1_FILIAL+L1_SERIE) == cFilSL1+cSerie .AND. AllTrim(SL1->L1_DOC) == "NFCE"
69              SL1->(DbSkip(-1))
70          EndDo
71          cCupom := SL1->L1_DOC
72          LjGrvLog("", "STBRetCup - Ultimo documento encontrado", {cCupom, cSerie})
73      EndIf
74
75      nTamDoc := Len(AllTrim( cCupom ))
76
77      cCupom := StrZero( Val(cCupom) + 1 , nTamDoc)
78
79      RestArea(aAreaSL1)
80      //cCupom := StrZero(Randomize(1,999999),6,0)
81      LjGrvLog("", "STBRetCup - Numero temporario de documento ", cCupom)
82  Else
83
84      If Empty( cCupom )
85          cCupom := AllTrim( STDGPBasket("SL1", "L1_NUM") + STDGPBasket("SL1", "L1_PDV") )
86      EndIf
87  EndIf
88
89 EndIf
90
91 Return cCupom
92
93 //-----
94 /*{Protheus.doc} STBConfPay
95 Confirmacao dos pagamentos
96
97 @param      oMdlGrd - Grid do Model
98 @param      aPaym   - Array de pagamentos
99 @param      oMdlPaym - Grid do Model de Pagamentos
100 @return     lRet - Se executou corretamente
101 @obs
102 @sample
103 */

```

```

104 //-----
105 Function STBConfPay(oMdlGrd, aPaym,oMdlPaym)
106
107 Local oTotal      := STGetTot()                //Recebe o objeto do model para recuperar os valores
108 Local lRet        := .T.                      //Variavel de retorno
109 Local nI          := 0                       //Variavel de loop
110 Local oTEF20      := IIF(ValType(STBGetTef()) == '0', STBGetTef(), STBGetChkRet())//Objeto do TEF ativo
111 Local lRecebTitle := STIGetRecTit()            //Retorna se e recebimento de titulo ou nao
112 Local oWFReceipt  := STIRetObjTit()           //Retorna o objeto da classe do recebimento de titulo
113 Local aValPre     := IIF(FindFunction("STBGetCodVP"),STBGetCodVP(), {}) //Carrega o codigo do vale presente
114 Local uResult     := Nil                     //Retorno do STBRemoteExecute de vale presente
115 Local lRetSTConfVen := .T.                  //Retorno do ponto de entrada
116 Local lFinServ    := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) //Define se habilita o controle de servicos financeiros
117 Local lPrintServ  := .F.                   //Verifica se imprime servicos financeiros
118 Local lRMS        := SuperGetMv("MV_LJRMS",,.F.) //Integracao com RMS
119 Local aMdlParc    := {}
120 Local cDoc        := ""                    //numero do documento
121 Local cSerie      := ""                    //serie do pdv
122 Local cEstacao    := ""                    //numero da estação
123 Local cPDV        := AllTrim(STDGPBasket("SL1" , "L1_PDV")) //Numero do pdv
124 Local cHora       := ""                    //hora da venda
125 Local cVend       := ""                    //codigo da venda
126 Local cCliente    := ""                    //codigo do cliente
127 Local cLoja       := ""                    //codigo da loja
128 Local dData       := cToD(" / / ")        //data da venda
129 Local nCount      := 0                     //contador
130 Local cTextScrpEd := ""                    //Texto Relatorio gerencial
131 Local lVendTpCard := .F.                   //Validação venda com cartão
132 Local aCupom      := {Space(6), Nil}      // Retorno Evento GetCupom
133 Local cCupom      := ""                    // Retorno número do Cupom
134 Local aPrinter    := {}                    // Armazena Retorno da impressora
135 Local lIsImpOrc    := If(FindFunction("STBIsImpOrc"),STBIsImpOrc(),!(Empty(STDGPBasket("SL1","L1_NUMORIG")))) //Verifica se é importação de orçamento
136 Local lMVLJPRDSV := SuperGetMv("MV_LJPRDSV",.F.,.F.) // Verifica se esta ativa a implementacao de venda com itens de "produto" e itens de "servico" em
    Notas Separadas (RPS)
137 Local lTemEntreg  := STBTemEntr()           //Verifica se tem item de "Entrega" ou "Retira Posterior" na venda
138 Local lRPS        := lMVLJPRDSV .And. (lIsImpOrc .Or. (ExistFunc("Lj7RPSNew") .And. Lj7RPSNew() .And. !STGetCfg("lUseECF"))) .And. STBTemServ() // Verifica se tem
    item de "Servico" (RPS)
139 Local cSerRPS     := ""
140 Local cDocRPS      := ""
141 Local cL1_SEND     := ""
142 Local nSendOn      := SuperGetMV("MV_LJSEND0",,0) //Retorno como sera a integracao da venda - 0 - via job - 1 online - 2 startjob
143 Local lEmitNfCe    := LjEmitNfCe()          //Sinaliza se utiliza NFC-e
144 Local lTefOk       := .F.                   //Validação se o Tef foi aprovado
145 Local lFinish      := .F.                   //Validação se a venda foi finalizada com sucesso
146 Local aMdlHeader  := {}                    //Variavel aHeader para o Ponto de entrada STConfSale
147 Local cNumOrig     := AllTrim(STDGPBasket( "SL1" , "L1_NUMORIG" )) //Numero de origem do orcamento
148 Local aRetFile     := {}
149 Local cMoedaSimb    := SuperGetMV( "MV_SIMB"+Str(STBGetCurrency(), 1) ) //Simbolo da moeda corrente
150 Local lPendSale    := .F.                   //Se FinishSale retornar .F., indica se a venda fica em aberto
151 Local cDocCliSL1   := ""                    //Retorna o L1_CGCLLI
152 Local aCustomer    := {}                    //Carrega informacoes do cliente Nome/Cpf
153 Local nCallCPF     := SuperGetMV("MV_LJDCCLI",,0) //O momento onde será mostrado o CPF na tela

```

```

154 Local lRetImpTef      := .T.           //Variavel de retorno impressao TEF
155 Local aDocSerie       := {}           //Armazeno o doc e série após processar o STWFinishSale()
156 Local cL1_Filial      := ""
157 Local aNotas          := {}
158 Local aPayment        := {}           //array com os pagamentos informados
159 Local nX              := 0           //contador
160 Local nY              := 0           //contador
161 Local lVendPed         := .F.         //Variavel de controle para gravação do L1_doc ou DOCPED
162 Local lIsValePres     := .F.         //Verifica se vale presente
163 Local cTextCred        := ""         //Texto comprovante de credito
164 Local lImpCompCred     := SuperGetMV("MV_LJIMPCR",,0) == 1 //Define se Imprime comprovante de credito
165 Local lSTConfRec       := ExistBlock("STConfRec") //Verifica se existe o ponto de entrada STConfRec
166 Local lSTConfSale     := ExistBlock("STConfSale") //Verifica se existe o ponto de entrada STConfSale
167 Local lExistSer        := .F.         //Verifica se existe a Serie Não Fiscal
168 Local lIntegrador      := IIF( ExistFunc("LjUsaIntgr"),LjUsaIntgr(), .T.) //Se utiliza MFe com Integrador
169 Local lUseSat          := LjUseSat() //Verifica se é SAT
170 Local aParamsImp       := {}         // Parâmetros para impressão do comprovante
171 Local aParams          := {}         // Parâmetros para a o RDMake de impressão
172 Local lLjVpCnf        := SuperGetMV("MV_LJVPCNF",,.F.) //verifica se imprime vale presente no cupom fiscal
    // Parâmetros para a o RDMake de impressão
173 Local oRaas           := Nil
174 Local oFidelityC       := Nil
175 Local nQtdItens        := 0
176 Local cMsgErrSer       := ""
177 Local aDescPgto        := {}
178 Local aItensFid        := {}
179 Local aFormPgto        := {}
180 Local aPayCols         := {}
181 Local oPhone           := Nil
182
183 Default oMdlGrd         := Nil
184 Default aPaym           := {}
185 Default oMdlPaym        := Nil
186
187 If lSTConfSale .OR. lSTConfRec
188
189     If oMdlGrd <> NIL
190         aMdlParc         := STBRPayaCols(oMdlGrd) //Parametro com o aCols do Grid
191         aMdlHeader       := oMdlGrd:GetOldData()[1] // Parametro com o aHeader do Grid
192     EndIf
193
194     If oMdlPaym <> Nil .And. Len(aMdlHeader) > 0
195         For nX := 1 to oMdlPaym:Length()
196             oMdlPaym:GoLine(nX)
197             If lRecebTitle
198                 If nX == oMdlPaym:Length()
199                     STBGrvTroco(oMdlPaym,nX, .T.)
200                 Else
201                     STDSPBasket("SL4","L4_TROCO",0,nX)
202                     oMdlPaym:LoadValue("L4_TROCO",0)
203                 EndIf
204             EndIf

```

```

205
206         aAdd( aPayment, {} )
207         For nY := 1 to Len(aMdlHeader)
208             aAdd( aPayment[nX],{aMdlHeader[nY][2], oMdlPaym:GetValue(aMdlHeader[nY][2])} )
209         Next nY
210     Next nX
211 EndIf
212
213 If lSTConfSale
214     LjGrvLog( NIL , "Antes da execução do PE STConfSale",{aMdlParc,aMdlHeader,lRecebTitle,oWFReceipt,aPayment})
215     lRetSTConfVen := ExecBlock("STConfSale",.F.,.F.,{aMdlParc,aMdlHeader,lRecebTitle,oWFReceipt,aPayment})
216     LjGrvLog( NIL , "Depois da execução do PE STConfSale",lRetSTConfVen)
217 EndIf
218 EndIf
219
220 If lRetSTConfVen .And. !Empty(cNumOrig) //Verifica se o orçamento ja foi importado
221     //Verifica se o arquivo já existe na retaguarda
222     aRetFile := STBCtrImpOrc( cNumOrig , cPdv , .T., .T. )
223
224     //se existir, nao permite finaliza-lo
225     If Len(aRetFile) > 0
226         If aRetFile[1]
227             STFMessage("STBPayment","STOP", STR0023) //"Não é possível finalizar a venda, este orçamento já foi finalizado por outro PDV."
228             STFShowMessage("STBPayment")
229             lRetSTConfVen := .F.
230         EndIf
231     EndIf
232
233 EndIf
234
235 If ValType(lRetSTConfVen) == 'L' .AND. lRetSTConfVen
236
237     If !lRecebTitle
238
239         /* Seta para .F. a variavel que controla se é P.O.S */
240         STISetContTef(.F.)
241         /*
242             Verifica se Todos os Itens do Cupom Fiscal foram cancelados. Caso afirmativo cancela o CF
243         */
244         lRet := STBCheckFiscalItens()
245
246         //Implementação antiga da rotina - retornava nil
247         If ValType(lRet) <> "L"
248             lRet := .T.
249         EndIf
250
251     If lRet
252
253         /*
254             Imprime os cheques
255         */
256         If STBGetCheck()

```

```

257
258         STWPayCheck()
259
260     EndIf
261
262     /*
263         Verificar se o cliente deseja doar para o Instituto Arredondar
264
265     */
266     If oMdlGrd:GetValue('L4_FORMA') = cMoedaSimb //Moeda corrente
267         STBInsArredondar(AlTrim(oMdlGrd:GetValue('L4_FORMA')))
268     EndIf
269
270     /*
271         Atualiza a cesta com os pagamentos
272     */
273     STIUpdBask()
274
275     //Se existir integracao com a RMS em CRM, executa as funcoes de Set dos pagamentos
276     If lRMS .AND. FindFunction("STFSetAuto")
277         STFSetAuto()
278     EndIf
279
280     //Verifico se o L1_CGCCLI está gravado no objeto, e se escolheu o cliente.
281     If lEmitNfce .AND. nCallCPF = 0 .AND. !STIGInfCPF() //Quando não chama o CPF para digitação (MV_LJDCCLI=0), gravará o L1_CGCCLI a partir do A1_CGC escolhido. A
função STIGInfCPF() garante que passou da tela do CPF ou não.
282         cDocCliSL1 := STDGPBasket("SL1","L1_CGCCLI")
283         If Empty(cDocCliSL1)
284             aCustomer := IIF(FindFunction('STDFindCust'),STDFindCust(),{}) //Carrega informacoes do cliente Nome/Cpf
285             If Len(aCustomer) >= 2 .AND. !Empty(aCustomer[2])
286                 STDSPBasket("SL1","L1_CGCCLI",aCustomer[2])
287             EndIf
288         EndIf
289     EndIf
290
291 EndIf
292
293 /*
294     Aplica Descontos do TEF
295 */
296
297 lVendTpCard := STIGetCard()
298
299 If lRet .AND. lVendTpCard
300     lRet := STBDescTEF(oMdlGrd, oTEF20, aPaym)
301 EndIf
302
303 /*
304     Imprime Desconto e Acréscimo
305 */
306 If lRet
307     STBDiscIncrease()

```

```

308      EndIf
309
310      /* Reorganiza o L2_ITEM */
311      STB0rgL2Item()
312
313      /*
314      Imprime as formas de pagamento
315      */
316      If lRet
317          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Imprime as formas de pagamento" ) //Gera LOG
318          lRet := STBPrintPay(oMdlGrd, oTEF20, aPaym)
319      EndIf
320
321      /*
322      Totalizando e fechando o cupom
323      */
324      If lRet
325          lRet := STBCloseCup(oTEF20, oTotal)
326
327          If lRet
328              /*
329              Antes de chamar a impressao do pagamento, o L1_SITUA sera mudado para '10' para possibilitar a recuperacao da venda
330              caso haja alguma falha (de energia por exemplo) durante a impressao dos pagamentos e comprovante TEF.
331              Caso ja esteja com situa = 10 não chama novamente para não duplicar a SL4
332              */
333              If AllTrim(STDGPBasket("SL1","L1_SITUA")) <> "10"
334                  STDSPBasket( "SL1" , "L1_SITUA", "10" )
335                  STDSaveSale()
336              Else
337                  LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Situa já foi definido para 10. Chamada Repetida. " )
338              EndIf
339          EndIf
340      EndIf
341
342      If lRet
343          /*
344          Se a forma de pagamento for VP entao atualiza o status do VP na retaguarda
345          */
346          STBStMdlVP(oMdlGrd)
347      EndIf
348
349
350      If !lRet
351          /*
352          Cancela a venda
353          */
354          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Força cancelamento da venda" ) //Gera LOG
355
356          STWCancelSale( .T. ) // Força cancelamento
357      Else
358
359          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Abre a gaveta" ) //Gera LOG

```

```

360
361      /* Abre a gaveta */
362      STB0penDrawer()
363  EndIf
364
365
366  If lRet
367
368      LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Cupom não fiscal" ) //Gera LOG
369
370      /*
371      Cupom não fiscal
372      */
373
374      /* Não imprime Cupom não fiscal antes da execução do TEF */
375      If !lVendTpCard
376          lPrintServ := STBPrintNotFiscal()[1]
377          If !lPrintServ
378              STFMessage("STBPayment","STOP",STR0026) //"Erro na impressão de comprovante não fiscal! "
379              STFShowMessage("STBPayment")
380              LjGrvLog("L1_NUM: "+cL1_SEND, STR0026 + " -1", cSerie)//"Erro na impressão de comprovante não fiscal! "
381              lRet:= .F.
382          Endif
383      EndIf
384
385      /*
386      Impressao de comprovante TEF
387      */
388      If lRet .AND. (lVendTpCard .OR. STBGetCheck())
389          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Impressao de comprovante TEF" ) //Gera LOG
390
391          /*Pega o numero do Comprovante Nao Fiscal, quando nao possui TEF, DOCPED é atualizado na rotina STBPrintNotFiscal()*/
392          If STBExistNItemFiscal()
393              If !STFGetCfg("lUseECF") //Nao utiliza ECF
394                  If lRPS
395                      cSerRPS := SuperGetMv("MV_LOJARPS",,"RPS")
396                      lExistSer:= LjxDNota( cSerRPS, 3, .F., 1, @aNotas,,,,,,,,,"RPS" ) // DOC/SERIE
397                      If lExistSer
398                          cDocRPS := aNotas[1][2]
399                      Else
400                          cMsgErrSer := STR0024 + " [" + cSerRPS + "]"
401                      EndIf
402                  EndIf
403
404                  If Empty(cMsgErrSer) .And. (!lRPS .Or. lTemEntreg)
405                      cSerie := STFGetStation("SERNFIS")
406                      lExistSer:= LjxDNota( cSerie, 3, .F., 1, @aNotas,,,,,,,,,"DOCNF" ) // DOC/SERIE
407                      If lExistSer
408                          cCupom := aNotas[1][2]
409                      Else
410                          cMsgErrSer := STR0024 + " [" + cSerie + "]"
411                      EndIf

```

```

412         EndIf
413
414         If !lExistSer
415             lRet := .F.
416             STFMessage("STBPayment","STOP",cMsgErrSer) //"Falha na obtenção da série não fiscal"
417             STFShowMessage("STBPayment")
418             LjGrvLog("L1_NUM: "+cL1_SEND, cMsgErrSer + " -2", cSerie)          //"Falha na obtenção da série não fiscal"
419         EndIf
420     ElseIf STFUseFiscalPrinter() //Utiliza impressora fiscal
421         aPrinter := STFFireEvent( ProcName(0) , "STGetReceipt" , aCupom )
422         If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aCupom) < 1
423             MsgStop(STR0017)          //"Erro ao pegar o número do cupom antes da impressão do comprovante. Verifique o ECF."
424         Else
425             cCupom := aCupom[1]
426             cCupom := StrZero( Val(cCupom)+1, Len(AllTrim(cCupom)) , 0 )
427             If lRPS
428                 cSerie := SuperGetMv("MV_LOJARPS",,"RPS")
429                 cDocRPS := cCupom
430                 cSerRPS := cSerie
431             Else
432                 cSerie := STFGetStation("SERIE")
433             EndIf
434         EndIf
435     EndIf
436
437     If !STBExistItemFiscal()
438         STDSPBasket( "SL1", "L1_DOC", "" )
439         STDSPBasket( "SL1", "L1_SERIE", "" )
440     EndIf
441     If lRPS //Tem item de "Servico" (RPS)
442         STDSPBasket( "SL1" , "L1_DOCRPS", cDocRPS )
443         STDSPBasket( "SL1" , "L1_SERRPS", cSerRPS )
444     EndIf
445
446     If (!lRPS .Or. lTemEntreg)
447         STDSPBasket( "SL1" , "L1_DOCPED", cCupom )
448         STDSPBasket( "SL1" , "L1_SERPED", cSerie )
449     EndIf
450 EndIf
451
452 If lEmitNfce .And. FindFunction("STWFinishValid")
453     lTefOk := .T.
454 Else
455     lRetImpTef := STWPrintComp(oTEF20)
456     //Se não imprimiu TEF, Zera Pagamentos se for apenas não fiscal
457     If !lRetImpTef .AND. !STBExistItemFiscal() .AND. STBExistNItemFiscal()
458         STIZeraPay()
459         lRet := .F.
460     EndIf
461 EndIf
462 EndIf
463

```

```

464      /* Imprime Relatório Gerencial caso a venda seja feita com cartão e possua itens não fiscais */
465      If lRet .AND. STBExistNItemFiscal() .AND. lVendTpCard
466
467          If lRPS // Tem item de "Servico" (RPS)
468              cTextScrPed := LJSCRPS( {.F., {}}, STBFactor()[5] )
469              STWManagReportPrint(cTextScrPed,1)
470          EndIf
471
472          If (!lRPS .Or. lTemEntreg)
473              cTextScrPed := LJSCRPED( {"",STBFactor()[4] } )
474              STWManagReportPrint(cTextScrPed,1)
475          EndIf
476
477          lPrintServ := .T.
478      EndIf
479
480      /*
481      Impressao de relatorio gerencial Garantia Estendida
482      */
483      If lRet .AND. lPrintServ .AND. ExistFunc("STWPrintGarEst")
484
485          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Impressao de relatorio gerencial Garantia Estendida" ) //Gera LOG
486
487          STWPrintGarEst()
488
489      EndIf
490
491      /*
492      Impressao de relatorio gerencial servicos financeiros
493      */
494      If lRet .AND. lFinServ .AND. lPrintServ
495
496          LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Impressao de relatorio gerencial servicos financeiros" ) //Gera LOG
497
498          STWPrintServ()
499
500      EndIf
501
502      /*
503      Verifico se um ShopCard esta sendo utilizado na venda e atualizo o saldo do mesmo.
504      */
505      If lRet .AND. STIShopCard()
506
507          For nI := 1 To oMdlGrd:Length()
508              If AllTrim(oMdlGrd:GetValue("L4_FORMA")) == "FID"
509                  lRet := STDUpdShopCardFunds(oMdlGrd,nI)
510              EndIf
511              If !lRet
512                  STFMessage(ProcName(),"STOP",STR0001+CRLF+STR0002) //"Nao foi possivel abater do saldo do cartao fidelidade" //"A venda nao podera ser finalizada"
513                  STFShowMessage(ProcName())
514                  STFCleanMessage(ProcName())
515                  Exit

```

```

516         EndIf
517     Next nI
518
519 EndIf
520
521 /*
522     Impressao do comprovante Brinde
523 */
524 If lRet .AND. AliasInDic("MGC")
525     STBPrintBrinde()
526 Endif
527
528 //Impressao do comprovante de crédito NCC pela função LJRUNSCRCD
529 If lRet .AND. lImpCompCred .AND. STDGPBasket("SL1", "L1_CREDITO") > 0 .AND. ( ExistBlock("SCRCD") .OR. ExistFunc("LJSCRCD") )
530
531     aParams := {STDGPBasket("SL1", "L1_CREDITO"), STDGetNCCs("1"), STDGetNCCs("2") , ;
532                 STDGPBasket("SL1", "L1_CLIENTE"), STDGPBasket("SL1", "L1_LOJA") }
533
534     If ExistBlock("SCRCD")
535         LjGrvLog("SCRCD", "Será impresso o comprovante de crédito do cliente.")
536         cTextCred := ExecBlock("SCRCD", .F., .F. , aParams )
537     Else
538         LjGrvLog("LJSCRCD", "Será impresso o comprovante de crédito do cliente.")
539         cTextCred := LJSCRCD( aParams )
540     EndIf
541
542     If !Empty(cTextCred)
543         STWManagReportPrint(cTextCred, 1)
544     EndIf
545 Endif
546
547 If lRet
548     /* Baixa o vale presente */
549     If Len(aValPre) > 0
550
551         //Verifico se atualizo o L1_DOC ou o L1_DOCPED
552         If Empty(STDGPBasket( "SL1", "L1_DOC"))
553             If !Empty(STDGPBasket( "SL1", "L1_DOCPED"))
554                 lVendPed := .T.
555             EndIf
556         EndIf
557
558         If lljVpCnf //verifica se imprime vale presente no cupom fiscal
559             /*Recupera os campos para serem atualizados na retaguarda*/
560             //numero do documento
561             cDoc := STDGPBasket( "SL1", "L1_DOC")
562             cDoc := IIF(!Empty(cDoc) , cDoc, STDGPBasket( "SL1", "L1_DOCPED"))
563
564             //serie do pdv
565             cSerie := STDGPBasket( "SL1", "L1_SERIE" )
566             cSerie := IIF(!Empty(cSerie), cSerie, STDGPBasket( "SL1", "L1_SERPED"))
567

```

```

568 Else
569     //Verifica se é vale presente:
570     lIsValePres := IIf(ExistFunc("STDExistVP"),STDExistVP(),.F.)
571
572     //Resgato a Serie e numero do DOC atual
573     cSerie := STFGetStation("SERNFIS")
574
575     If lIsValePres
576         cDoc := STDGPBasket("SL1", "L1_DOCPED")
577         cSerie := IIF(!Empty(cSerie),cSerie, STDGPBasket( "SL1" , "L1_SERPED"))
578     EndIf
579
580     //Se nao encontrou Doc, busca nova numeracao
581     If Empty(cDoc)
582         lExistSer:= LjxDNota( cSerie, 3, .F., 1, @aNotas )
583         If lExistSer
584             cDoc := aNotas[1][2]
585         Else
586             lRet := .F.
587             STFMessage("STBPayment","STOP",STR0024) //"Falha na obtenção da série não fiscal!"
588             STFS showMessage("STBPayment")
589             LjGrvLog("L1_NUM: "+cL1_SEND, STR0024 + " -3", cSerie)      //"Falha na obtenção da série não fiscal!"
590         EndIf
591     EndIf
592 EndIf
593
594 If lRet
595     //Atualizo a cesta
596     If lVendPed .Or. lIsValePres
597
598         STDSPBasket("SL1","L1_SERPED",cSerie)
599         STDSPBasket("SL1","L1_DOCPED",cDoc)
600
601         If lIsValePres .AND. !lLjVpCnf
602             For nX := 1 To STDPBLength("SL2")
603                 STDSPBasket( "SL2" , "L2_DOCPED" , cDoc , nX)
604                 STDSPBasket( "SL2" , "L2_SERPED" , cSerie , nX)
605                 STDSPBasket( "SL2" , "L2_DOC" , cDoc , nX)
606                 STDSPBasket( "SL2" , "L2_SERIE" , cSerie , nX)
607                 STDSaveSale(nX)
608             Next nX
609         Endif
610
611     EndIf
612     STDSPBasket("SL1","L1_SERIE",cSerie)
613     STDSPBasket("SL1","L1_DOC",cDoc)
614
615     cEstacao := STDGPBasket( "SL1" , "L1_ESTACAO" )//numero da estacao
616     cPDV := STDGPBasket( "SL1" , "L1_PDV" )//numero do pdv
617     dData := STDGPBasket( "SL1" , "L1_EMISNF" )//data da venda
618     cHora := SUBSTR(TIME(), 1, 5)//hora da venda
619     cVend := STDGPBasket( "SL1" , "L1_VEND" )//numero da venda

```

```

620         cCliente      := STDGPBasket( "SL1" , "L1_CLIENTE"      )//codigo do cliente
621         cLoja          := STDGPBasket( "SL1" , "L1_LOJA"        )//codigo da loja
622
623         FOR nCount := 1 To Len(aValPre)
624             /*Da baixa no vale presente na retaguarda */
625             MsgRun(STR0020,STR0021,{|| ; //"Executando as devidas configurações do Vale Presente/Crédito na retaguarda" ##"Aguarde....."
626                 STBRemoteExecute("STBBaixaPr" ,{      aValPre[nCount]      , STBGetVlrVP() , cDoc      , cSerie;;
627                                     cEstacao      , cPDV      , dData      , cHora ;;
628                                     cVend      , cCliente      , cLoja      }, NIL,.T.,@uResult))}
629         NEXT nCount
630
631         //Zera array codigos vale presentes
632         If ExistFunc("STBSetCodVP")
633             STBSetCodVP()
634         EndIf
635         //Zera array dos valores vale presentes
636         If ExistFunc("STBSetVlrVP")
637             STBSetVlrVP()
638         EndIf
639     EndIf
640 EndIf
641
642 If lRet .AND. lUseSat
643     STBGrvSatT()      //Prepara as informações de impostos PIS, Cofins, CST etc. para XML SAT
644 EndIf
645
646 If lRet
647     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Finaliza venda" ) //Gera LOG
648
649     cL1_Filial := STDGPBasket('SL1','L1_FILIAL')      //Numero L1 para envio online para retaguarda
650     cL1_SEND := STDGPBasket('SL1','L1_NUM')      //Numero L1 para envio online para retaguarda
651     cEstacao := STDGPBasket( "SL1" , "L1_ESTACAO")//numero da estacao para envio online para retaguarda
652     nQtdItens := STDPBLength("SL2")
653
654     //Itens da Venda
655     //carrega itens da venda neste ponto, pois quando é chamada a rotina StwfinishSale()
656     //os objetos da venda são reinicializados.
657     For nX := 1 To nQtdItens
658         aAdd(aItensFid, {      nX      ;;
659                             STDGPBasket("SL2", "L2_DESCRI" ,nX),;
660                             STDGPBasket("SL2", "L2_PRODUTO" ,nX),;
661                             STDGPBasket("SL2", "L2_QUANT" ,nX),;
662                             STDGPBasket("SL2", "L2_VLRITEM" ,nX) + STDGPBasket("SL2", "L2_VALDESC" ,nX),;
663                             STDGPBasket("SL2", "L2_VLRITEM" ,nX) })
664     Next nX
665
666     /*
667     Finaliza a venda
668     */
669     lFinish := STWFinishSale(@lPendSale)
670     cDoc := ""
671     cSerie := ""

```

```

672      If FindFunction("STWGetDocSerie")
673          aDocSerie := STWGetDocSerie()
674          If Len(aDocSerie) >= 2
675              cDoc      := aDocSerie[1]
676              cSerie    := aDocSerie[2]
677          EndIf
678      EndIf
679
680      If lFinish .And. ExistFunc("LjxRaasInt") .And. LjxRaasInt()
681
682          // -- Finaliza BonusHub
683          oRaas := STBGetRaas()
684
685          If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
686
687              oFidelityC := oRaas:GetFidelityCore()
688
689              //Formas de Pagamento da Venda
690              For nX := 1 To oMdlGrd:Length()
691                  aPayCols := STBRPayaCols(oMdlGrd)
692                  aDescPgto := FWGetSX5("24", aPayCols[nX][4])
693                  aAdd(aFormPgto, {      "",;
694                                      aDescPgto[1][4],;
695                                      aPayCols[nX][3]})
696              Next nX
697
698              oPhone := LjPhone():New(      POSICIONE("SA1", 1, xfilial("SA1") + SL1->L1_CLIENTE + SL1->L1_LOJA, "A1_DDD" ),;
699                                      POSICIONE("SA1", 1, xfilial("SA1") + SL1->L1_CLIENTE + SL1->L1_LOJA, "A1_TEL" ))
700
701              // -- Envia todas as vendas
702              oFidelityC:SendSale(      ,;
703                                      POSICIONE("SA1", 1, xfilial("SA1") + SL1->L1_CLIENTE + SL1->L1_LOJA, "A1_NOME" ),; //cBusinessUnitId
704                                      POSICIONE("SA3", 1, xfilial("SA3") + SL1->L1_VEND, "A3_NOME" ),; //cCustomerName
705                                      SL1->L1_FILIAL + SL1->L1_NUM,; //cSellerName
706                                      SL1->L1_VLRLIQ,; //cSaleId
707                                      STFGetStat("CODIGO" ) ,; //nNetSaleValue
708                                      nQtdItens,; //cPosCode
709                                      SL1->L1_DOC+SL1->L1_SERIE,; //nTotalQuantityItems
710                                      aItensFid,; //cFiscalId
711                                      aFormPgto,; //Itens
712                                      SL1->L1_CGCCLI,; //Pagamentos
713                                      oPhone,; //CGC
714                                      SL1->L1_VEND,; //Telefone
715                                      SL1->L1_LOJA ) //Vendedor
716                                  //Loja
717          EndIf
718      EndIf
719
720      If lFinish
721          //cria o arquivo no host superior (retaguarda ou central) para que o orcamento nao seja importado novamente
722          STBCtrImpOrc( cNumOrig , cPdv )
723      Else

```

```

724         // indica se a venda ficara pendente (aguardando ser finalizada)
725         If !lPendSale
726             lRet := .F.
727         EndIf
728     Endif
729
730     //Imprime o TEF
731     If ValType(lFinish) == "L" .AND. lFinish .AND. lTEF0k .AND. lEmitNFCe
732         //Resposta fiscal para o validador fiscal do MFE
733         If lMFE .And. lIntegrador .And. llJrspFisc .And. ExistFunc("STWGetIdPgto")
734             LjRspFisc(oTEF20,.F.,STWGetIdPgto())
735             STWZeraIdPgto() //Limpa variavel que armazena os Id's dos pagamentos enviados ao MFE
736         EndIf
737
738         STWPrintComp(oTEF20)
739     EndIf
740
741     //Transmite a venda para retaguarda
742     If !lPendSale .AND. nSendOn == 1
743         MsgRun(STR0019,'Aguarde...',{||STDSEndSale(cL1_SEND,cEstacao,.F.,cEmpAnt,cFilAnt)}) // "Transmitindo venda"
744     ElseIf !lPendSale .AND. nSendOn == 2
745         StartJob("STDSEndSale", GetEnvServer(), .F., cL1_SEND,cEstacao,.T.,cEmpAnt,cFilAnt)
746     EndIf
747
748     //Seta a quantidade de cartoes na venda
749     //para 0 apos a finalizacao. Usado na homologacao do TEF
750     Iif(!lPendSale, STIsetUsedCard(), Nil)
751
752     If !lPendSale
753         If ExistBlock("STFinishSale")
754             LjGrvLog( "L1_NUM: "+cL1_SEND, "Antes P.E STFinishSale | L1_SITUA: " + STDGPBasket('SL1','L1_SITUA') )
755             ExecBlock("STFinishSale",.F.,.F.,;
756                 {cL1_Filial,cL1_SEND,cDoc,cSerie}) //Filial, número do orçamento, doc. e série
757             LjGrvLog( "L1_NUM: "+cL1_SEND, "Depois P.E STFinishSale | L1_SITUA: " + STDGPBasket('SL1','L1_SITUA') )
758         EndIf
759     EndIf
760     If FindFunction("STWSetDocSerie")
761         STWSetDocSerie("", "") //Limpa os campos de Doc e Série após o uso do Ponto de Entrada STFinishSale()
762     EndIf
763
764     EndIf
765 EndIf
766
767 EndIf
768 Else
769     If ValType(oWFRceipt) == '0'
770         STFMessage(ProcName(0), "STOP", STR0022 ) //"Aguarde. Efetuando baixa do título."
771         STFShowMessage(ProcName(0))
772         If oWFRceipt:DropTitles(oMdlGrd,STIGetIsCont())
773             oWFRceipt:Print(oMdlGrd)
774             STBOpenDrawer()// Abre a gaveta
775             If lSTConfRec

```

```

776         ExecBlock("STConfRec",.F.,.F.,{oWFReceipt,aPayment})
777     EndIf
778     If ChkFile("MHJ") .AND. ChkFile("MHK")
779         STBIncMHX(oWFReceipt,oMdlGrd)
780     EndIf
781     lRet := .T.
782     STFSetTot( 'L1_VLRTOT' , 0 )
783     STISetRecTit(.F.) // Depois de finalizar, seta que nao eh mais recebimento
784 Else
785     lRet := .F.
786 EndIf
787
788 If lRet
789     STISetRecTit(.F.)
790     STFRestart()
791 EndIf
792 EndIf
793 EndIf
794 Else
795     lRet := .F.
796 EndIf
797
798 Return lRet
799
800
801 //-----
802 /*{Protheus.doc} STBDescTEF
803 Aplica desconto concedido no TEF na venda
804 @param      oMdlGrd - Grid do Model
805 @param      oTEF20 - Objeto do TEF
806 @param      aPaym   - Array de pagamentos
807 @return     lRet - Se executou corretamente
808 @obs
809 @sample
810 */
811 //-----
812 Function STBDescTEF(oMdlGrd, oTEF20, aPaym)
813
814 Local oForma      := Nil      //Objeto de forma formas de pagamento do tef
815 Local lRet        := .T.      //Variavel de retorno
816 Local nI          := 0        //Variavel de Loop
817 Local nConta      := 1        //Contador das formas de pagamento do TEF
818 Local nVlrDescTEF := 0        //Valor de desconto do TEF
819
820 Default oMdlGrd    := Nil
821 Default oTEF20     := Nil
822 Default aPaym      := {}
823
824 If ExistFunc("LjUsePayHub") .And. LjUsePayHub()
825     oForma := IIF(STIGetCard(),oTEF20:Cupom():RetornarFormas("V",IIF(oTEF20:oConfig:ISPgtoDig(),oTEF20:PgtoDigital():GetTotalizador(),oTEF20:Cartao():GetTotalizador()),
826 "C"),Nil)
827 Else

```

```

827     oForma := IIF(STIGetCard(),oTEF20:Cupom():RetornarFormas("V",oTEF20:Cartao():GetTotalizador(), "C"),Nil)
828 EndIf
829
830 For nI := 1 To oMdlGrd:Length()
831
832     oMdlGrd:GoLine(nI)
833
834     /* Se for cartao e o L4_TEF == .T. */
835     If oMdlGrd:GetValue( "L4_TEF" )
836
837         /*
838            Enquanto as formas retornadas pelo oForma não forem todas impressas, fica no Loop
839         */
840         If oForma:Count() >= nConta
841
842             // Pega os valores de desconto da venda, obs: pegar apenas 1 vez ja esta acumulado
843             nVlrDescTEF := oForma:Elements(nConta):nVlrDescTEF
844         Else
845             // Final do oForma:Count(), entao forca a saida do loop
846             Exit
847         EndIf
848
849         nConta++
850
851     EndIf
852
853     /*
854        Se teve desconto do TEF, aplica na venda antes
855        de imprimir os pagamentos
856    */
857     If nVlrDescTEF > 0
858         STWTotDisc( nVlrDescTEF , "V" , "Desc. TEF" , .T. )
859     EndIf
860
861 Next nI
862
863 Return lRet
864
865
866 //-----
867 /*{Protheus.doc} STBPrintPay
868 Envia para a impressora as formas de pagamento utilizadas na venda
869
870 @param      oMdlGrd - Grid do Model
871 @param      oTEF20 - Objeto do TEF
872 @param      aPaym   - Array de pagamentos
873 @return     lRet - Se executou corretamente
874 @obs
875 @sample
876 */
877 //-----
878 Function STBPrintPay(oMdlGrd, oTEF20, aPaym)

```

```

879
880 Local oForma      := Nil                                //Objeto de forma formas de pagamento do tef
881 Local nI          := 0                                  //Variavel de Loop
882 Local cForma      := ""                                 //Forma de pagamento
883 Local nValForma    := 0                                  //Valor da forma de pagamento
884 Local lRet         := .F.                               //Variavel de retorno
885 Local nVlTotal     := 0                                  //Variavel de total do pagamento Somado com valor de saque
886 Local lTroco       := SuperGetMv("MV_LJTROCO",, .F.)
887 Local lItemNFiscal := .F.
888 Local cDescForma   := ""
889 Local cVlrForma    := ""
890 Local aImpForma     := {}
891 Local nPosForma     := 0
892 Local lIsTef        := .F.
893 Local oTotal        := STFGetTot()                       // Totalizador
894 Local nSaldoAtual   := 0                                 // Variavel de controle para valor
895 Local nSaldoTotal   := STBRoundCurrency(oTotal:GetValue("L1_VLRTOT") * STBFactor()[1]) //Armazena o Total Fiscal da venda
896
897 Default oMdlGrd     := Nil
898 Default oTEF20      := Nil
899 Default aPaym       := {}
900
901 If STFGetCfg("lUseSAT",.F.)
902     STFMessage("STIPayment","STOP", cSiglaSat + " - Transmitindo venda" ) //"SAT - Transmitindo venda ..."
903     STFShowMessage("STIPayment")
904 Else
905     STFMessage("STIPayment","STOP", STR0010 ) //"Imprimindo/Preparando Documento Fiscal..."
906     STFShowMessage("STIPayment")
907 EndIf
908
909 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), STR0010 ) //"Imprimindo/Preparando Documento Fiscal..."
910
911 ParamType 0 Var      oMdlGrd      As Object      Default Nil
912 ParamType 2 Var      aPaym        As Array       Default {}
913
914 If ExistFunc("LjUsePayHub") .And. LjUsePayHub()
915     oForma := IIF(STIGetCard(),oTEF20:Cupom():RetornarFormas("V",IIF(oTEF20:oConfig:ISPgtoDig(),oTEF20:PgtoDigital():GetTotalizador(),oTEF20:Cartao():GetTotalizador()),
916         "C"),Nil)
917 Else
918     oForma := IIF(STIGetCard(),oTEF20:Cupom():RetornarFormas("V",oTEF20:Cartao():GetTotalizador(), "C"),Nil)
919 EndIf
920 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Quantidade de Formas de Pagamento", oMdlGrd:Length() ) //Gera LOG
921
922 //Passa a primeira vez para impressao das formas em cartao.
923 //Pois as mesmas devem ser impressas primeiro que as demais formas de pagamento
924 For nI := 1 To oMdlGrd:Length()
925
926     oMdlGrd:GoLine(nI)
927
928     /*
929     Se houver transação TEF na venda E o campo L4_TEF da parcela for True OU

```

```

930 não há transação TEF E a forma de pagamento da parcela é igual CC ou CD (caixa sem permissão para usar TEF)
931 */
932 cForma := AllTrim(oMdlGrd:GetValue( "L4_FORMA" ))
933
934 If cForma $ "CC/CD/PD/PX"
935
936     nVlTotal := oMdlGrd:GetValue( "L4_VALOR" )
937     lIsTef := oMdlGrd:GetValue( "L4_TEF" )
938
939     //Proporcionaliza por Tipo de Cupom (Fiscal e Não fiscal)
940     nValForma := nVlTotal * STBFactor()[1]
941
942     //Array de controle de formas de pagamentos CC e CD
943     If (nPosForma := Ascan(aImpForma, {|x| AllTrim(Upper(x[1])) == cForma })) == 0
944         cDescForma := Tabela("24",cForma)
945         aAdd(aImpForma, {cForma,; //1 - Tipo da forma (CC ou CD)
946                         cDescForma,; //2 - Descrição da forma
947                         nValForma,; //3 - Valor total da forma
948                         iIf(lIsTef,"1","0")}) //4 - É TEF?
949     Else
950         aImpForma[nPosForma][3] += nValForma
951     EndIf
952
953 EndIf
954
955 Next nI
956
957 //Faz a impressão das formas aglutinadas por tipo
958 For nI := 1 To Len(aImpForma)
959
960     cTef := aImpForma[nI][4]
961     cVlrForma := AllTrim(Str(STBRoundCurrency(aImpForma[nI][3]),14,2))
962     cForma := AllTrim(aImpForma[nI][2]) + "|" + cVlrForma + "|"
963
964     /*
965     Chamando o evento para impressao das formas de pagamento
966     */
967     If STWGetIsOpenReceipt() .AND. !Empty(cForma)
968
969         aRet := STFFireEvent(ProcName(0), "STPayment", {cForma, cTef, cVlrForma, Nil, Nil, Nil})
970
971         While Len(aRet) == 0 .OR. aRet[1] <> 0
972
973             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Impressora não responde." ) //Gera LOG
974
975             If MsgYesNo(STR0018) //"Impressora não responde. Deseja tentar novamente?"
976                 aRet := STFFireEvent(ProcName(0), "STPayment",{cForma, cTef, cVlrForma, Nil, Nil, Nil})
977             Else
978                 Return .F.
979                 cForma := ""
980             EndIf
981         End

```

```

982
983     If Len(aRet) > 0 .OR. aRet[1] == 0
984         cForma := ""
985         lRet := .T.
986     EndIf
987
988     ElseIf !Empty(cForma)
989         If !STFGetCfg("lUseECF") // Nao Utiliza impressora fiscal
990             lRet := .T.
991         EndIf
992     EndIf
993
994 Next nI
995
996
997 lItemNFiscal := STBExistNItemFiscal() //Verifica se tem item nao fiscal na venda.
998
999 //Formas que nao sao Cartao devem ser impressas por ultimo
1000 For nI := 1 To oMdlGrd:Length()
1001
1002     cForma := ""
1003
1004     oMdlGrd:GoLine(nI)
1005
1006     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "L4_FORMA", AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) ) //Gera LOG
1007     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "L4_VALOR", oMdlGrd:GetValue( "L4_VALOR" )) //Gera LOG
1008
1009     /*
1010     Se NAO for uma transação TEF ( caso seja uma venda com CC ou CD
1011     e o Caixa NÃO tem permissão de Usar TEF, é caracterizado uma transação NÃO TEF )
1012     */
1013     If !(AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) $ "CC/CD/PD/PX")
1014
1015         /*
1016         Verifica se será necessario ajuste no troco, ajusta troco com total para diferenças menores que 0.05 centavos
1017         */
1018
1019         If AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "CR"
1020             nSaldoAtual += STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1])
1021         Else
1022             If AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "BF"
1023                 nSaldoAtual += STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ))
1024             ElseIf AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "R$"
1025                 If lTroco .And. !lItemNFiscal
1026                     nSaldoAtual += STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1])
1027                 Else
1028                     nSaldoAtual += STBRoundCurrency(( oMdlGrd:GetValue( "L4_VALOR" ) - STBGetTroco()) * STBFactor()[1])
1029                 EndIf
1030             Else
1031                 nSaldoAtual += STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1])
1032             EndIf
1033         EndIf

```

```

1034
1035     If nI == oMdlGrd:Length()
1036         If nSaldoAtual > nSaldoTotal
1037             nSaldoAtual := Abs(nSaldoTotal - nSaldoAtual )
1038             If nSaldoAtual < 0.05
1039                 oMdlGrd:LoadValue( "L4_VALOR", oMdlGrd:GetValue( "L4_VALOR" ) - nSaldoAtual )
1040             EndIf
1041         EndIf
1042         If nSaldoAtual < nSaldoTotal
1043             nSaldoAtual := Abs(nSaldoTotal - nSaldoAtual )
1044             If nSaldoAtual < 0.05
1045                 oMdlGrd:LoadValue( "L4_VALOR", oMdlGrd:GetValue( "L4_VALOR" ) + nSaldoAtual )
1046             EndIf
1047         EndIf
1048     EndIf
1049
1050     /*
1051     Imprime as formas que não fazem parte do TEF
1052     */
1053     If AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "CR"
1054         // NCCs devem ser tratadas de forma diferente pois não faz parte das formas de pagamento, portanto não está em aPaym.
1055         cForma := "CREDITO|" + AllTrim(Str(STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]),14,2)) + "|"
1056     Else
1057         If AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "BF"
1058             cForma := "BONIFICACAO|" + AllTrim(Str(STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" )),14,2)) + "|"
1059         ElseIf AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "DC"
1060             cForma := STBSX5Dsc("DC") + "|" + AllTrim(Str(STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" )),14,2)) + "|"
1061         ElseIf AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "R$"
1062             //Tratamento para exibicao de troco em dinheiro
1063             If lTroco .And. !lItemNFiscal
1064                 cForma := AllTrim(aPaym[Ascan(aPaym,{|x| x[1] == AllTrim(oMdlGrd:GetValue( "L4_FORMA" ))}][3]) ;
1065                     + "|" + AllTrim(Str(STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]),14,2)) + "|"
1066             Else
1067                 cForma := AllTrim(aPaym[Ascan(aPaym,{|x| x[1] == AllTrim(oMdlGrd:GetValue( "L4_FORMA" ))}][3]) ;
1068                     + "|" + AllTrim(Str(STBRoundCurrency((oMdlGrd:GetValue( "L4_VALOR" ) - STBGetTroco()) * STBFactor()[1]),14,2)) + "|"
1069             EndIf
1070         Else
1071             cForma := AllTrim(aPaym[Ascan(aPaym,{|x| x[1] == AllTrim(oMdlGrd:GetValue( "L4_FORMA" ))}][3]) ;
1072                 + "|" + AllTrim(Str(STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]),14,2)) + "|"
1073         EndIf
1074     EndIf
1075
1076     //Tratamento dinheiro
1077     If AllTrim(oMdlGrd:GetValue( "L4_FORMA" )) == "R$"
1078         //Exibe Troco
1079         If lTroco .And. !lItemNFiscal
1080             nValForma := oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]
1081         Else
1082             nValForma := (oMdlGrd:GetValue( "L4_VALOR" ) - STBGetTroco()) * STBFactor()[1]
1083         EndIf
1084     Else
1085

```

```

1086         nValForma := oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]
1087     EndIf
1088
1089 EndIf
1090
1091 /*
1092     Chamando o evento para impressao das formas de pagamento
1093 */
1094 If STWGetIsOpenReceipt() .AND. !Empty(cForma) .AND. STBRoundCurrency(oMdlGrd:GetValue( "L4_VALOR" ) * STBFactor()[1]) >= 0.01
1095
1096     aRet := STFFireEvent(ProcName(0), "STPayment", {cForma, IIF(oMdlGrd:GetValue("L4_TEF"),"1","0"), AllTrim(Str(nValForma,14,2)), Nil, Nil, Nil})
1097
1098     While Len(aRet) == 0 .OR. aRet[1] <> 0
1099
1100         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Impressora não responde" ) //Gera LOG
1101
1102         If MsgYesNo(STR0018) // "Impressora não responde. Deseja tentar novamente?"
1103             aRet := STFFireEvent(ProcName(0), "STPayment", {cForma, IIF(oMdlGrd:GetValue("L4_TEF"),"1","0"), AllTrim(Str(nValForma,14,2)), Nil, Nil, Nil})
1104         Else
1105             Return .F.
1106             cForma := ""
1107         EndIf
1108
1109     End
1110
1111     If Len(aRet) > 0 .OR. aRet[1] == 0
1112         lRet := .T.
1113         cForma := ""
1114     EndIf
1115 ElseIf !STWGetIsOpenReceipt()
1116     //Para impressoes nao fiscais
1117     lRet := .T.
1118 EndIf
1119
1120 Next nI
1121
1122 Return lRet
1123
1124
1125 //-----
1126 /*{Protheus.doc} STBCloseCup
1127 Totalizando e fechando o cupom fiscal
1128
1129 @param1     oTEF20 - Objeto TEF
1130 @param2     oTotal - Objeto TOTAL
1131 @return     lRet - Se executou corretamente
1132 @obs
1133 @sample
1134 */
1135 //-----
1136 Function STBCloseCup(oTEF20, oTotal)
1137

```

```

1138 Local cReturn      := ""          //Retorno do comprovante TEF
1139 Local lRet          := .F.         //Retorno da funcao
1140 Local nOpt          := 2           //Opção selecionada pelo usuário
1141 Local lTentativa    := .T.         //controle de tentativa de impressao
1142 Local cEOF          := CHR(10)+CHR(13)
1143 Local cMsgCupom     := ""
1144 Local cNumDAV       := ""
1145 Local nVlrTotal     := 0
1146 Local nDecimais     := MsDecimais(STBGetCurrency())
1147
1148 Local nTotICMS       := 0
1149 Local nTotISS        := 0
1150 Local cAuxMsgCup     := ""
1151 Local lEmitNfce      := Iif(ExistFunc("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
1152
1153 Default oTEF20       := Nil
1154 Default oTotal       := Nil
1155
1156
1157 cNumDAV := STDGPBasket("SL1","L1_NUMORC")
1158 /*
1159 HOMOLOGACAO PAF-ECF 2013
1160 a Ordem das Mensagens Impressas deve ser:
1161 1 - MD5 ; 2 - Msg do TPL PCL ou DAV/PRE-VENDA XXXXX ; 3 - Msgs dos Estados (Minas Legal, etc.)
1162 4 - mensagens adicionais
1163 */
1164 If STBIsPAF() .AND. !Empty(cNumDAV) //L1_NUMORC
1165     If SuperGetMv("MV_LJPRVEN",,.F.)
1166         cMsgCupom := "PV"+AllTrim(cNumDAV)+CRLF
1167     Else
1168         cMsgCupom := "DAV"+AllTrim(cNumDAV)+CRLF
1169     EndIf
1170 EndIf
1171
1172 nVlrTotal := STDGPBasket("SL1","L1_VLRTOT")
1173
1174 If !lEmitNfce // Se estiver com NFCE não executa legislações referente a cupom fiscal.
1175
1176     //Verifica se deve imprimir mensagem "MINAS LEGAL" de Minas Gerais
1177     cMsgCupom += Lj950MinasL(nVlrTotal,nDecimais) //L1_VLRTOT - nao precisa alterar
1178
1179     //Verifica se deve imprimir mensagem "PARAIBA LEGAL" da Paraíba
1180     cMsgCupom += Lj950PBLeg(nVlrTotal,nDecimais,STDGPBasket("SL1","L1_DOC")) //L1_VLRTOT Lj950PBLeg(nVlrTotal, nDecimais , cCooCup) //lpos e objeto de cliente
1181 EndIf
1182
1183 //Verifica se deve imprimir mensagem "Nota Legal" do Distrito Federal para evitar looping desnecessario no aItens
1184 If LJAnalisaLeg(62)[1]
1185     //acumula valor de iss dos produtos
1186     nTotICMS := oTotal:GetValue("L1_VALICM")
1187
1188     nTotISS := oTotal:GetValue("L1_VALISS")
1189

```

```

1190     cMsgCupom  += Lj950NotaL( nTotICMS , nTotISS , nDecimais, STDGPBasket("SL1","L1_CGCCLI") ) //sem parametros
1191 EndIf
1192
1193 cAuxMsgCup := AllTrim(SuperGetMV("MV_LJFISMS",,""))
1194
1195 If Substr(cAuxMsgCup,1,1)=="&"
1196     cAuxMsgCup := &( Substr(cAuxMsgCup,2,Len(cAuxMsgCup)) )
1197 EndIf
1198
1199 If Empty(AllTrim(cAuxMsgCup))
1200     cAuxMsgCup := STFGetStation("MSGCUP") + cEOF
1201 Else
1202     cAuxMsgCup += cEOF
1203 EndIf
1204
1205 cMsgCupom  += cAuxMsgCup
1206
1207 // Impressao da mensagem referente ao FECF
1208 If ExistFunc("Lj950FECF")
1209     nValFECF := STBTaxRet( Nil , "NF_VALFECF")
1210     cAuxMsgCup := AllTrim(Lj950FECF(nValFECF))
1211     If !Empty(cAuxMsgCup)
1212         cMsgCupom += cAuxMsgCup + cEOF
1213     EndIf
1214 EndIf
1215
1216 /*
1217 Para Homologacao TEF eh preciso esperar um tempo
1218 para enviar o fechamento do cupom apos o envio do pagamento
1219 pois eles fazem um teste de desligamento, e se nao esperar ele
1220 confirma que fechou o cupom mesmo sem ter impresso o fechamento
1221 */
1222 If SuperGetMV("MV_LJHMTF", ,.F.)
1223     Sleep(5000)
1224 EndIf
1225
1226 If STIGetCard()
1227
1228     If ExistFunc("LjUsePayHub") .And. LjUsePayHub()
1229         cReturn := oTEF20:Cupom():CupomReduzido(.T., cMsgCupom + STR0003, "V" + IIF(oTEF20:oConfig:ISPgtoDig(),oTEF20:PgtoDigital():GetTotalizador(),oTEF20:Cartao():GetTotali
zador()) + "C") //"Obrigado e volte sempre"
1230     Else
1231         cReturn := oTEF20:Cupom():CupomReduzido(.T., cMsgCupom + STR0003, "V" + oTEF20:Cartao():GetTotalizador() + "C") //"Obrigado e volte sempre"
1232     EndIf
1233
1234     If Empty(cReturn)
1235         cReturn := cMsgCupom
1236     EndIf
1237 Else
1238     cReturn := cMsgCupom
1239 EndIf
1240

```

```

1241 If STWGetIsOpenReceipt()
1242
1243     IF (ExistFunc("Lj950ImpCpf") .And. Lj950ImpCpf(nVlrTotal)) .Or. (ExistFunc("LjInfDocCli") .And. LjInfDocCli() > 1)
1244         lTentaTiva := .T.
1245         cAuxMsgCup := "" //No modelo bematech a número do CPF é no comando de fechamento portanto mando essa variável
1246         STWIdCliente@lTentaTiva,@cAuxMsgCup)
1247
1248         If !Empty(AllTrim(cAuxMsgCup))
1249             cReturn := cAuxMsgCup + cReturn //Já vem com a quebra de linha, por isso somente concatena
1250         EndIf
1251
1252         lTentaTiva := .T. //Retorna o conteudo da variavel para prosseguimento do programa
1253     EndIf
1254
1255     aRet := STFFireEvent(ProcName(0), "STCloseReceipt", {cReturn,;
1256         (oTotal:GetValue("L1_VLRTOT") + oTotal:GetValue("L1_DESCONT")) - oTotal:GetVaLue("L1_NOTFISCAL"), Nil})
1257
1258     If Valtype(aRet) == "A" .AND. Len(aRet) > 0 .AND. aRet[1] <> 0 //Erro na impressão
1259
1260         While lTentaTiva
1261
1262             STFMessage("TEFImprime", "YESNO", STR0006 + " " + STR0007) //"Impressora não responde.##"Deseja imprimir novamente?"
1263             nOpt := If(STFShowMessage("TEFImprime"),2,0)
1264
1265             //2=SIM
1266             If nOpt == 2
1267
1268                 Sleep(5000)
1269                 aRet := STFFireEvent(ProcName(0), "STCloseReceipt", {cReturn,;
1270                     (oTotal:GetValue("L1_VLRTOT") + oTotal:GetValue("L1_DESCONT")) - oTotal:GetVaLue("L1_NOTFISCAL"), Nil})
1271
1272                 If Valtype(aRet) == "A" .AND. Len(aRet) > 0 .AND. aRet[1] <> 0 //Erro na impressão
1273                     lTentaTiva := .T.
1274                     lRet := .F.
1275                 Else
1276                     lTentaTiva := .F.
1277                     lRet := .T.
1278                 EndIf
1279             Else
1280                 lTentaTiva := .F.
1281                 lRet := .F.
1282
1283                 /*/
1284                 Desfazimento Transação TEF
1285                 /*/
1286                 oTEF20 := STBGetTef()
1287                 If ValType(oTEF20) == '0'
1288                     oTEF20:Desfazer()
1289                     STFMessage("TEF", "POPUP",STR0008)//"Transação não foi efetuada. Favor reter o cupom."
1290                     STFShowMessage( "TEF")
1291                 EndIf

```

```

1293
1294         EndIf
1295
1296         EndDo
1297
1298
1299     Else
1300         lRet := .T.
1301     EndIf
1302
1303     //se fechou o cupom
1304     If lRet
1305         STWSetIsOpenReceipt( .F. )
1306     EndIf
1307
1308 Else
1309
1310     lRet := .T.
1311
1312 EndIf
1313
1314 Return lRet
1315
1316
1317 //-----
1318 /*{Protheus.doc} STBCalcSald
1319 Saldo restante do pagamento
1320
1321 @param1      cType - Tipo de pagamento
1322 @param2      nValue - Valor do pagamento
1323 @return      uRet - Saldo restante do pagamento
1324 @obs
1325 @sample
1326 */
1327 //-----
1328 Function STBCalcSald(cType, nValue,aResPay)
1329 Local uRet := Nil          //Total do saldo
1330 Local oTotal := STFGetTot() //Recebe o objeto do model para recuperar os valores
1331 Local nTotal := 0          //Acumulador
1332
1333 Default cType      := ""
1334 Default nValue     := 0
1335 Default aResPay    := {}
1336
1337 ParamType 0 Var      cType      As Character      Default      ""
1338 ParamType 1 Var      nValue     As Numeric        Default      0
1339 If nVLBF <> 0
1340     nTotal := nVLBF
1341 Else
1342     nTotal := STIGetTotal()
1343 Endif
1344

```

```

1345 If nTotal < 0 //Bonificacao
1346     nTotal:= -1 * nTotal
1347 Endif
1348
1349 Do Case
1350     Case cType == "1"
1351         uRet := IIF((oTotal:GetValue("L1_VLRTOT") - STIGetTotal()) <= 0, 0, oTotal:GetValue("L1_VLRTOT") - STIGetTotal())
1352     Case cType == "2"
1353         uRet := ( (STIGetTotal() + nValue) >= oTotal:GetValue("L1_VLRTOT") )
1354 EndCase
1355
1356 Return uRet
1357
1358
1359 //-----
1360 /*{Protheus.doc} STBUpdPaym
1361 Atualiza o model com todas as parcelas a serem gravadas na SL4
1362
1363 @param      oMdl - Model das parcelas do pagamento
1364 @return     lRet - Se executou corretamente
1365 @obs
1366 @sample
1367 */
1368 //-----
1369 Function STBUpdPaym(oMdl)
1370
1371 Local oMdlGrd      := oMdl:GetModel("PARCELAS")           //Seta o model do grid
1372 Local oMdlPaym     := oMdl:GetModel("APAYMENTS")         //Seta o model do grid
1373 Local aFieldsGrd   := oMdlGrd:GetStruct():GetFields()     //Estrutura do model
1374 Local nJ           := 0                                  //Variavel de loop
1375 Local nPosIni      := 1
1376 Local lTemDin      := .F.                                //Ja existe dinheiro nos pagamentos?
1377 Local cForma       := ""
1378
1379 Default oMdl       := Nil
1380
1381 ParamType 0 Var     oMdl      As Object      Default Nil
1382
1383     cForma := AllTrim(oMdlGrd:GetValue('L4_FORMA')) //Forma de pagamento
1384
1385     /* Tratamento para somar vários pagamentos em dinheiro */
1386     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "01 Forma de Pagamento." + cForma+"." )
1387     If cForma == 'R$'
1388         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "02 Forma de Pagamento." + oMdlGrd:GetValue('L4_FORMA')+ "." )
1389         //O metodo SeekLine substitui o aScan no acols (que nao esta sendo mais utilizado para a versao 12)
1390         //Se retornar .T. é que foi encontrado a busca e automaticamente ja esta sendo posicionada na linha desejada.
1391         If oMdlPaym:SeekLine({"L4_FORMA", 'R$'})
1392             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Pagamento em R$" ) //Gera LOG
1393             lTemDin := .T.
1394         Else
1395             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Não tem dinheiro lTemDin" )
1396         Endif

```

```

1397 Endif
1398
1399 //So adiciona o registro caso o primeiro esteja vazio, para nao duplicar o objeto
1400 If !(lTemDin .Or. (oMdlPaym:Length() == 1 .And. Empty(oMdlPaym:GetValue("L4_FORMA")) .And. Empty(oMdlPaym:GetValue("L4_VALOR"))))
1401     oMdlPaym:AddLine(.T.)
1402 EndIf
1403
1404 If !(cForma $ 'CR')
1405     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "L4_FORMA != CR" ) //Gera LOG
1406
1407     If !IsMoney(cForma) .AND. !(cForma $ 'CC|CD|CH|FI|VA|VP|CO|FID')
1408         //Entra nesse IF quando o cliente customiza alguma forma de pagamento na SX5 tabela 24
1409         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Chama STBGerParc" ) //Gera LOG
1410         oMdlPaym := STBGerParc(oMdlGrd, oMdlPaym, aFieldsGrd, nPosIni)
1411     Else
1412         If !(cForma $ 'CC|CD|CH|FI|CO') .and. !(ProcName(2) == 'STIFICONFPAY')
1413
1414             For nJ := 1 To Len(aFieldsGrd)
1415                 oMdlPaym:LoadValue(aFieldsGrd[nJ][3], oMdlGrd:GetValue(aFieldsGrd[nJ][3]))
1416             Next nJ
1417
1418             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Formas de pgto nao consideradas como entrada:", cMvEntExce )
1419             If !Empty(cForma) .And. oMdlGrd:GetValue("L4_DATA") == dDataBase .And. !(" "+cForma+" " $ cMvEntExce)
1420                 nEntrada += oMdlGrd:GetValue("L4_VALOR")
1421             EndIf
1422
1423         Else
1424             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Chama STBGerParc" ) //Gera LOG
1425             oMdlPaym := STBGerParc(oMdlGrd, oMdlPaym, aFieldsGrd, nPosIni)
1426         EndIf
1427     EndIf
1428
1429 EndIf
1430
1431 Return .T.
1432
1433
1434 //-----
1435 /*{Protheus.doc} STBGerParc
1436 Geracao das parcelas para os tipos CD,CC e CH
1437
1438 @param
1439 @return     oMdlPaym - Model de Pagamentos
1440 @obs
1441 @sample
1442 */
1443 //-----
1444 Function STBGerParc( oMdlGrd, oMdlPaym, aFieldsGrd , nPos )
1445
1446 Local nI           := 0                //Variavel de loop
1447 Local nJ           := 0                //Variavel de loop
1448 Local nIntervalo   := SuperGetMV("MV_LJINTER") //Intervalo das parcelas

```

```

1449 Local nDiferenca      := 0                      //Diferenca
1450 Local aParcelas      := STBGetParc()           //Recebe as parcelas da condicao de pagamento
1451 Local nTotParc       := Len(aParcelas)          //Total de parcelas
1452 Local aRetCheck      := STWGetCkRet()           //Dados do cheque
1453 Local lFormaImp      := ExistFunc("STBGFormImp") .And. STBIsImpOrc()
1454 Local aFormasImp     := {}
1455 Local nPosImp        := 0
1456 Local dData          := Nil
1457 Local nValor         := 0
1458 Local lGFIAltImp     := ExistFunc("STIGFiAltImp")
1459 Local lVldForm       := .F. //Valida as formas de pagamento
1460 Local nTotAcrCar     := 0 //Valor de Acréscimo do cartão (Total)
1461 Local nVrAcrCart     := 0 //Valor de Acréscimo do cartão (Por parcela)
1462 Local nDifAcrCar     := 0 //Diferença de Valor de Acrescimo do Cartao
1463
1464 Default oMdlGrd       := Nil
1465 Default oMdlPaym      := Nil
1466 Default aFieldsGrd   := {}
1467 Default nPos         := 0
1468
1469 ParamType 0 Var      oMdlGrd      As Object      Default Nil
1470 ParamType 1 Var      oMdlPaym     As Object      Default Nil
1471 ParamType 2 Var      aFieldsGrd   As Array       Default {}
1472 ParamType 3 Var      nPos         As Numeric     Default 0
1473
1474
1475 If AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH'
1476     nContCheque++
1477 EndIf
1478
1479 If nTotParc > 0
1480     /* Lancar as parcelas para cartao e cheque */
1481     For nI := 1 To nTotParc
1482
1483         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Total de Parcelas (nTotParc): " + STR(nTotParc) ) //Gera LOG
1484         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Parcela: " + Str(nI) + " de " + STR(nTotParc) ) //Gera LOG
1485
1486         IIF(nPos == 1 .AND. nI == 1, Nil, oMdlPaym:AddLine(.T.))
1487
1488         For nJ := 1 To Len(aFieldsGrd)
1489
1490             If AllTrim(aFieldsGrd[nJ][3]) == "L4_DATA"
1491
1492                 oMdlPaym:LoadValue("L4_DATA", aParcelas[nI][1] )
1493
1494             ElseIf AllTrim(aFieldsGrd[nJ][3]) == "L4_VALOR"
1495
1496                 oMdlPaym:LoadValue("L4_VALOR", aParcelas[nI][2] )
1497
1498             ELSEIF ValType(aRetCheck) == 'A' .AND. Len(aRetCheck) > 0 .AND. AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. PadR(aFieldsGrd[nJ][3], 10) $
1499                 "L4_ADMINIS|L4_NUMCART|L4_AGENCIA|L4_CONTA |L4_RG |L4_TELEFON|L4_TERCEIR|L4_NOMECLI"

```

```

1500         STBStDadosCH(oMdlPaym, aFieldsGrd, aRetCheck, nContCheque, nJ, nI)
1501
1502     Else
1503
1504         oMdlPaym:LoadValue(aFieldsGrd[nJ][3], oMdlGrd:GetValue(aFieldsGrd[nJ][3]))
1505
1506     EndIf
1507
1508 Next nJ
1509
1510 Next nI
1511
1512 Else
1513
1514     nDiferenca := STBRound( oMdlGrd:GetValue("L4_VALOR") - (( STBRound( oMdlGrd:GetValue("L4_VALOR") / oMdlGrd:GetValue("L4_PARC"),2) ) * oMdlGrd:GetValue("L4_PARC")), 2)
1515     If oMdlGrd:HasField('L4_ACRCART')
1516         nTotAcrCar := oMdlGrd:GetValue('L4_ACRCART') //Valor Total do Acrescimo do Cartão
1517         nDifAcrCar := nTotAcrCar
1518     Else
1519         nTotAcrCar := 0
1520         nDifAcrCar := 0
1521     EndIf
1522
1523     If lFormaImp
1524         lVldForm := STBVldForm(oMdlGrd,STBGFormImp())
1525     EndIf
1526
1527     For nI := 1 To oMdlGrd:GetValue("L4_PARC")
1528
1529         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "(ELSE) Total de Parcelas (nTotParc): " + STR(oMdlGrd:GetValue("L4_PARC")) ) //Gera LOG
1530         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "(ELSE) Parcela: " + Str(nI) + " de " + STR(oMdlGrd:GetValue("L4_PARC")) ) //Gera LOG
1531
1532         IIF(nPos == 1 .AND. nI == 1, Nil, oMdlPaym:AddLine(.T.))
1533
1534         nValor      := 0
1535         nVrAcrCart  := 0
1536         dData       := cToD(" / / ")
1537         If lFormaImp .And. lVldForm
1538             aFormasImp := STBGFormImp()
1539             If Len(aFormasImp) > 0 .AND. ( lGFIAltImp .AND. !STIGFiAltImp() )
1540                 If (nPosImp := aScan(aFormasImp, {|x| AllTrim(x[1])=AllTrim(oMdlGrd:GetValue('L4_FORMA'))} .And. x[4]=nI .And. (oMdlGrd:GetValue('L4_FORMAID')=x[5] .Or.
AllTrim(x[5])='') }))) > 0
1541                     nValor := aFormasImp[nPosImp][2] //Valor do cheque
1542                     dData   := SToD(aFormasImp[nPosImp][3]) //Vencimento do cheque
1543                 EndIf
1544             EndIf
1545         Endif
1546
1547     For nJ := 1 To Len(aFieldsGrd)
1548
1549         Do Case
1550             Case AllTrim(aFieldsGrd[nJ][3]) == "L4_DATA"

```

```

1551         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Forma" + oMdlGrd:GetValue('L4_FORMA') ) //Gera LOG
1552
1553         If AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. ValType(aRetCheck[1][1][1]) == 'A' .AND. Len(aRetCheck) > 0 .AND.
1554         Len(aRetCheck[nContCheque][nI][1]) >= 11 .AND. aRetCheck[nContCheque][nI][1][10] <> Nil
1555             oMdlPaym:LoadValue("L4_DATA",aRetCheck[nContCheque][nI][1][10])
1556         ElseIf AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. ValType(aRetCheck) == 'A' .AND. Len(aRetCheck) > 0 .AND. Len(aRetCheck[1][1]) >= 11
1557             oMdlPaym:LoadValue("L4_DATA",aRetCheck[nI][1][10])
1558         Else
1559             If Empty(dData)
1560                 dData := oMdlGrd:GetValue(aFieldsGrd[nJ][3]) + IIf(nI = 1, 0, nIntervalo*(nI-1))
1561             EndIf
1562             oMdlPaym:LoadValue("L4_DATA", dData )
1563         EndIf
1564
1565         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_VALOR"
1566             If AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. ValType(aRetCheck[1][1][1]) == 'A' .AND. Len(aRetCheck) > 0 .AND.
1567             Len(aRetCheck[nContCheque][nI][1]) >= 11
1568                 oMdlPaym:LoadValue("L4_VALOR", aRetCheck[nContCheque][nI][1][11] )
1569             ElseIf AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. ValType(aRetCheck) == 'A' .AND. Len(aRetCheck) > 0 .AND. Len(aRetCheck[1][1]) >= 11
1570                 oMdlPaym:LoadValue("L4_VALOR", aRetCheck[nI][1][11] )
1571             Else
1572                 If nValor == 0
1573                     nValor := STBRound( oMdlGrd:GetValue(aFieldsGrd[nJ][3]) / oMdlGrd:GetValue('L4_PARC'), 2 )
1574                 EndIf
1575                 oMdlPaym:LoadValue("L4_VALOR", nValor )
1576
1577                 If nI == oMdlGrd:GetValue('L4_PARC') .AND. !lVldForm
1578                     oMdlPaym:LoadValue("L4_VALOR", STBRound( oMdlPaym:GetValue("L4_VALOR") + nDiferenca, 2 ))
1579                 EndIf
1580             Endif
1581
1582             If oMdlPaym:GetValue("L4_DATA") == dDataBase .AND. !(("|" + Alltrim(oMdlGrd:GetValue('L4_FORMA'))+"|") $ cMvEntExce)
1583                 nEntrada += oMdlPaym:GetValue("L4_VALOR")
1584             EndIf
1585
1586         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_ACRSFIN"
1587
1588             oMdlPaym:LoadValue("L4_ACRSFIN", STBRound( oMdlGrd:GetValue(aFieldsGrd[nJ][3]) / oMdlGrd:GetValue('L4_PARC'), 2 ) )
1589
1590         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_ACR CART"
1591
1592             If nTotAcrCar > 0
1593                 nVrAcrCart := STBRound( nTotAcrCar / oMdlGrd:GetValue('L4_PARC'), 2 )
1594                 nDifAcrCar -= nVrAcrCart
1595                 oMdlPaym:LoadValue("L4_ACR CART", nVrAcrCart )
1596
1597                 If nI == oMdlGrd:GetValue('L4_PARC') .AND. !lVldForm
1598                     oMdlPaym:LoadValue("L4_ACR CART", oMdlPaym:GetValue("L4_ACR CART") + nDifAcrCar)
1599                 EndIf
1600             EndIf
1601
1602         Case ValType(aRetCheck) == 'A' .AND. Len(aRetCheck) > 0 .AND. AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'CH' .AND. PadR(aFieldsGrd[nJ][3], 10) $

```

```

        "L4_ADMINIS|L4_NUMCART|L4_AGENCIA|L4_CONTA   |L4_RG       |L4_TELEFON|L4_TERCEIR|L4_NOMECLI"
1601
1602                STBStDadosCH(oMdlPaym, aFieldsGrd, aRetCheck, nContCheque, nJ, nI)
1603
1604                Otherwise
1605
1606                        oMdlPaym:LoadValue(aFieldsGrd[nJ][3], oMdlGrd:GetValue(aFieldsGrd[nJ][3]))
1607
1608                EndCase
1609
1610                Next nJ
1611
1612                Next nI
1613
1614                aRetCheck := {}
1615
1616        EndIf
1617
1618        //Volta ao estado original da variavel que indica se houve alteração em alguma informação do orçamento do tipo FI
1619        If ExistFunc("STISFiAltImp")
1620                STISFiAltImp()
1621        EndIf
1622
1623        Return oMdlPaym
1624
1625
1626        //-----
1627        /*{Protheus.doc} STBGetEnt
1628        Retorna o valor de entrada
1629
1630        @param
1631        @return      nEntrada - Valor de entrada
1632        @obs
1633        @sample
1634        */
1635        //-----
1636        Function STBGetEnt()
1637        Return nEntrada
1638
1639
1640        //-----
1641        /*{Protheus.doc} STBSetEnt
1642        Set na variavel nEntrada
1643
1644        @param
1645        @return      lRet - Se executou corretamente
1646        @obs
1647        @sample
1648        */
1649        //-----
1650        Function STBSetEnt()
1651        nEntrada := 0

```

```

1652 Return .T.
1653
1654
1655 //-----
1656 /*{Protheus.doc} STBOpenDrawer
1657 Abre a gaveta
1658
1659 @param
1660 @return
1661 @obs
1662 @sample
1663 */
1664 //-----
1665 Function STBOpenDrawer()
1666
1667 Local aRet := {}
1668
1669 aRet := STFFireEvent( ProcName(0), "STOpDrawer", {} )
1670
1671 If Len(aRet) == 0 .OR. aRet[1] <> 0
1672
1673     STFMessage("STOpDrawer ", "STOP", STR0004)//"Erro ao abrir gaveta"
1674     STFShowMessage("STOpDrawer ")
1675
1676 EndIf
1677
1678 Return .T.
1679
1680 //-----
1681 /*{Protheus.doc} STBUdpResPg
1682 Atualiza o list box do resumo de pagamento
1683
1684 @param      oListTpPaym - Envio o objeto para que seja possivel bloquear as ações dele ate que seja concluido ou cancelado a forma de pagamento atual.
1685 @return
1686 @obs
1687 @sample
1688 */
1689 //-----
1690 Function STBUdpResPg(nAjustCols, lFirstPay,nVLBonif,oListTpPaym)
1691
1692 Local oMdl           := STISetMdlPay()           //Get no objeto oModel
1693 Local oModel         := oMdl:GetModel('PARCELAS') //Get no model parcelas
1694 Local nX             := 0                       //Variavel de loop
1695 Local aRet           := {}                      //Variavel de retorno
1696 Local oMdlCdPg       := Nil                     //Model da condicao de pagamento
1697 Local oCondPgModel   := Nil                     //Recebe o set do model CDPGDETAILS
1698 Local nTotalNCCs     := STDGetNCCs("2")         //Valor total das NCCs utilizada na venda
1699 Local aOrc           := {}                      //Orcamento
1700 Local nI             := 0                       //Variavel de loop
1701 Local lPayImport     := STIGetPayImp()           //Variavel que controla para nao passar duas vezes pelo pagamento no orcamento
1702 Local aCallAdm       := {}                      //Armazena formas de pagamento a serem chamadas interface para informar Adm. Fin.
1703 Local nContCard      := 0                       //Contador

```

```

1704 Local cFormPgto      := ""                                //Armazena a forma de pagamento de acordo com o ValType do a0rc
1705 Local oTotal         := STFGetTot()                       //Total da venda
1706 Local lZeraPayImport := IIf(ExistFunc("STiGetZrPg"),STiGetZrPg(),.F.)//Variavel de controle que indica se considerará ou não os pagamentos do orçamento
1707 Local lSTISetPgOr     := ExistFunc("STISetPgOr")
1708 Local cEspaco         := ""                                //Variavel criada para dar espaco entre os intems do painel de bagamento no webapp
1709
1710 Default nAjustCols    := 0
1711 Default lFirstPay     := .F.
1712 Default nVLBonif     := 0
1713 Default oListTpPaym   := Nil                                //Objeto da lista de formas de pagamento
1714
1715
1716 If Empty(nVLBF)
1717     nVLBF := nVLBonif
1718 Endif
1719
1720 If Empty( STDGPBasket( "SL1" , "L1_NUMORIG" ) )
1721     If lFirstPay .AND. nTotalNCCs == 0 .AND. nVLBF == 0
1722         //Reseta a variável static sCondicao
1723         If FindFunction("STBRstCdPg")
1724             STBRstCdPg()
1725         Endif
1726         /* Criacao do model da cond de pagamento */
1727         ModelCdPg()
1728
1729         /* Add as Cond pgto no model */
1730         STIAddCPg( Nil , .T. )
1731
1732         /* Marca no model a condicao de pagamento a ser considerada */
1733         STISelectCP()
1734
1735         /* Chama o workflow para lancar a condicao de pagamento */
1736         oMdlCdPg      := STIGetMdlCpPg()
1737         oCondPgModel   := oMdlCdPg:GetModel("CDPGDETAILS")
1738         STWPgCdPg(oCondPgModel)
1739     EndIf
1740 Else
1741     If !lPayImport .AND. nVLBF == 0
1742
1743         If lSTISetPgOr
1744             STISetPgOr ( ,.T.)
1745         Endif
1746
1747         STISetPayImp(.T.) //Apenas alimenta a variavel lPayImport como .T.
1748
1749         If oTotal:GetValue( "L1_VLRTOT" ) >= nTotalNCCs .AND. !lZeraPayImport
1750
1751             If Valtype(oListTpPaym) == "0"
1752                 oListTpPaym:BWHEN := {|| .F. }
1753             Endif
1754
1755             a0rc := STBFormat0rcs()

```

```

1756
1757     For nI := 1 To Len(a0rc)
1758
1759         If Valtype(a0rc[nI][1]) == "A"
1760             cFormPgto := a0rc[nI][1][3]
1761         Else
1762             cFormPgto := a0rc[nI][3]
1763         EndIf
1764
1765         If cFormPgto <> Nil
1766
1767             If lSTISetPgOr
1768                 STISetPgOr(cFormPgto)
1769             Endif
1770
1771             Do Case
1772                 Case IsMoney(cFormPgto)
1773                     STICS0rc( STOD(a0rc[nI][2]) ,; //Data
1774                             a0rc[nI][1] ) //Valor
1775
1776
1777                 Case cFormPgto == 'CH'
1778                     STICheck0rc( STOD(a0rc[nI][2]) ,; //Data
1779                                a0rc[nI][1] ,; //Valor
1780                                a0rc[nI][4] ) //Parcelas
1781
1782
1783                 Case cFormPgto $ 'CC|CD'
1784                     /*
1785                     retiramos o trecho que faz a chamada da tela do TEF, pois ao importar um orçamento
1786                     que possuía TEF com Clisitef, a tela do TEF já era chamada, não permitindo que o Caixa
1787                     editasse o orçamento, ao menos que ele cancelasse a tela do TEF manualmente
1788                     */
1789                     If Valtype(a0rc[nI][1]) == "A"
1790                         For nContCard := 1 To Len(a0rc[nI])
1791                             AADD( aCallADM , { STOD(a0rc[nI][nContCard][2]) , a0rc[nI][nContCard][1] , a0rc[nI][nContCard][4] , '' , a0rc[nI][nContCard][3] , .F. } )
1792                         //Preenche o aCallADM para importação de orçamento com mais de um Id de Cartão
1793                         Next nContCard
1794                     Else
1795                         AADD( aCallADM , { STOD(a0rc[nI][2]) , a0rc[nI][1] , a0rc[nI][4] , '' , a0rc[nI][3] , .F. } ) //Chamar preencher ADM para importação
1796                     EndIf
1797
1798                     STISetaCallADM(aCallADM)
1799
1800                 Case cFormPgto == 'FI'
1801                     AADD( aCallADM , { STOD(a0rc[nI][2]) , a0rc[nI][1] , a0rc[nI][4] , '' , a0rc[nI][3] , .F. } ) //Chamar preencher ADM para importação
1802                     STISetaCallADM(aCallADM)
1803                 Otherwise
1804                     AADD( aCallADM , { STOD(a0rc[nI][2]) , a0rc[nI][1] , a0rc[nI][4] , '' , a0rc[nI][3] , .F. } ) //Chamar preencher ADM para importação
1805                     STISetaCallADM(aCallADM)
1806             EndCase

```

```

1807
1808
1809         EndIf
1810
1811     Next nI
1812
1813     // Função presente no Fonte STFTotalUpdate, responsavel por realizar o Back-Up de valores dos totalizadores do Rodapé
1814     If ExistFunc("STFBkpTot")
1815         STFBkpTot()
1816     EndIf
1817
1818 EndIf
1819 EndIf
1820 EndIf
1821
1822 For nx := 1 To (nAjustCols/8)
1823     cEspaco += chr(160)
1824 Next nx
1825
1826 For nX := 1 To oModel:Length()
1827     oModel:GoLine(nX)
1828     if lClientHtml
1829         Add(aRet, oModel:GetValue('L4_FORMA') + cEspaco + Str(oModel:GetValue('L4_VALOR'),10,2) + cEspaco + AllTrim(Str(oModel:GetValue('L4_PARC'))))
1830     Else
1831         Add(aRet, oModel:GetValue('L4_FORMA') + Space(nAjustCols * 0.055) + Str(oModel:GetValue('L4_VALOR'),10,2) + Space(nAjustCols * 0.095) +
1832         AllTrim(Str(oModel:GetValue('L4_PARC'))))
1833     Endif
1834
1835     If !Empty(oModel:GetValue('L4_FORMA'))
1836         Add(aTotal, {oModel:GetValue('L4_FORMA') ,oModel:GetValue('L4_VALOR')})
1837     Endif
1838 Next nX
1839
1840 Return aRet
1841
1842 //-----
1843 /*{Protheus.doc} STBGetTroco
1844 Retorna o troco
1845
1846 @param
1847 @return
1848 @obs
1849 @sample
1850 */
1851 //-----
1852 Function STBGetTroco()
1853
1854 Local nTroco      := 0                //Valor do troco
1855 Local oTotal      := Nil //Recebe o objeto do model para recuperar os valores
1856 Local nTotal      := STIGetTotal() //Total da nota
1857 Local lRecebTitle := STIGetRecTit()   //Retorna se e recebimento de titulo ou nao

```

```

1858
1859
1860 If !lRecebTitle
1861     oTotal := STFGetTot() //Recebe o objeto do model para recuperar os valores
1862     If nTotal < 0
1863         nTotal := -1 * nTotal //Bonificacao
1864     Endif
1865
1866     If STBCalcSald("1") <= 0
1867         nTroco := nTotal - oTotal:GetValue("L1_VLRTOT")
1868     EndIf
1869 Else
1870     oTotal := STIRetObjTit()
1871     If oTotal:GetTotal() > 0
1872         nTroco := nTotal - oTotal:GetTotal()
1873     EndIf
1874 EndIf
1875
1876 If nTroco < 0
1877     nTroco := 0
1878 Endif
1879
1880 Return nTroco
1881
1882
1883 //-----
1884 /*{Protheus.doc} STBRetCoD
1885 Retorna o numero do Contador de Documento
1886
1887 @param
1888 @return      cCupom - Numero do cupom
1889 @obs
1890 @sample
1891 */
1892 //-----...
1893 Function STBRetCoD()
1894
1895 Local aDados      := {"35", Space(6)}
1896 Local cContDoc    := ""
1897
1898 aRet := STFFireEvent(ProcName(0), "STPrinterStatus", aDaDos)
1899
1900 If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) < 1
1901     STFMessage("STIPayment", "STOP", STR0009) //"Falha na obtenção do Contador de Documento!"
1902     STFShowMessage("STIPayment")
1903 Else
1904     cContDoc := aDados[2]
1905 EndIf
1906
1907 Return cContDoc
1908
1909

```

```

1910 //-----
1911 /*{Protheus.doc} STBInsArredondar
1912 Aparecerá uma pergunta se o cliente deseja doar para o Instituto Arredondar
1913
1914 @param
1915 @return
1916 @obs
1917 @sample
1918 */
1919 //-----
1920 Function STBInsArredondar( cForma )
1921
1922 Local nAux      := 0                      // Pergunta - valor que irá doar
1923 Local lArredondar := SuperGetMV( "MV_LJINSAR",,.F. ) // Parâmetro para habilitar doação ao Inst. Arredondar
1924 Local nRet      := 0                      // Parâmetro de Saida - valor da doação
1925
1926 Default cForma := "R$"                  // Forma de Pagamento
1927
1928 // Se ativado a doação para o Instituto Arredondar
1929 IF lArredondar
1930
1931     If IsMoney(cForma) // Pagamento em Dinheiro
1932         nAux := STBGetTroco()
1933         nAux := nAux-NoRound(nAux,0)
1934     Elseif cForma $ "CC.CD" // Pagamento em Cartão de Débito/Crédito
1935         nAux := STDGPBasket( "SL1" , "L1_VLRTOT" ) // Total da nota
1936         nAux := nAux-NoRound(nAux,0) // Retiro o inteiro, ficam os centavos
1937         nAux := iif(nAux>0,1-nAux,nAux) // Converter em Troco
1938     Else
1939         //Não irá passar pela pergunta, só irá perguntar se duas condições acima combinam.
1940         //(1 é dinheiro e 2 é Tef CD/CC).
1941         Return(STBGetInsArr())
1942     EndIf
1943
1944     If nAux > 0
1945         If MsgNoYes(STR0013 + Alltrim( Transform( nAux, "@E 999.99" ) ) ;
1946             + STR0011, STR0012) // "Deseja doar 999.99 para o Instituto Arredondar?"#"Atenção"
1947             nRet := nAux
1948         EndIf
1949     EndIf
1950
1951 EndIf
1952
1953 Return STBSetInsArr(nRet)
1954
1955
1956 //-----
1957 /*{Protheus.doc} STBGetInsArr
1958 Retorna o valor que o cliente fez doação ao Instituto Arredondar
1959 @param
1960 @return nArredondar - Retorna o valor que o cliente fez doação ao Instituto Arredondar
1961 @obs

```

```

1962 @sample
1963 /*/
1964 */
1965 //-----
1966 Function STBGetInsArr()
1967 Return nArredondar
1968
1969
1970 //-----
1971 /*/{Protheus.doc} STBSetInsArr
1972 Atribui o valor que o cliente fez doação ao Instituto Arredondar
1973
1974 @param      nValor
1975 @return     Nil
1976 @obs
1977 @sample
1978 /*/
1979 */
1980 //-----
1981 Function STBSetInsArr( nValor )
1982
1983 Default nValor := 0
1984
1985 nArredondar := nValor
1986
1987 // Se zero o nArredondar, reinicializo também o nDArredondar
1988 // nArredondar = Doação sem desconto da taxa administrativa do cartão crédito
1989 // nDArredondar = Doação COM desconto da taxa administrativa do cartão crédito
1990 If nValor = 0
1991     nDArredondar := nArredondar
1992 EndIf
1993
1994 Return Nil
1995
1996
1997 //-----
1998 /*/{Protheus.doc} STB0rgL2Item
1999 Reorganiza o L2_Item para nao gerar erro no momento de executar o GrvBatch
2000
2001 @param
2002 @return
2003 @obs
2004 @sample
2005 /*/
2006 //-----
2007 Static Function STB0rgL2Item()
2008
2009 Local nX      := 0                      //Variavel de loop
2010 Local oMdl    := STDGPBModel()          //Recupera o Model
2011 Local oMdLite := oMdl:GetModel('SL2DETAIL') //Model da SL2
2012 Local cL2Item := ''                    //Valor que sera gravado no L2_ITEM
2013 Local nCont   := 0                      //Contador

```

```

2014 Local cL1Num      := STDGPBasket( 'SL1' , 'L1_NUM' ) //L1_NUM da venda
2015 Local lFinServ := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
2016
2017 For nX := 1 To oMdlIte:Length()
2018     oMdlIte:GoLine(nX)
2019     If !oMdlIte:IsDeleted()
2020         If SL2->(DbSeek(xFilial('SL2')+cL1Num+STDGPBasket( 'SL2' , 'L2_ITEM', nX )))
2021             nCont += 1
2022
2023             If Len(AllTrim(Str(nCont))) == 1
2024                 cL2Item := '0' + AllTrim(Str(nCont))
2025             Else
2026                 cL2Item := AllTrim(Str(nCont))
2027             EndIf
2028
2029             If Val(cL2Item) > 99
2030                 cL2Item := STBPegaIT(Val(cL2Item))
2031             EndIf
2032
2033             RecLock("SL2",.F.)
2034             SL2->L2_ITEM := cL2Item
2035             MsUnLock()
2036
2037             STDSPBasket( 'SL2' , 'L2_ITEM', cL2Item, nX )
2038         EndIf
2039     EndIf
2040
2041     //Tratamento para adicionar cobertura Servico Financeiro na importacao de Orcamentos
2042     If lFinServ .And. STBIsFinService(SL2->L2_PRODUTO)
2043         MBF->(dbSetOrder(1))
2044
2045         //Verifica se Produto Financeiro vinculado e sem amarracao
2046         If MBF->(dbSeek(xFilial("MBF") + SL2->L2_PRODUTO)) .And. Empty(SL2->L2_PRDCOBE)
2047             STDFinVinc(oMdlIte, nX)
2048         EndIf
2049     EndIf
2050
2051 Next nX
2052
2053 Return .T.
2054
2055
2056 //-----
2057 /*{Protheus.doc} STBSumImp
2058 Soma o total dos impostos da lei dos impostos
2059
2060 @param
2061 @return      nTotImp
2062 @obs
2063 @sample
2064 */
2065 //-----

```

[illegible]

```

2118 Return nVLTotAl
2119
2120
2121 //-----
2122 /*/{Protheus.doc} STBRmsCheck
2123 Retorna os dados do cheque para integração de CRM com a RMS
2124
2125 */
2126 //-----
2127 Function STBRmsCheck()
2128
2129 Local aRms := Array(14) //Variavel de retorno
2130
2131 aRms[1] := Val(STDGPBasket("SL1", "L1_FILIAL"))
2132 aRms[2] := Val(STDGPBasket("SL1", "L1_ESTACAO"))
2133 aRms[3] := Val(STDGPBasket("SL1", "L1_DOC"))
2134 aRms[4] := STDGPBasket("SL1", "L1_OPERADO")
2135 aRms[5] := SubStr(DToC(STDGPBasket("SL4", "L4_DATA")),Len(DToC(STDGPBasket("SL4", "L4_DATA")))-3,4) + "-" + SubStr(DToC(STDGPBasket("SL4",
"L4_DATA")),Len(DToC(STDGPBasket("SL4", "L4_DATA")))-6,2)+"-" + SubStr(DToC(STDGPBasket("SL4", "L4_DATA")),1,2)
2136 aRms[6] := STDGPBasket("SL4", "L4_ADMINIS")
2137 aRms[7] := STDGPBasket("SL4", "L4_AGENCIA")
2138 aRms[8] := STDGPBasket("SL4", "L4_CONTA")
2139 aRms[9] := STDGPBasket("SL4", "L4_NUMCART")
2140 aRms[10] := STDGPBasket("SL4", "L4_CODCRM")
2141 aRms[11] := STFCpfCli()
2142 aRms[12] := STDGPBasket("SL4", "L4_VALOR")
2143 aRms[13] := ""
2144 aRms[14] := "0"
2145
2146 Return aRms
2147
2148
2149 //-----
2150 /*/{Protheus.doc} STBRmsConv
2151 Retorna os dados do convenio para integração de CRM com a RMS
2152
2153 */
2154 //-----
2155 Function STBRmsConv()
2156
2157 Local aRms := Array(9) //Variavel de retorno
2158
2159 aRms[1] := Val(STDGPBasket("SL1", "L1_FILIAL"))
2160 aRms[2] := Val(STDGPBasket("SL1", "L1_ESTACAO"))
2161 aRms[3] := Val(STDGPBasket("SL1", "L1_DOC"))
2162 aRms[4] := STDGPBasket("SL1", "L1_OPERADO")
2163 aRms[5] := STDGPBasket("SL4", "L4_CODCRM")
2164 aRms[6] := IIF(!Empty(STDGPBasket("SL4", "L4_NUMCART")),2,1)
2165 aRms[7] := IIF(!Empty(STDGPBasket("SL4", "L4_NUMCART")),STDGPBasket("SL4", "L4_NUMCART"),STFCpfCli()) //STDGPBasket("SL4", "L4_NUMCART")
2166 aRms[8] := STDGPBasket("SL4", "L4_VALOR")
2167 aRms[9] := "0"
2168

```

```

2169 Return aRms
2170
2171
2172 //-----
2173 /*{Protheus.doc} STBGetDInsArr
2174 Retorna o valor que o cliente fez doação ao Instituto Arredondar
2175 @return      nDArredondar - Retorna o valor que o cliente fez doação ao Instituto Arredondar com a taxa adm. descontada
2176 @obs
2177 @sample
2178 */
2179 */
2180 //-----
2181 Function STBGetDInsArr()
2182 Local nRet := 0
2183
2184 If nDArredondar == 0
2185     nRet := nArredondar
2186 Else
2187     nRet := nDArredondar
2188 EndIf
2189
2190 Return nRet
2191
2192 //-----
2193 /*{Protheus.doc} STBSetDInsArr
2194 Atribui o valor que o cliente fez doação ao Instituto Arredondar
2195
2196 @param1      nValor
2197 @return      Nil
2198 @obs
2199 @sample
2200 */
2201 */
2202 //-----
2203 Function STBSetDInsArr( nValor )
2204
2205 Default nValor := 0
2206
2207 nDArredondar := nValor
2208
2209 Return Nil
2210
2211 //-----
2212 /*{Protheus.doc} STBAdjustingPay
2213 Soma os valores referentes as formas realizadas manualmente
2214
2215 @param      nVlTotal, cForma, oMdlGrd
2216 @return      Nil
2217 @obs
2218 @sample
2219 */
2220 */

```

```

2221 //-----
2222
2223 Function STBAdjustingPay(nVlTotal,cForma)
2224
2225 Local aArea          := GetArea()
2226 Local nValorTot      := 0
2227 Local nX              := 0
2228 Local cFormaTEF      := ""
2229 Local lPOS           := FindFunction("STFIsPOS") .AND. STFIsPOS()
2230 Local oModel         := Nil
2231 Local oMdlGrd        := Nil
2232
2233 If lPOS
2234     oModel := STISetMdlPay()
2235     If ValType(oModel) <> "U"
2236         oMdlGrd := oModel:GetModel("PARCELAS") //Seta o model do grid
2237     EndIf
2238 EndIf
2239
2240 If oMdlGrd <> Nil
2241
2242     DbSelectArea("SX5")
2243     DbSeek(xFilial("SX5") + "24")
2244     //Procura a forma na tabela 24
2245     While !Eof() .AND. SX5->X5_FILIAL == xFilial("SX5") .AND. SX5->X5_TABELA == "24"
2246         //Verifica se encontrou a forma TEF recebida na funcao
2247         If Upper(AllTrim(SX5->X5_DESCRI)) == Upper(AllTrim(cForma))
2248             //Guarda o tipo da forma TEF
2249             cFormaTEF := AllTrim(SX5->X5_CHAVE)
2250             Exit
2251         EndIf
2252         DbSkip()
2253     End
2254
2255     // Soma forma passada para funcao
2256     // No array oMdlGrd:aCols não existe forma CC ou CD e sim a descricao da forma
2257     For nX := 1 to Len(oMdlGrd:aCols)
2258         oMdlGrd:GoLine(nX)
2259         If AllTrim(oMdlGrd:aCols[nX][4]) == cFormaTEF
2260             nValorTot += oMdlGrd:aCols[nX][3]
2261         Endif
2262     Next
2263
2264     RestArea(aArea)
2265 EndIf
2266
2267 Return(nValorTot)
2268
2269
2270 //-----
2271 /*/{Protheus.doc} STBValFormPay
2272 Executa validação do botão OK das formas de pagamento

```

```

2273
2274 @param1      cTipoForma - Tipo da forma de pagamento
2275 @param2      nValor - Valor da forma de pagamento
2276 @param3      nParc - Quantidade de parcelas
2277 @return      Nil
2278 @obs
2279 @sample
2280 /*/
2281 //-----
2282 Function STBValFormPay(cTipoForma,nValor,nParc)
2283 Local lRet      := .T.
2284 Local lSTValFormPay := ExistBlock("STValFormPay")
2285
2286 Default cTipoForma := ""
2287 Default nValor      := 0
2288 Default nParc       := 1
2289
2290 //Parcela tem que ser 1 - mesmo se for a vista
2291 If nParc < 1
2292     nParc := 1
2293 EndIf
2294
2295 If lSTValFormPay
2296     lRet := ExecBlock("STValFormPay",.F.,.F.,{cTipoForma,nValor,nParc})
2297 EndIf
2298
2299 If lRet .AND. ExistFunc('STBValParc')
2300     lRet := STBValParc(nParc,nValor)
2301 EndIf
2302
2303 // Chama Ponto de NEtrada STISelForm
2304 If lRet .AND. FindFunction("STICallPSF")
2305     lRet := STICallPSF(cTipoForma, nParc, nValor)
2306 EndIf
2307
2308 Return lRet
2309
2310 //-----
2311 /*/{Protheus.doc} STBLDsAcFin
2312 Limpa o desconto financeiro, acrescimo financeiro e o desconto na
2313 multi negociação
2314
2315 @return      Nil
2316 /*/
2317 //-----
2318 Function STBLDsAcFin()
2319 Local oTotal      := STFGetTot() //Totalizador
2320 Local cCondicao    := ""         //Condição de pagamento
2321 Local lExistSE4    := .F.
2322 Local lSTIDescMltN := ExistFunc("STIDescMltNeg") .And. STIDescMltNeg()
2323 Local lDesTot      := ExistFunc("STBCDPGDes") .And. STBCDPGDes()
2324 Local lMVLJJURCC   := SuperGetMV( "MV_LJJURCC",.F. ) // calcula juros da Adm financeira

```

```

2325
2326 //Busca a condição de pagamento
2327 cCondicao := STBGetCdPg()
2328
2329 /* Limpa o desconto financeiro e desconto na multinegociação */
2330 DbSelectArea("SE4")
2331 SE4->(DbSetOrder(1))
2332 lExistSE4 := SE4->(DbSeek(xFilial("SE4")+ cCondicao))
2333 If ( lExistSE4 .AND. (SE4->(E4_DESCFIN + E4_ACRSFIN) > 0) ) .OR. lSTIDescMltN .OR. (oTotal:GetValue("L1_ACRSFIN") > 0 .AND. lMVLJJURCC)
2334
2335     //Zera descontos
2336     STIClearDisc()
2337
2338     /* Limpa o acrescimo e o desconto financeiro */
2339     STFSetTot( "L1_VLRTOT", oTotal:GetValue("L1_VLRTOT") - oTotal:GetValue("L1_ACRSFIN") - oTotal:GetValue("L1_DESCFIN"))
2340     STFSetTot( "L1_ACRSFIN", 0 )
2341     STFSetTot( "L1_JUROS", 0 )
2342     STFSetTot( "L1_VLRJUR", 0 )
2343
2344     If lDesTot
2345         STFSetTot("L1_DESCFIN",0)
2346         STFSetTot("L1_DESCNF",0)
2347     EndIf
2348
2349     /* Necessário limpar e atualizar os valores de impostos e totais da Matxfis pois sistema não
2350     estava zerando os valores após selecionar uma condição de pagamento com acrescimo. */
2351
2352     //Altera os valores de impostos e bases do item
2353     STBTaxAlt( "NF_ACRESCI" , 0 )
2354
2355     //Atualiza os valores de totais da MatxFis
2356     STFRefTot()
2357 EndIf
2358
2359 Return Nil
2360
2361 //-----
2362 /*{Protheus.doc} STBGrvTroco
2363 Efetua a gravação dos campos de Valor e Troco da forma de pagamento dinheiro
2364
2365 @param      oMdlGrd - Objeto com os pagamentos
2366 @param      nX - linha da forma de pagamento
2367 @return     Nil
2368 @obs
2369 @sample
2370 */
2371
2372 //-----
2373 Function STBGrvTroco(oMdlGrd,nX, lAtuModel)
2374 Local lMvLjTroco      := SuperGetMV("MV_LJTROCO", ,.F.) // Verifica se utiliza troco nas diferentes formas de pagamento
2375 Local nMvLjTrDin      := SuperGetMV("MV_LJTRDIN", , 0 ) // Determina se utiliza troco para diferentes formas de pagamento
2376 Local nTroco          := STBGetTroco() // Valor de troco

```

```

2377 Local nValor      := oMdlGrd:GetValue("L4_VALOR")      // Valor total da venda
2378
2379 Default lAtuModel := .f.
2380 If lMvLjTroco
2381     nValor := nValor - IIf(nMvLjTrDin == 0,0,nTroco)
2382 Else
2383     nValor := nValor - nTroco
2384     nTroco := 0
2385 EndIf
2386
2387 STDSPBasket("SL4","L4_VALOR",nValor,nX)
2388 STDSPBasket("SL4","L4_TROCO",nTroco,nX)
2389 If lAtuModel
2390     oMdlGrd:LoadValue("L4_VALOR",nValor)
2391     oMdlGrd:LoadValue("L4_TROCO",nTroco)
2392 EndIf
2393
2394
2395 Return Nil
2396
2397 //-----
2398 /*{Protheus.doc} STBClearCk
2399 Limpar count estatica referente aos cheques
2400
2401 @param
2402 @return      Nil
2403 @obs
2404 @sample
2405 */
2406
2407 //-----
2408 Function STBClearCk()
2409
2410 nContCheque := 0
2411
2412 Return .T.
2413
2414 //-----
2415 /*{Protheus.doc} STBGFormImp
2416 Retorna o array com as formas de pagamento do orçamento importado
2417
2418 @param
2419 @return      Nil
2420 @obs
2421 @sample
2422 */
2423
2424 //-----
2425 Function STBGFormImp()
2426 Return aFormasImp
2427
2428 //-----

```

```

2429 */{Protheus.doc} STSSFormImp
2430 Seta o array com as formas de pagamento do orçamento importado
2431
2432 @param      aForma - Forma de pagamento a ser adicionada (
2433              aForma[1] - "C" - Forma (R$/CC/CD/CH...)
2434              aForma[2] - "N" - Valor
2435              aForma[3] - "C" - Data
2436              aForma[4] - "N" - Parcela
2437 @return     .T.
2438 @obs
2439 @sample
2440 */
2441
2442 //-----
2443 Function STBSFormImp(aForma)
2444
2445 Default aForma := {}
2446
2447 If Len(aForma) > 0
2448     aAdd(aFormasImp, aForma )
2449 Else
2450     aFormasImp := {}
2451 EndIf
2452
2453 Return .T.
2454
2455 //-----
2456 */{Protheus.doc} STBClearPay
2457 Função responsável por limpar todos os arrays estaticos do pagamento
2458 Obs.: Nao existia uma unica função que faça isso, e as função de Zera pagamento
2459 e restart/start da venda fica chamando as mesmas funções.
2460 Favor centralizar aqui caso seja uma forma de pagamento que deve ser zerada no Zerar Pagmento
2461 e no restart ou start da venda.
2462
2463 @param
2464 @return     Nil
2465 @obs
2466 @sample
2467 */
2468
2469 //-----
2470 Function STBClearPay()
2471
2472 STBSFormImp() //Limpar array de formas de pagamentos da importação do orçamento.
2473
2474 Return Nil
2475
2476
2477 //-----
2478 */{Protheus.doc} STBRPayaCols
2479 Função responsável montar o acols dos pagamentos. Fez-se necessario
2480 esta função pois o metodo GetOldData() não retorna corretamente

```

```

2481 o aCols.
2482
2483 @param      oMdlGrd - Objeto de grid dos pagamentos.
2484 @return     aRetaCols - Array com as formas de pagamento
2485 @obs
2486 @sample
2487 */
2488 //-----
2489 Function STBRPayCols(oMdlGrd)
2490 Local aRetaCols := {}
2491 Local aData      := {}
2492 Local nI          := 0
2493
2494 Default oMdlGrd := Nil
2495
2496 If oMdlGrd <> NIL
2497
2498     aData := oMdlGrd:GetData()
2499
2500     For nI := 1 To Len(aData)
2501         aAdd(aRetaCols, aClone(aData[nI][1][1]))
2502     Next nI
2503 EndIf
2504
2505 Return aRetaCols
2506
2507 //-----
2508 /*{Protheus.doc} STBVldForm
2509 Função responsável por validar se o usuario alterou tanto a quantidade de parcelas ou
2510 valor do que foi negociado na pre-venda.
2511 Caso a quantidade ou valor tenha sido alterado, vamos considerar que o PDV ira recalcular o
2512 valor das parcelas e a data de vencimento.
2513
2514 @param      oMdlGrd - Model com as parcelas
2515 @param      aFormasImp - Array com as formas negociadas na retaguarda
2516 @return     lRet
2517 @obs
2518 @sample
2519 */
2520 //-----
2521 Static Function STBVldForm(oMdlGrd,aFormasImp)
2522 Local lRet := .F. //Variavel de retorno
2523 Local nX := 0 //Variavel de loop
2524 Local nContador := 0 //Contador das formas de pagamento
2525 Local nValor := 0 //Valor das parcelas
2526
2527 For nX := 1 To Len(aFormasImp)
2528     If AllTrim(aFormasImp[nX][1]) == AllTrim(oMdlGrd:GetValue('L4_FORMA')) .And.;
2529     aFormasImp[nX][5] == oMdlGrd:GetValue('L4_FORMAID')
2530
2531         nContador++
2532         nValor += aFormasImp[nX][2]

```

```

2533     EndIf
2534 Next nX
2535
2536 If nContador == oMdlGrd:GetValue('L4_PARC') .And. nValor == oMdlGrd:GetValue('L4_VALOR')
2537     lRet := .T.
2538 Else
2539     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), ;
2540         "Houve alteração do valor ou das parcelas pelo usuario," +;
2541         " neste caso tanto as datas e o valor serao recalculados pelo PDV." ) //Gera LOG
2542     lRet := .F.
2543 EndIf
2544
2545 Return lRet
2546
2547 //-----
2548 /*{Protheus.doc} STBIncMHX
2549 Função responsável por realizar a preparação dos dados para a gravação nas tabelas MHJ e MHK
2550
2551 @param      oMdlGrd - Model com as parcelas
2552 @param      oWfReceipt - Objeto com as informações referente ao recebimento realizado
2553 @return
2554 @obs
2555 @sample
2556 */
2557 //-----
2558 Static Function STBIncMHX(oWfReceipt,oMdlGrd)
2559 Local nX      := 0 //Variavel para for
2560 Local nI      := 0 //Variavel para for
2561 Local aMHJ    := {} //Array para gravação da tabela
2562 Local aMHK    := {} //Array para gravação da tabela
2563 Local cNumMHJ := "" //Variavel de incremental
2564 Local cNumMHK := "" //Variavel de incremental
2565 Local cLoteMHJ := "" //Variavel de incremental
2566 Local cMovimento := AllTrim(STDNumMov()) //Numero do movimento atual
2567 Local dMovimento := dDataBase //Data do movimento
2568
2569 cLoteMHJ := GetSxeNum("MHJ","MHJ_LOTE","MHJ_LOTE" + xFilial("MHJ") )
2570 ConfirmSx8()
2571
2572 If Len(oWfReceipt:aListTitles) > 0
2573
2574     For nX := 1 To len(oWfReceipt:aListTitles[4]) //Pego a lista de titulos baixados
2575
2576         If oWfReceipt:aListTitles[4][nX][1]
2577
2578             cNumMHJ := GetSxeNum("MHJ","MHJ_NUM")
2579             ConfirmSx8()
2580
2581             aAdd( aMHJ,{} )
2582             aAdd( aMHJ[len(aMHJ)], { "MHJ_FILIAL", xFilial("MHJ") } )
2583             aAdd( aMHJ[len(aMHJ)], { "MHJ_NUM", cNumMHJ } )
2584             aAdd( aMHJ[len(aMHJ)], { "MHJ_PRXTIT", oWfReceipt:aListTitles[4][nX][2] } )

```

```

2585         aAdd( aMHJ[len(aMHJ)], { "MHJ_NUMTIT", oWFReceipt:aListTitles[4][nX][3] } )
2586         aAdd( aMHJ[len(aMHJ)], { "MHJ_PARTIT", oWFReceipt:aListTitles[4][nX][4] } )
2587         aAdd( aMHJ[len(aMHJ)], { "MHJ_LOTE" , cLoteMHJ } )
2588         aAdd( aMHJ[len(aMHJ)], { "MHJ_VALOR" , oWFReceipt:aListTitles[4][nX][10] } )
2589         aAdd( aMHJ[len(aMHJ)], { "MHJ_SITUA" , "00" } )
2590
2591     EndIf
2592
2593 Next Nx
2594
2595 Else
2596
2597     cNumMHJ := GetSxeNum("MHJ","MHJ_NUM")
2598     ConfirmSx8()
2599
2600     aAdd( aMHJ,{ } )
2601     aAdd( aMHJ[len(aMHJ)], { "MHJ_FILIAL", xFilial("MHJ") } )
2602     aAdd( aMHJ[len(aMHJ)], { "MHJ_NUM" , cNumMHJ } )
2603     aAdd( aMHJ[len(aMHJ)], { "MHJ_PRXTIT", oWFReceipt:cPrefix } )
2604     aAdd( aMHJ[len(aMHJ)], { "MHJ_NUMTIT", oWFReceipt:cNumber } )
2605     aAdd( aMHJ[len(aMHJ)], { "MHJ_PARTIT", oWFReceipt:cParcel } )
2606     aAdd( aMHJ[len(aMHJ)], { "MHJ_LOTE" , cLoteMHJ } )
2607     aAdd( aMHJ[len(aMHJ)], { "MHJ_VALOR" , oWFReceipt:nValueContingency } )
2608     aAdd( aMHJ[len(aMHJ)], { "MHJ_SITUA" , "00" } )
2609
2610 Endif
2611
2612 For nI := 1 To oMdlGrd:Length()
2613
2614     oMdlGrd:GoLine(nI)
2615
2616     cNumMHK := GetSxeNum("MHK","MHK_NUM")
2617     ConfirmSx8()
2618
2619     aAdd( aMHK, { } )
2620     aAdd( aMHK[len(aMHK)], { "MHK_FILIAL", xFilial("MHK") } )
2621     aAdd( aMHK[len(aMHK)], { "MHK_NUM" , cNumMHK } )
2622     aAdd( aMHK[len(aMHK)], { "MHK_LOTE" , cLoteMHJ } )
2623     aAdd( aMHK[len(aMHK)], { "MHK_NUMMOV", cMovimento } )
2624     aAdd( aMHK[len(aMHK)], { "MHK_BANCO" , xNumCaixa() } )
2625     aAdd( aMHK[len(aMHK)], { "MHK_DATMOV", dMovimento } )
2626     aAdd( aMHK[len(aMHK)], { "MHK_TIPOP", oMdlGrd:getvalue("L4_FORMA") } )
2627     aAdd( aMHK[len(aMHK)], { "MHK_CODADM", SUBSTR(oMdlGrd:getvalue("L4_ADMINIS"),1,3) } )
2628     aAdd( aMHK[len(aMHK)], { "MHK_DESADM", SUBSTR(oMdlGrd:getvalue("L4_ADMINIS"),6,20) } )
2629     aAdd( aMHK[len(aMHK)], { "MHK_VALOR" , oMdlGrd:getvalue("L4_VALOR") } )
2630     aAdd( aMHK[len(aMHK)], { "MHK_SITUA" , "00" } )
2631
2632 Next nI
2633
2634 If ExistFunc("STDGrvMHX")
2635     STDGrvMHX(aMHJ,aMHK)
2636 Else

```

```

2637     LjGrvLog("STBIncMXH","Função STDGrvMXH não existe no RP0, a ausencia desta função implicara na conferência de caixa não listando os recebimentos realizados.")
2638 EndIf
2639
2640 Return
2641
2642 //-----
2643 /*{Protheus.doc} STBCancPay
2644 Função responsavel por avaliar se o caixa tem a permissão para cancelar as formas de pagamento definida no orçamento
2645
2646 @return      lRet - indica se o usuario tem permissão para alterar as formas de pagamento
2647 @obs
2648 @sample
2649 */
2650 //-----
2651
2652 Function STBCancPay()
2653 Local lRet      := .T.                // Variavel para retorno
2654 Local lImport   := !Empty(STDGPBasket("SL1" , "L1_NUMORIG")) // Variavel de controle que identifica se é importação de orçamento
2655 Local cMsg      := ""
2656 Local lSTIGetPgOr := ExistFunc("STIGetPgOr") //rotina que congtrola se existe formas de pagamento do orçamento a seres processadas
2657 Local lSemPerm   := .T.
2658
2659 If lImport
2660
2661     If (lSTIGetPgOr .AND. Empty(STIGetPgOr()))
2662         lSemPerm := .F.
2663     Endif
2664
2665     IF !STFProfile(41)[1] .AND. lSemPerm
2666         lRet := .F.
2667         cMsg:= STR0025//"Usuario sem permissão para alterar a negociação do orçamento."
2668     Elseif !STFProfile(9)[1] .AND. lSemPerm
2669         lRet := .F.
2670         cMsg:= STR0027//"Sem permissão para Alterar Parcelas"
2671     Endif
2672
2673     If !lRet
2674         STFMessage( ProcName(),"ALERT", cMsg)
2675         STFShowMessage( ProcName() )
2676     EndIf
2677
2678     If lRet .AND. ExistFunc("SetOrcP0ri")
2679         // Zera array com os pagamentos do orçamento importado.
2680         SetOrcP0ri({"",0,0})
2681     EndIf
2682
2683 Endif
2684
2685 Return lRet
2686
2687
2688 //-----

```

```

2689 /*{Protheus.doc} STBGrvSatT
2690 Prepara as informações de impostos PIS, Cofins, CST etc. para XML SAT
2691
2692 @param      nil
2693 @return     nil
2694 @obs       Pré-requisito: SAT validado
2695 @sample
2696 */
2697 //-----
2698 Function STBGrvSatT()
2699
2700 Local lExistGetTriSat := ExistFunc("LjGetTriSat") .AND. ExistFunc("LjSetTriSat")
2701 Local nX              := 0
2702 Local aSATtrib        := {}                                //Tributação produto SAT
2703 Local nItAux          := 0                                //Tratamento aSATtrib
2704 Local nItAnt          := 0                                //Tratamento aSATtrib
2705 Local nItAtual        := 0                                //Tratamento aSATtrib
2706
2707 If lExistGetTriSat      //Somente SAT
2708
2709     //aSatTrib: É um conjunto de elementos, lido pelo MatxFis, e gravado em XML Sat sem alterar qualquer elemento armazenado em Matxfis.
2710     //Para o TOTVS PDV e SAT, sua matriz é composta de itens que não foram deletados. Os deletados previamente não constam em alguma linha de array.
2711     aSATtrib            := LjGetTriSat()    //Tributação produto SAT
2712
2713     //aSATtrib[x][1] Numero do item real, contando os deletados. Aqui não constam mais os itens deletados.
2714     //aSATtrib[x][2] Campo a ser pesquisado
2715     //aSATtrib[x][3] Valor do campo
2716     //aSATtrib[x][4] Numero do item SEM CONTAR OS DELETADOS, após novo filtro antes do envio XML SAT (Somente TOTVS PDV)
2717
2718     //Exemplo:
2719     //ITEM 1 - DELETADO - não consta mais no array aSATtrib
2720     //ITEM 2 - DELETADO - não consta mais no array aSATtrib
2721     //ITEM 3 - ABERTO
2722     //ITEM 4 - DELETADO - não consta mais no array aSATtrib
2723     //ITEM 5 - ABERTO
2724
2725     //Após o tratamento abaixo, gravará no elemento 4 o item 1 no item 3 e o item 2 no item 5.
2726     //3 - IT_BASECOF - 3,00 - 1
2727     //5 - IT_BASECOF - 3,00 - 2
2728     //O resultado é lido no XML SAT a partir do elemento 4.
2729
2730     nItAux := 0
2731     nItAtual := 0
2732     For nX := 1 to Len(aSatTrib)
2733         nItAnt := aSatTrib[nX][1]
2734         If nItAnt <> nItAux
2735             nItAtual++                //Recontagem do número de itens e atribuo mais tarde no elemento 4
2736         EndIf
2737         nItAux := nItAnt
2738         aSatTrib[nX][4] := nItAtual    //Atualizo o número do item a ser passado pelo XML SAT
2739     Next
2740     LjSetTriSat(aSATtrib)

```

```

2741 EndIf
2742
2743 Return nil
2744
2745
2746 /*{Protheus.doc} STBSX5Dsc()
2747     Retorna a descrição da forma de pagamento cadastrado no SX5 tabela 24.
2748     @type Function
2749     @param cForma, caractere, código da forma de pagamento
2750     @return cRet, caractere, descrição da forma de pagamento
2751     */
2752 Static Function STBSX5Dsc(cForma)
2753 Local cRet      := ""
2754 Local nPos      := 0
2755 Local aFormaSX5 := FWGetSX5("24")
2756
2757 If !Empty(cForma)
2758     nPos:= aScan(aFormaSX5,{|x| Trim(x[3]) == cForma})
2759     If nPos>0
2760         cRet:= aFormaSX5[nPos][4]
2761     Endif
2762 Endif
2763
2764 Return cRet
2765
2766 /*{Protheus.doc} STBStDadosCH
2767     Função criada para gravar dados do cheque como L4_ADMINIS,L4_NUMCART,L4_AGENCIA,L4_CONTA,L4_RG,L4_TELEFON,L4_TERCEIR e L4_NOMECLI na tabela SEF
2768     @type Function
2769     @param oMdlPaym,object, objeto model do pagamento
2770     @param aFieldsGrd, array, array com campos das informações da SL4
2771     @param aRetCheck, array, array com as informações do cheque
2772     @param nContCheque,Numérico, Numero indica que esta sendo utilizado a forma de pagamento 'CH'
2773     @param nJ,Numérico,Numero de variavel de loop.
2774     @param nI,Numérico,Numero de variavel de loop.
2775     @return nil
2776     */
2777 Static Function STBStDadosCH(oMdlPaym, aFieldsGrd, aRetCheck, nContCheque, nJ, nI)
2778 Local aDadosCH:= {}
2779
2780 iif( !ValType(aRetCheck[1][1][1])=="A", aAdd(aDadosCH, aRetCheck), aDadosCH:=aClone(aRetCheck) )
2781
2782 If ValType(aDadosCH[1][1][1]) == 'A' .AND. Len(aDadosCH[nContCheque][nI][1]) >= 10
2783
2784     Do Case
2785         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_ADMINIS"
2786             oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][1])
2787         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_NUMCART"
2788             oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][2])
2789         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_AGENCIA"
2790             oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][3])
2791         Case AllTrim(aFieldsGrd[nJ][3]) == "L4_CONTA"
2792             oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][4])

```

```
2793      Case AllTrim(aFieldsGrd[nJ][3]) == "L4_RG"
2794          oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][8])
2795      Case AllTrim(aFieldsGrd[nJ][3]) == "L4_TELEFON"
2796          oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][7])
2797      Case AllTrim(aFieldsGrd[nJ][3]) == "L4_TERCEIR"
2798          oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][9])
2799      Case AllTrim(aFieldsGrd[nJ][3]) == "L4_NOMECLI"
2800          oMdlPaym:LoadValue(aFieldsGrd[nJ][3], aDadosCH[nContCheque][nI][1][6])
2801      EndCase
2802 EndIf
2803
2804 Return
```

Componentes/Payment/STBPaymentOptionsCreator.prw

189 linhas após remoção de metadados de autoria/data

```

1 #include 'PROTHEUS.CH'
2
3 // Interface de array para ser enviada para o DAO
4 #DEFINE TAMANHOPAY      3 // Opcional
5 #DEFINE TIPO            1 // Obrigatório
6 #DEFINE DESCRICAO       2 // Opcional
7 #DEFINE CODIGO          3 // Opcional
8
9 Function STBPaymentOptionsCreator ; Return      // "dummy" function - Internal Use
10
11
12 //-----
13 /*{Protheus.doc} STBPaymentOptionsCreator
14 Classe STBPaymentOptionsCreator, cria e mostra opções de pagamento
15 @param
16 @return Self
17 @obs
18 @sample
19 */
20 //-----
21 Class STBPaymentOptionsCreator
22
23 Data OptPay
24
25 Data oPayX5
26 Data oAdmin
27 Data oConPay
28 Data oOptPag
29
30 //Construtor
31 Method STBPaymentOptionsCreator(oPayX5, oAdmin, oConPay)
32
33 // Public
34 Method OptionsCreator()
35
36 // Interno
37 Method PayDefault()
38 Method AdmOptionsCreator()
39 Method CondPagOptionsCreator()
40
41 EndClass
42
43 //-----
44 /*{Protheus.doc} STBPaymentOptionsCreator
45 Classe STBPaymentOptionsCreator, metodo construtor
46 @param oFmX5      Model com formas de Pagamento
47 @param oAdmin     Model com Adm Financeira (detalhes: Juros e Descontos)
48 @param oConPag    Model com Condição de pagamento
49 @return Self
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Method STBCPaymentOptionsCreator(oPayX5, oAdmin, oConPay) Class STBCPaymentOptionsCreator
55
56 Self:oPayX5      := oPayX5
57 Self:oAdmin      := oAdmin
58 Self:oConPay     := oConPay
59
60 Self:oOptPag := STDAOptionsPayments():STDAOptionsPayments()
61
62 Return
63
64 //-----
65 /*/{Protheus.doc} OptionsCreator
66 @param
67 @return Self
68 @obs
69 @sample
70 */
71 //-----
72 Method OptionsCreator() Class STBCPaymentOptionsCreator
73
74 Self:PayDefault()
75 Self:AdmOptionsCreator()
76 Self:CondPagOptionsCreator()
77
78 Self:oOptPag:GetAll()
79
80 Return()
81
82 //-----
83 /*/{Protheus.doc} AdmOptionsCreator
84 Metodo cria opcao baeada no SAE
85 @param
86 @return Self
87 @obs
88 @sample
89 */
90 //-----
91 Method AdmOptionsCreator() Class STBCPaymentOptionsCreator
92
93 Local cTypePag := "" //Tipo de pagamento
94 Local nX      := 1 //Variavel de loop
95 Local aOpt    := Array(TAMANHOPAY) //Tamanho do array
96
97 For nX := 1 To oPayX5:Lengt()
98
99     oPayX5:GoLine(nX)
100     cTypePag := oPayX5:GetValue("X5_TYPE")
101
102     If Self:oAdmin:SeekLine({{ "X5_TYPE", cTypePag }})
103         aOpt[TIP0] := oPayX5:GetValue("X5_TYPE")
104         aOpt[DESCRICA0] := oPayX5:GetValue("X5_DESC")

```

```

105         aOpt[CODIGO]      := ""
106         Self:oOptPag:Add(aOpt)
107         // Renicia Array para não levar valor da posicao anterior
108         aOpt               := Array(TAMANHOPAY)
109     EndIf
110
111 Next nX
112
113 Return
114
115 //-----
116 /*{Protheus.doc} AdmOptionsCreator
117 Metodo cria opcao baseada no se4
118 @param
119 @return Self
120 @obs
121 @sample
122 */
123 //-----
124 Method CondPagOptionsCreator() Class STBCPaymentOptionsCreator
125
126 Local nX      := 1      // Contador
127 Local aOpt    := Array(TAMANHOPAY) //Tamanho do array
128
129 For nX := 1 To oConPay:Lengt()
130
131     oConPay:GoLine(nX)
132
133     aOpt[TIPO]      := oPayX5:GetValue("CP")
134     aOpt[DESCRICA0] := oPayX5:GetValue("E4_DESCRI")
135     aOpt[CODIGO]    := oPayX5:GetValue("E4_CODIGO")
136
137     Self:oOptPag:Add(aOpt)
138     // Renicia Array para não levar valor da posicao anterior
139     aOpt          := Array(TAMANHOPAY)
140
141 Next nX
142
143 Return
144
145 //-----
146 /*{Protheus.doc} AdmOptionsCreator
147 Metodo cria opcao de Default
148 @param
149 @return Self
150 @obs
151 @sample
152 */
153 //-----
154 Method PayDefaut() Class STBCPaymentOptionsCreator
155
156 Local nX      := 1      // Contador

```

```
157 Local aOpt      := Array(TAMANHOPAY) //Tamanho do array
158
159 // Cheque
160 aOpt[TIPO]       := oPayX5:GetValue("CH")
161 aOpt[DESCRICA0]  := oPayX5:GetValue("Cheque")
162 aOpt[CODIGO]     := ""
163 Self:oOptPag:Add(aOpt)
164 // Renicia Array para não levar valor da posicao anterior
165 aOpt := Array(TAMANHOPAY)
166
167
168 // Cond Negociada
169 aOpt[TIPO]       := "CN"
170 aOpt[DESCRICA0]  := "Cond Negociada"
171 aOpt[CODIGO]     := ""
172 Self:oOptPag:Add(aOpt)
173 // Renicia Array para não levar valor da posicao anterior
174 aOpt := Array(TAMANHOPAY)
175
176
177 // Cond Negociada
178 aOpt[TIPO]       := "MN"
179 aOpt[DESCRICA0]  := "Multi Negociação"
180 aOpt[CODIGO]     := ""
181 Self:oOptPag:Add(aOpt)
182 // Renicia Array para não levar valor da posicao anterior
183 aOpt := Array(TAMANHOPAY)
184
185
186 // Analisar
187 //Self:oOptPag:Add("Diheiro")
188
189 Return
```

Componentes/Payment/STBWCPayment.prw

140 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 //-----
6 // Funcao dumb
7 //-----
8 Function STBWCPayment() ; Return
9
10 //-----
11 /*/{Protheus.doc} STBWCPayment
12 Classe responsavel por fazer todos WorkFlow e chamar outros Workflow de
13 pagamento
14 @param
15 @return Self
16 @see
17 @obs
18 */
19 //-----
20 Class STBWCPayment
21
22 Data oCache      // Objeto cache
23 Data oPayX5      // Objeto Pagamento SX5
24 Data oAdmin      // Objeto Adminstrador
25 Data oConPay      // Objeto Condicao pagamento
26 Data oOptPay      // Objeto opcao de pagamento
27 Data oSelPay      // Objeto selecao de pagamento
28
29 Method STBWCPayment()
30 Method STBWCPaymentStartPayment()
31 Method STBWCPaymentOptionsCreator()
32 Method Show()
33
34 EndClass
35
36
37 //-----
38 /*/{Protheus.doc} STBWCPayment
39 Metodo construtor
40 pagamento
41 @param
42 @return Nil
43 @see
44 @obs
45 */
46 //-----
47 Method STBWCPayment() Class STBWCPayment
48
49 Self:oCache := STFWCMemCache():STFWCMemCache()
50
51 Self:oPayX5 := STDTTypePayX5():STDTTypePayX5(Self:oCache)
52 Self:oPayX5:GetAllData()

```

```

53
54 Self:oAdmin := STDAAdministratorFinacial():STDAAdministratorFinacial(Self:oCache)
55 Self:oAdmin:GetAllData()
56
57 //TODO:
58 //Self:oConPay := STDAPayConditionPayment():STDAPayConditionPayment(Self:oCache)
59 //Self:oConPay:GetAllData()
60
61 // Retorno já com os dados, u seja não é necessario usar o GetAll, apesar de existir,
62 // esse Metodo Privado
63 Self:oOptPay := Self:STBCPaymentOptionsCreator(Self:oCache)
64
65 Return Nil
66
67
68 //-----
69 /*{Protheus.doc} STBWStartPayment
70 Classe responsavel iniciar Workflow de Pagamento
71 pagamento
72 @param
73 @return Self
74 @see
75 @obs
76 */
77 //-----
78 Method STBWStartPayment() Class STBWCPayment
79
80 Self:Show()
81
82 //TODO: implementar???
83
84 If oOptPay:cRet := "CC"
85
86 ElseIf oOptPay:cRet := "CD"
87
88 ElseIf oOptPay:cRet := "FI"
89
90 ElseIf oOptPay:cRet := "CP"
91
92 ElseIf oOptPay:cRet := "CN"
93
94 ElseIf oOptPay:cRet := "MN"
95
96 EndIf
97
98 Return Nil
99
100
101 //-----
102 /*{Protheus.doc} STBCPaymentOptionsCreator
103 Classe responsavel Chamar classe para criar opcoes de pagamento
104 pagamento

```

```
105 @param
106 @return Nil
107 @see
108 @obs
109 /*/
110 //-----
111 Method STBCPaymentOptionsCreator() Class STBWCPayment
112
113 Self:oSelPay := STBCPaymentOptionsCreator():STBCPaymentOptionsCreator(Self:oPayX5, Self:oAdmin, Self:oConPay) // ajustar
114
115 Return Nil
116
117
118 //-----
119 /*/{Protheus.doc} Show
120 Metodos responsavel Chamar classe de View
121 pagamento
122 @param
123 @return Nil
124 @see
125 @obs
126 /*/
127 //-----
128 Method Show() Class STBWCPayment
129
130 Local nX // Contador
131
132 For nX := 1 to Len(oSelPay)
133
134     CoNout("R$")
135     CoNout("CC")
136     CoNout("CD")
137
138 Next nX
139
140 Return Nil
```

Componentes/Payment/STDATypePayX5.prw

209 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static oStructSX5 := Nil
5
6 //-----
7 //STDTypePayX5 function dumb
8 //-----
9 Function STDTypePayX5() ; Return
10
11
12 //-----
13 // Declaracao da class
14 //-----
15 Class STDTypePayX5
16
17 Data oCache
18
19 // Construtor
20 Method STDTypePayX5(oCache)
21
22 // Publico
23 Method GetAllData()
24
25 // Privado
26 Method GetModel()
27 Method CreateStruct()
28
29 EndClass
30
31
32 //-----
33 /*{Protheus.doc} STDTypePayX5
34 Metodo Construtor
35 @param
36 @return Self
37 @obs
38 @sample
39 */
40 //-----
41 Method STDTypePayX5(oCache) Class STDTypePayX5
42 Self:oCache := oCache
43 Return Self
44
45
46 //-----
47 /*{Protheus.doc} GetModel
48 Pega o model de pagamentos SX5
49 @param
50 @return oModel - Retorna o Model
51 @obs
52 @sample

```

```

53 /*/
54 //-----
55 Method GetModel() Class STDTypePayX5
56
57 Local oMasterStr      := Nil // Estrutura SLQ
58 Local oGridStr        := Nil // Estrutura SLR
59 Local oModel          := Nil // Model
60 Local aTabela         := {} // Informacoes da tabela
61 Local oNewModel       := Nil // Modelo preenchido retornado
62
63 //Monta estrutura de tabela de cabecario
64 oMasterStr := Self:CreateStruct()
65 oGridStr   := Self:CreateStruct()
66
67 //Instacia Objeto
68 oModel := MPFormModel():New( "X524", /*bPreValidacao*/, /*bPosValidacao*/, /*{ |oMdl| xFRT80MVCGR( oMdl ) }*/, /*bCancel*/ )
69
70 oModel:AddFields( "MasterStr" , /*cOwner*/, oMasterStr )
71 oModel:AddGrid("GridStr" , "MasterStr", oGridStr, /*B*/ , /*bLinePost*/, /*bPreVal*/, /*bPosVal*/, /*BLoad*/ )
72 oModel:SetRelation("GridStr" , { { "X5_NUM", 'X5_NUM' }, { "X5_NUM", "X5_NUM" } }, "X5_NUM + X5_NUM" )
73
74 Return oModel
75
76
77 //-----
78 /*/{Protheus.doc} GetAllData
79 pega model
80 @param
81 @return oNewModel retorno Model
82 @obs
83 @sample
84 */
85 //-----
86 Method GetAllData() Class STDTypePayX5
87
88 Local oNewModel      := Nil // Retorno do Model
89 Local nLine          := 0 // Numero de linha do Grid
90 Local lFirstLine     := .T. // Primeiro regisro incluso
91 Local cNum           := "001" // Numero de cada Registro
92 Local aRetSx5        := {} // Array com retorno de registros da SX5 tabela 24
93 Local nCont          := 0 // contador de registros
94
95 oNewModel := Self:GetModel() // Chama a estrutura de dados
96
97 oNewModel:SetOperation( 3 )
98 oNewModel:SetDescription("Condição de Pagamento")
99
100 oNewModel:Activate()
101
102 // Dados
103 oModelMarte := oNewModel:GetModel("MasterStr")
104

```

```

105 aRetSx5 := FWGetSX5('24')
106
107 If Len(aRetSx5) > 0
108
109     oModelMarte:SetValue("X5_NUM"      ,   cNum)
110     oModelMarte:SetValue("X5_TYPE"     ,   Alltrim(aRetSx5[1][3]))
111     oModelMarte:SetValue("X5_DESC"     ,   Left(Alltrim(aRetSx5[1][4]),30))
112
113     //Carrega todas as Adm na 1ª Linha
114     oModSlr := oNewModel:GetModel("GridStr")
115
116     For nCont:= 1 to Len(aRetSx5)
117
118         If !lFirstLine
119             nLine := oModSlr:AddLine()
120         EndIf
121
122         oModSlr:SetValue("X5_NUM"      ,   cNum)
123         oModSlr:SetValue("X5_TYPE"     ,   Alltrim(aRetSx5[nCont][3]))
124         oModSlr:SetValue("X5_DESC"     ,   Left(Alltrim(aRetSx5[nCont][4]),30))
125
126         lFirstLine := .F.
127
128     Next
129
130 Endif
131
132 Return oNewModel
133
134
135 //-----
136 /*{Protoheus.doc} CreateStruct
137 Cria estrutura
138 @param
139 @return oStruct   retorna struct
140 @obs
141 @sample
142 */
143 //-----
144 Method CreateStruct() Class STDTypePayX5
145
146     Local cAlias      := "SX5"                                // Alias que criara estrutura
147
148     If ValType(oStructSX5) = '0'
149         oStructSX5:Deactivate()
150         oStructSX5:Activate()
151     Else
152
153         oStructSX5 := FWFormModelStruct():New()    // Estrutura
154
155         oStructSX5:AddTable(
156             "SX5"
157             ,
158             ,
159             // [01] Alias da tabela

```

```

157         StrTokArr( "X5_TYPE", '+' ) , ; // [02] Array com os campos que correspondem a primary key
158         "TypePay" ) // [03] Descrição da tabela
159
160     oStructSX5:AddField( ;
161         "NUM" , ; // [01] Titulo do campo
162         "NUM" , ; // [02] Desc do campo
163         "X5_NUM" , ; // [03] Id do Field
164         "C" , ; // [04] Tipo do campo
165         3 , ; // [05] Tamanho do campo
166         0 , ; // [06] Decimal do campo
167         Nil , ; // [07] Code-block de validação do campo
168         Nil , ; // [08] Code-block de validação When do campo
169         Nil , ; // [09] Lista de valores permitido do campo
170         Nil , ; // [10] Indica se o campo tem preenchimento obrigatório
171         Nil , ; // [11] Code-block de inicializacao do campo
172         NIL , ; // [12] Indica se trata-se de um campo chave
173         NIL , ; // [13] Indica se o campo pode receber valor em uma operação de update.
174     ) // [14] Indica se o campo é virtual
175
176     oStructSX5:AddField( ;
177         "TYPE" , ; // [01] Titulo do campo
178         "TYPE" , ; // [02] Desc do campo
179         "X5_TYPE" , ; // [03] Id do Field
180         "C" , ; // [04] Tipo do campo
181         3 , ; // [05] Tamanho do campo
182         0 , ; // [06] Decimal do campo
183         Nil , ; // [07] Code-block de validação do campo
184         Nil , ; // [08] Code-block de validação When do campo
185         Nil , ; // [09] Lista de valores permitido do campo
186         Nil , ; // [10] Indica se o campo tem preenchimento obrigatório
187         Nil , ; // [11] Code-block de inicializacao do campo
188         NIL , ; // [12] Indica se trata-se de um campo chave
189         NIL , ; // [13] Indica se o campo pode receber valor em uma operação de update.
190     ) // [14] Indica se o campo é virtual
191
192     oStructSX5:AddField( ;
193         "DESCRICAO" , ; // [01] Titulo do campo
194         "DESCRICAO" , ; // [02] Desc do campo
195         "X5_DESC" , ; // [03] Id do Field
196         "C" , ; // [04] Tipo do campo
197         30 , ; // [05] Tamanho do campo
198         0 , ; // [06] Decimal do campo
199         Nil , ; // [07] Code-block de validação do campo
200         Nil , ; // [08] Code-block de validação When do campo
201         Nil , ; // [09] Lista de valores permitido do campo
202         Nil , ; // [10] Indica se o campo tem preenchimento obrigatório
203         Nil , ; // [11] Code-block de inicializacao do campo
204         NIL , ; // [12] Indica se trata-se de um campo chave
205         NIL , ; // [13] Indica se o campo pode receber valor em uma operação de update.
206     ) // [14] Indica se o campo é virtual
207 EndIf
208

```

209 Return oStructSX5

Componentes/Payment/STDAdministratorFinancial.prw

417 linhas após remoção de metadados de autoria/data

```

1 #include 'PROTHEUS.CH'
2 #Include "FWMVCDEF.CH"
3 #INCLUDE "STDADMINISTRATORFINANCIAL.CH"
4
5 Static oStructSAE := Nil
6 Static oModelAdmF := Nil
7
8 Function STDAAdministratorFinancial ; Return    // "dummy" function - Internal Use
9
10 //-----
11 /*{Protheus.doc} STDTAdministratorFinancial
12 Classe STDTAdministratorFinancial, que cria model baseado no Alias SAE
13 @param
14 @return Self
15 @see
16 @todo
17 @obs
18 */
19 //-----
20 Class STDAAdministratorFinancial
21
22 Data oCache
23
24 // Construtor
25
26 Method STDAAdministratorFinancial(oCache)
27 // Publico
28 Method GetAllData()
29
30 // Privado
31 Method GetModel()
32 Method CreateMasterStruct()
33 Method CreateGridStruct()
34 Method CreateDetGridStruct()
35
36 EndClass
37
38 //-----
39 /*{Protheus.doc} STDTAdministratorFinancial
40 Metodo construtor da classe STDTAdministratorFinancial
41 @param oCache componente para realizar cache
42 @return Self
43 @see
44 @todo
45 @obs
46 */
47 //-----
48 Method STDAAdministratorFinancial(oCache) Class STDAAdministratorFinancial
49 Self:oCache := oCache
50 Return Self
51
52 //-----

```

```

53 /*/{Protheus.doc} GetModel
54 Cria e retorna estrutura de dados do Model
55 @param
56 @return Self
57 @see
58 @todo
59 @obs
60 */
61 //-----
62 Method GetModel() Class STDAAdministratorFinancial
63 Local oMasterStr := Nil // Estrutura SLQ
64 Local oGridStr := Nil // Estrutura SLR
65 Local oModel := Nil // Model
66 Local oGridDetStr := Nil // Strutura de Projetos
67
68 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
69 //³Monta estrutura de tabela de cabecario³
70 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
71 oMasterStr := Self:CreateMasterStruct()
72
73 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
74 //³Monta estrutura de tabela de Itens³
75 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
76 oGridStr := Self:CreateGridStruct()
77
78 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
79 //³Monta estrutura de tabela de Detal³
80 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
81 oGridDetStr := Self:CreateDetGridStruct()
82
83 //UAAAAAAAAAAAAAAAAA;
84 //³Instacia Objeto³
85 //AAAAAAAAAAAAAAAAU
86 oModel := MPFormModel():New( 'SAE', /*bPreValidacao*/, /*bPosValidacao*/, /*{ |oMdl| xFRT80MVCGR( oMdl ) }*/, /*bCancel*/ )
87
88 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
89 //³Adocionas os model e os relcionam³
90 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
91 oModel:AddFields( "MasterStr" , /*cOwner*/, oMasterStr )
92
93 oModel:AddGrid("GridStr" , "MasterStr", oGridStr, /*B*/ , /*bLinePost*/, /*bPreVal*/, /*bPosVal*/, /*BLoad*/ )
94 oModel:SetRelation("GridStr" , { { "AE_FILIAL", 'xFilial( "SAE" )' }, { "AE_COD", "AE_COD" } }, 'AE_FILIAL + AE_COD' )
95
96 oModel:AddGrid("GridDetStr" , "GridStr" , oGridDetStr, /*B*/ , /*bLinePost*/, /*bPreVal*/, /*bPosVal*/, /*BLoad*/ )
97 oModel:SetRelation("GridDetStr" , { { 'MEN_FILIAL', 'xFilial( "MEN" )' }, { 'MEN_CODADM' , 'AE_COD' } } , MEN->(IndexKey(2)) )
98
99 oModel:GetModel( "GridDetStr" ):SetOptional(.T.)
100
101 Return oModel
102
103 //-----
104 /*/{Protheus.doc} GetAllData

```

```

105 Inclui os dados no model e retorna
106 @param
107 @return Self
108 @see
109 @todo
110 @obs
111 */
112 //-----
113 Method GetAllData() Class STDAAdministratorFinancial
114
115 Local oNewModel      := Nil           // Retorno do Model
116 Local nX              := 0            // Contador do Struct
117 Local nY              := 0            // Contador do Struct Men
118 Local cCampo         := ""           // Campo da tabela que incluirá novo Registros
119 Local lFirstLine     := .T.           // Primeiro registro incluso
120 Local oModelMater     := Nil          // Model do Field Master
121 Local oModelGrid     := Nil          // Model do Grid
122 Local aStrGrid       := {}           // Vetor com estrutura de dados
123 Local oModelDetGrid  := Nil          // Model do Field detalhe Grid
124 Local aStrDetGrid    := {}           // Vetor com estrutura de dados
125 Local lDetaFirstLin  := .T.           // Primeira Linha
126 Local nLineDt        := 0
127
128 oNewModel := Self:GetModel() // Chama a estrutura de dados
129
130 oNewModel:SetOperation( 3 )
131 oNewModel:SetDescription(STR0001) //"Administradora Financeira"
132
133 oNewModel:Activate()
134
135 DbSelectArea("SAE")
136 DbSetOrder(1)
137 SAE->(DbSeek(xFilial("SAE")))
138
139 // Dados
140 oModelMater := oNewModel:GetModel("MasterStr")
141 oModelMater:SetValue("AE_FILIAL" , SAE->AE_FILIAL)
142 oModelMater:SetValue("AE_COD" , SAE->AE_COD )
143
144 // Carrega todas as Adm na 1ª Linha
145 oModelGrid := oNewModel:GetModel("GridStr")
146 aStrGrid := oModelGrid:GetStruct() // Retorna a estrutura de dados para Put dos valores
147
148
149 // Carrega todas do detalhes da Adm na 1ª Linha
150 oModelDetGrid := oNewModel:GetModel("GridDetStr")
151 aStrDetGrid := oModelDetGrid:GetStruct() // Retorna a estrutura de dados para Put dos valores
152
153 If ValType(oModelAdmF) = '0'
154     oNewModel := LjGetMdAdm()
155 Else
156

```

```

157 // Carrega todas as Adm
158 While !SAE->(EOF()) .And. SAE->AE_FILIAL == xFilial("SAE")
159
160     If SAE->(FieldPos("AE_MSBLQL")) > 0 .AND. SAE->AE_MSBLQL == "1"
161         // Desconsidera Adm Bloqueada
162         SAE->(DbSkip())
163     Loop
164 EndIf
165
166 If !lFirstLine
167     nLine := oModelGrid:AddLine()
168 EndIf
169
170 For nX := 1 TO Len(aStrGrid:aFields)
171     If !aStrGrid:aFields[nX][MODEL_FIELD_VIRTUAL]
172         cCampo := aStrGrid:aFields[nX][3]
173         oModelGrid:SetValue(cCampo , &("SAE->" + cCampo))
174     EndIf
175 Next nX
176
177 // Carrega Detalhes da Adm Fin (Juros, Descontos e etc)
178 DbSelectArea("MEN")
179 DbSetOrder(2)
180 If DbSeek(xFilial("MEN") + SAE->AE_COD)
181
182     // SOmente dados da Adm atual do ponteiro
183     lDetaFirstLin := .T.
184     While (!MEN->(EOF())) .AND. (MEN->MEN_FILIAL == xFilial("MEN")) .AND. (MEN->MEN_CODADM == SAE->AE_COD)
185
186         If !lDetaFirstLin
187             nLineDt := oModelDetGrid:AddLine()
188
189         EndIf
190         Conout(Str(nLineDt))
191
192         For nY := 1 TO Len(aStrDetGrid:aFields)
193             If !aStrDetGrid:aFields[nY][MODEL_FIELD_VIRTUAL]
194                 cCampo := aStrDetGrid:aFields[nY][3]
195                 oModelDetGrid:SetValue(cCampo , &("MEN->" + cCampo))
196             EndIf
197         Next nY
198         lDetaFirstLin := .F.
199         MEN->(DbSkip())
200     End
201
202 EndIf
203 nLineDt := 1
204 lFirstLine := .F.
205 SAE->(DbSkip())
206 End
207 EndIf
208

```

```

209 If ValType(oModelAdmF) = 'U'
210     LjSetMdAdm(oNewModel)
211 EndIf
212
213 Return oNewModel
214
215 //-----
216 /*{Protheus.doc} CreateMasterStruct
217 Cria estrutura do Model Master
218 @param
219 @return Struc
220 @see
221 @todo
222 @obs
223 */
224 //-----
225 Method CreateMasterStruct() Class STDAAdministratorFinancial
226
227     Local cAlias      := "SAE"                // Alias que criara estrutura
228     Local aArea       := Nil
229     Local cX2Unico    := "AE_FILIAL+AE_COD" //X2_UNICO da tabela SAE
230
231     If ValType(oStructSAE) = '0'
232         oStructSAE:Deactivate()
233         oStructSAE:Activate()
234     Else
235
236         oStructSAE := FWFormModelStruct():New()
237
238         aArea := GetArea() // Guarda area
239         //Carrega informacoes da Tabela
240         SX2->( DbSetOrder( 1 ) )
241         SX2->( DbSeek( cAlias ) )
242
243         If ExistFunc('FWX2Unico')
244             cX2Unico := FWX2Unico(cAlias)
245         EndIf
246
247         oStructSAE:AddTable(
248             FWX2CHAVE()
249             StrTokArr( cX2Unico, '+' )
250             FWX2Nome(cAlias) )
251             ,
252             ,
253             ,
254             ,
255             ,
256             ,
257             ,
258             ,
259             ,
260             ,
261             ;
262             ;
263             ;
264             ;
265             ;
266             ;
267             ;
268             ;
269             ;
270             ;
271             ;
272             ;
273             ;
274             ;
275             ;
276             ;
277             ;
278             ;
279             ;
280             ;
281             ;
282             ;
283             ;
284             ;
285             ;
286             ;
287             ;
288             ;
289             ;
290             ;
291             ;
292             ;
293             ;
294             ;
295             ;
296             ;
297             ;
298             ;
299             ;
300             ;
301             ;
302             ;
303             ;
304             ;
305             ;
306             ;
307             ;
308             ;
309             ;
310             ;
311             ;
312             ;
313             ;
314             ;
315             ;
316             ;
317             ;
318             ;
319             ;
320             ;
321             ;
322             ;
323             ;
324             ;
325             ;
326             ;
327             ;
328             ;
329             ;
330             ;
331             ;
332             ;
333             ;
334             ;
335             ;
336             ;
337             ;
338             ;
339             ;
340             ;
341             ;
342             ;
343             ;
344             ;
345             ;
346             ;
347             ;
348             ;
349             ;
350             ;
351             ;
352             ;
353             ;
354             ;
355             ;
356             ;
357             ;
358             ;
359             ;
360             ;
361             ;
362             ;
363             ;
364             ;
365             ;
366             ;
367             ;
368             ;
369             ;
370             ;
371             ;
372             ;
373             ;
374             ;
375             ;
376             ;
377             ;
378             ;
379             ;
380             ;
381             ;
382             ;
383             ;
384             ;
385             ;
386             ;
387             ;
388             ;
389             ;
390             ;
391             ;
392             ;
393             ;
394             ;
395             ;
396             ;
397             ;
398             ;
399             ;
400             ;
401             ;
402             ;
403             ;
404             ;
405             ;
406             ;
407             ;
408             ;
409             ;
410             ;
411             ;
412             ;
413             ;
414             ;
415             ;
416             ;
417             ;
418             ;
419             ;
420             ;
421             ;
422             ;
423             ;
424             ;
425             ;
426             ;
427             ;
428             ;
429             ;
430             ;
431             ;
432             ;
433             ;
434             ;
435             ;
436             ;
437             ;
438             ;
439             ;
440             ;
441             ;
442             ;
443             ;
444             ;
445             ;
446             ;
447             ;
448             ;
449             ;
450             ;
451             ;
452             ;
453             ;
454             ;
455             ;
456             ;
457             ;
458             ;
459             ;
460             ;
461             ;
462             ;
463             ;
464             ;
465             ;
466             ;
467             ;
468             ;
469             ;
470             ;
471             ;
472             ;
473             ;
474             ;
475             ;
476             ;
477             ;
478             ;
479             ;
480             ;
481             ;
482             ;
483             ;
484             ;
485             ;
486             ;
487             ;
488             ;
489             ;
490             ;
491             ;
492             ;
493             ;
494             ;
495             ;
496             ;
497             ;
498             ;
499             ;
500             ;
501             ;
502             ;
503             ;
504             ;
505             ;
506             ;
507             ;
508             ;
509             ;
510             ;
511             ;
512             ;
513             ;
514             ;
515             ;
516             ;
517             ;
518             ;
519             ;
520             ;
521             ;
522             ;
523             ;
524             ;
525             ;
526             ;
527             ;
528             ;
529             ;
530             ;
531             ;
532             ;
533             ;
534             ;
535             ;
536             ;
537             ;
538             ;
539             ;
540             ;
541             ;
542             ;
543             ;
544             ;
545             ;
546             ;
547             ;
548             ;
549             ;
550             ;
551             ;
552             ;
553             ;
554             ;
555             ;
556             ;
557             ;
558             ;
559             ;
560             ;
561             ;
562             ;
563             ;
564             ;
565             ;
566             ;
567             ;
568             ;
569             ;
570             ;
571             ;
572             ;
573             ;
574             ;
575             ;
576             ;
577             ;
578             ;
579             ;
580             ;
581             ;
582             ;
583             ;
584             ;
585             ;
586             ;
587             ;
588             ;
589             ;
590             ;
591             ;
592             ;
593             ;
594             ;
595             ;
596             ;
597             ;
598             ;
599             ;
600             ;
601             ;
602             ;
603             ;
604             ;
605             ;
606             ;
607             ;
608             ;
609             ;
610             ;
611             ;
612             ;
613             ;
614             ;
615             ;
616             ;
617             ;
618             ;
619             ;
620             ;
621             ;
622             ;
623             ;
624             ;
625             ;
626             ;
627             ;
628             ;
629             ;
630             ;
631             ;
632             ;
633             ;
634             ;
635             ;
636             ;
637             ;
638             ;
639             ;
640             ;
641             ;
642             ;
643             ;
644             ;
645             ;
646             ;
647             ;
648             ;
649             ;
650             ;
651             ;
652             ;
653             ;
654             ;
655             ;
656             ;
657             ;
658             ;
659             ;
660             ;
661             ;
662             ;
663             ;
664             ;
665             ;
666             ;
667             ;
668             ;
669             ;
670             ;
671             ;
672             ;
673             ;
674             ;
675             ;
676             ;
677             ;
678             ;
679             ;
680             ;
681             ;
682             ;
683             ;
684             ;
685             ;
686             ;
687             ;
688             ;
689             ;
690             ;
691             ;
692             ;
693             ;
694             ;
695             ;
696             ;
697             ;
698             ;
699             ;
700             ;
701             ;
702             ;
703             ;
704             ;
705             ;
706             ;
707             ;
708             ;
709             ;
710             ;
711             ;
712             ;
713             ;
714             ;
715             ;
716             ;
717             ;
718             ;
719             ;
720             ;
721             ;
722             ;
723             ;
724             ;
725             ;
726             ;
727             ;
728             ;
729             ;
730             ;
731             ;
732             ;
733             ;
734             ;
735             ;
736             ;
737             ;
738             ;
739             ;
740             ;
741             ;
742             ;
743             ;
744             ;
745             ;
746             ;
747             ;
748             ;
749             ;
750             ;
751             ;
752             ;
753             ;
754             ;
755             ;
756             ;
757             ;
758             ;
759             ;
760             ;
761             ;
762             ;
763             ;
764             ;
765             ;
766             ;
767             ;
768             ;
769             ;
770             ;
771             ;
772             ;
773             ;
774             ;
775             ;
776             ;
777             ;
778             ;
779             ;
780             ;
781             ;
782             ;
783             ;
784             ;
785             ;
786             ;
787             ;
788             ;
789             ;
790             ;
791             ;
792             ;
793             ;
794             ;
795             ;
796             ;
797             ;
798             ;
799             ;
800             ;
801             ;
802             ;
803             ;
804             ;
805             ;
806             ;
807             ;
808             ;
809             ;
810             ;
811             ;
812             ;
813             ;
814             ;
815             ;
816             ;
817             ;
818             ;
819             ;
820             ;
821             ;
822             ;
823             ;
824             ;
825             ;
826             ;
827             ;
828             ;
829             ;
830             ;
831             ;
832             ;
833             ;
834             ;
835             ;
836             ;
837             ;
838             ;
839             ;
840             ;
841             ;
842             ;
843             ;
844             ;
845             ;
846             ;
847             ;
848             ;
849             ;
850             ;
851             ;
852             ;
853             ;
854             ;
855             ;
856             ;
857             ;
858             ;
859             ;
860             ;
861             ;
862             ;
863             ;
864             ;
865             ;
866             ;
867             ;
868             ;
869             ;
870             ;
871             ;
872             ;
873             ;
874             ;
875             ;
876             ;
877             ;
878             ;
879             ;
880             ;
881             ;
882             ;
883             ;
884             ;
885             ;
886             ;
887             ;
888             ;
889             ;
890             ;
891             ;
892             ;
893             ;
894             ;
895             ;
896             ;
897             ;
898             ;
899             ;
900             ;
901             ;
902             ;
903             ;
904             ;
905             ;
906             ;
907             ;
908             ;
909             ;
910             ;
911             ;
912             ;
913             ;
914             ;
915             ;
916             ;
917             ;
918             ;
919             ;
920             ;
921             ;
922             ;
923             ;
924             ;
925             ;
926             ;
927             ;
928             ;
929             ;
930             ;
931             ;
932             ;
933             ;
934             ;
935             ;
936             ;
937             ;
938             ;
939             ;
940             ;
941             ;
942             ;
943             ;
944             ;
945             ;
946             ;
947             ;
948             ;
949             ;
950             ;
951             ;
952             ;
953             ;
954             ;
955             ;
956             ;
957             ;
958             ;
959             ;
960             ;
961             ;
962             ;
963             ;
964             ;
965             ;
966             ;
967             ;
968             ;
969             ;
970             ;
971             ;
972             ;
973             ;
974             ;
975             ;
976             ;
977             ;
978             ;
979             ;
980             ;
981             ;
982             ;
983             ;
984             ;
985             ;
986             ;
987             ;
988             ;
989             ;
990             ;
991             ;
992             ;
993             ;
994             ;
995             ;
996             ;
997             ;
998             ;
999             ;
1000            ;

```

```

261         Nil          , ; // [07] Code-block de validação do campo
262         Nil          , ; // [08] Code-block de validação When do campo
263         Nil          , ; // [09] Lista de valores permitido do campo
264         Nil          , ; // [10] Indica se o campo tem preenchimento obrigat?io
265         Nil          , ; // [11] Code-block de inicializacao do campo
266         NIL          , ; // [12] Indica se trata-se de um campo chave
267         NIL          , ; // [13] Indica se o campo pode receber valor em uma operação de update.
268     )                // [14] Indica se o campo ?virtual
269
270     oStructSAE:AddField( ;
271         "CODIGO"      , ; // [01] Titulo do campo
272         "CODIGO"      , ; // [02] Desc do campo
273         "AE_COD"      , ; // [03] Id do Field
274         "C"           , ; // [04] Tipo do campo
275         TamSX3("AE_COD")[1], ; // [05] Tamanho do campo
276         0             , ; // [06] Decimal do campo
277         Nil           , ; // [07] Code-block de validação do campo
278         Nil           , ; // [08] Code-block de validação When do campo
279         Nil           , ; // [09] Lista de valores permitido do campo
280         Nil           , ; // [10] Indica se o campo tem preenchimento obrigat?io
281         Nil           , ; // [11] Code-block de inicializacao do campo
282         NIL           , ; // [12] Indica se trata-se de um campo chave
283         NIL           , ; // [13] Indica se o campo pode receber valor em uma operação de update.
284     )                // [14] Indica se o campo ?virtual
285     EndIf
286 Return oStructSAE
287
288 //-----
289 /*{Protheus.doc} CreateGridStruct
290 Cria estrutura do Model Grid
291 @param
292 @return Struc
293 @see
294 @todo
295 @obs
296 */
297 //-----
298 Method CreateGridStruct() Class STDAAdministratorFinancial
299
300     Local oStruct      := FWFormModelStruct():New() // Estrutura
301     Local nX           := 0                        // Contador
302     Local cAlias       := "SAE"                    // Alias que criara estrutura
303     Local cX2Unico     := "AE_FILIAL+AE_COD" //X2_UNICO da tabela SAE
304
305     //Carrega informacoes da Tabela
306     SX2->( DbSetOrder( 1 ) )
307     SX2->( DbSeek( cAlias ) )
308
309     If ExistFunc('FWX2Unico')
310         cX2Unico := FWX2Unico(cAlias)
311     EndIf
312

```

```

313   oStruct:AddTable(
314     FWX2CHAVE()
315     StrTokArr( cX2Unico, ' + ' )
316     FWX2Nome(cAlias) )
317
318   //Carrega informacoes de campos
319   DbSelectArea("SX3")
320   SX3->(DbSetOrder(1))
321   SX3->(DbSeek("SAE"))
322
323   While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == cAlias
324     oStruct:AddField(
325       AllTrim( X3Titulo() )
326       AllTrim( X3Descric() )
327       AllTrim( SX3->X3_CAMPO )
328       SX3->X3_TIPO
329       SX3->X3_TAMANHO
330       SX3->X3_DECIMAL
331       Nil
332       Nil
333       StrTokArr( AllTrim( X3CBox() ),',' )
334       Nil
335       Nil
336       NIL
337       NIL
338       ( SX3->X3_CONTEXT == 'V' ) )
339     SX3->(DbSkip())
340   End
341
342   Return oStruct
343
344   /*-----
345   /*{Protheus.doc} CreateDetGridStruct
346   Cria estrutura dos Detalhes de cada parcela
347   @param
348   @return Struc
349   @see
350   @todo
351   @obs
352   /*
353   /*-----
354   Method CreateDetGridStruct() Class STDAAdministratorFinancial
355
356   Local oStruct := FWFormModelStruct():New() // Estrutura
357   Local nX := 0 // Contador
358   Local cAlias := "MEN" // Alias que criara estrutura
359   Local cX2Unico := "MEN_FILIAL+MEN_CODADM+MEN_ITEM" //X2_UNICO da tabela MEN
360
361   //Carrega informacoes da Tabela
362   SX2->( DbSetOrder( 1 ) )
363   SX2->( DbSeek( cAlias ) )
364

```

```

365 If ExistFunc('FWX2Unico')
366     cX2Unico := FWX2Unico(cAlias)
367 EndIf
368
369 oStruct:AddTable(
370     FWX2CHAVE()
371     StrTokArr( cX2Unico, ' + ' )
372     FWX2Nome(cAlias) )
373
374 //Carrega informacoes de campos
375 DbSelectArea("SX3")
376 SX3->(DbSetOrder(1))
377 SX3->(DbSeek(cAlias))
378
379 While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == cAlias
380     oStruct:AddField(
381         AllTrim( X3Titulo() )
382         AllTrim( X3Descric() )
383         AllTrim( SX3->X3_CAMPO )
384         SX3->X3_TIPO
385         SX3->X3_TAMANHO
386         SX3->X3_DECIMAL
387         Nil
388         Nil
389         StrTokArr( AllTrim( X3CBox() ),',' )
390         Nil
391         Nil
392         NIL
393         NIL
394         ( SX3->X3_CONTEXT == 'V' ) )
395     SX3->(DbSkip())
396 End
397
398 Return oStruct
399
400 //-----
401 /*{Protheus.doc} LjGetMdAdm
402 Retorno o objeto estatico oModelAdmF
403 @return oModelAdmF, Objeto, Model SAE MEN
404 */
405 //-----
406 Function LjGetMdAdm()
407 Return oModelAdmF
408 //-----
409 /*{Protheus.doc} LjSetMdAdm
410 Seta valor ao objeto estatico oModelAdmF
411 @param oModel, objeto, Model SAE e MEN carregado pelo metodo GetAllData
412 @return oModelAdmF, Objeto, Model SAE e MEN
413 */
414 //-----
415 Function LjSetMdAdm(oModel)
416 oModelAdmF := oModel

```

417 return

Componentes/Payment/STDChgPay.prw

35 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2
3 //-----
4 /*{Protheus.doc} STDChangePay
5 Retorna uma string com as formas de pagamento cadastradas no DE/PARA
6 @param      cTpForm,cChgPay
7 @return     cChgPay
8 */
9 //-----
10 Function STDChgPay(cTpForm, cChgPay)
11 Local aArea      := GetArea()
12 Local aRet       := {}
13 Local cFilMHI    := ""
14 Local cRet       := ""
15 Local cSep       := ";"
16 Local nCount     := 0
17
18 MHI->(DbSetOrder(1))
19 If MHI->(DbSeek(xFilial("MHI")+AllTrim(cTpForm)))
20     cFilMHI := MHI->MHI_FILIAL
21     While !MHI->(EOF()) .AND. cFilMHI == MHI->MHI_FILIAL .AND. AllTrim(cTpForm) == AllTrim(MHI->MHI_FRMPRI)
22         Add(aRet,AllTrim(MHI->MHI_FRMTRO) + " - " + POSICIONE( "SX5", 1, xFilial( "SX5" ) + "24"+ MHI->MHI_FRMTRO, "X5_DESCRI" ))
23         MHI->(DbSkip())
24     End
25     If Len(aRet) > 0
26         cChgPay := AllTrim(aRet[1])
27         For nCount := 2 To Len(aRet)
28             cChgPay += cSep + AllTrim(aRet[nCount])
29         Next nCount
30     EndIf
31 EndIf
32
33 RestArea(aArea)
34
35 Return cChgPay

```

Componentes/Payment/STDOptionsPayments.prw

322 linhas após remoção de metadados de autoria/data

```

1 #include 'totvs.ch'
2 // Interface de array para ser enviada para o DAO
3 #DEFINE TAMANHOPAY 3 //
4 #DEFINE TIPO 1 // Obrigatório
5 #DEFINE DESCRICAO 2 // Opcional
6 #DEFINE CODIGO 3 // Opcional
7
8 Function STDAOptionsPayments() ; Return
9
10 //-----
11 /*{Protheus.doc} STDAOptionsPayments
12 Classe opções de pagamento
13 @param
14 @return
15 @obs
16 @sample
17 */
18 //-----
19 Class STDAOptionsPayments
20
21     Data oCache
22     Data aOpc
23
24     // Construtor
25     Method STDAOptionsPayments(oCache)
26
27     // Publico
28     Method GetAllData()
29     Method Add(aOpc)
30
31     // Privado
32     Method GetModel()
33     Method CreateMasterStruct()
34     Method CreateGridStruct()
35
36 EndClass
37
38 //-----
39 /*{Protheus.doc} STDAOptionsPayments
40 Metodo creator
41 @param oCache
42 @return
43 @obs
44 @sample
45 */
46 //-----
47 Method STDAOptionsPayments(oCache) Class STDAOptionsPayments
48     Self:oCache := oCache
49 Return Self
50
51 //-----
52 /*{Protheus.doc} GetModel

```

```

53 Cria e retorna o model
54 @param   oCache
55 @return  oModel
56 @obs
57 @sample
58 */
59 //-----
60 Method GetModel() Class STDAOptionsPayments
61
62 Local oMasterStr      := Nil // Estrutura SLQ
63 Local oGridStr        := Nil // Estrutura SLR
64 Local oModel          := Nil // Model
65
66 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
67 //³Monta estrutura de tabela de cabecario³
68 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAÜ
69 oMasterStr := Self.CreateMasterStruct()
70
71 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
72 //³Monta estrutura de tabela de Itens³
73 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAÜ
74 oGridStr := Self.CreateGridStruct()
75
76 //UAAAAAAAAAAAAAAAAAAAA;
77 //³Instacia Objeto³
78 //AAAAAAAAAAAAAAAAAAAÜ
79 oModel := MPFormModel().New( "SE4", /*bPreValidacao*/, /*bPosValidacao*/, /*{ |oMdl| xFRT80MVCGR(oMdl)}*/ , /*bCancel*/ )
80
81 oModel.AddFields(    "MasterStr"           , /*cOwner*/, oMasterStr )
82 oModel.AddGrid(     "GridStr"              , "MasterStr", oGridStr, /*B*/ , /*bLinePost*/, /*bPreVal*/, /*bPosVal*/, /*BLoad*/ )
83 oModel.SetRelation( "GridStr"              , { { "E4_FILIAL", 'xFilial( "SE4" )' }, { "E4_CODIGO", "E4_CODIGO" } }, "AE_FILIAL + E4_CODIGO" )
84
85 Return oModel
86
87 //-----
88 /*/{Protheus.doc} Add
89 Atualiza propriedade aOpc
90 @param   aOpc
91 @return
92 @obs
93 @sample
94 */
95 //-----
96 Method Add(aOpc) Class STDAOptionsPayments
97
98 AAdd(Self:aOpc, {})
99 // Executar Por aClone para não criar referencia
100 Self:aOpc[Len(Self:aOpc)] := AClone(aOpc)
101
102 Return
103
104 //-----

```

```

105 *//{Protheus.doc} GetAllData
106 Incluir informacoes no model
107 @param
108 @return oNewModel
109 @obs
110 @sample
111 */
112 //-----
113 Method GetAllData() Class STDAOptionsPayments
114
115 Local oNewModel      := Nil           // Retorno do Model
116 Local nX             := 0            // Contador do Struct
117 Local cCampo         := ""           // Campo da tabela que incluirá novo Registros
118 Local cCacheID       := "STDAOptionsPayments_GetAllData" // Id de Cache
119 Local nLine          := 0
120 Local lFirstLine     := .T.          // Primeiro registro incluso
121
122 If Self:oCache != Nil .And. Self:oCache:Contains( cCacheID )
123   oNewModel := Self:oCache:Get( cCacheID )
124 EndIf
125
126 If oNewModel == Nil
127
128   oNewModel := Self:GetModel() // Chama a estrutura de dados
129
130   oNewModel:SetOperation( 3 )
131   oNewModel:SetDescription("Opcao de Pagamento")
132
133   oNewModel:Activate()
134
135
136   // Dados
137   oModelSlq := oNewModel:GetModel("MasterStr")
138   oModelSlq:SetValue("OPP_FILIAL" , xFilial("OPP"))
139   oModelSlq:SetValue("OPP_CODIGO" , SAE->AE_COD )
140
141   // Carrega todas as Opcoes na 1ª Linha
142   oModSlr := oNewModel:GetModel("GridStr")
143
144   aTestStr := oModSlr:GetStruct()
145
146   // Carrega todas as Adm
147   For nX := 1 to Len(Self:a0pc)
148
149     If !lFirstLine
150       nLine := oModSlr:AddLine()
151     EndIf
152
153     oModSlr:SetValue("OPP_CODIGO" , Self:a0pc[nX][CODIGO])
154     oModSlr:SetValue("OPP_DESCRI" , Self:a0pc[nX][DESCRICA0])
155     oModSlr:SetValue("OPP_TIPO" , Self:a0pc[nX][TIPO])
156

```

```

157         lFirstLine := .F.
158
159     Next nX
160
161     ConOut("Não pegou do cache")
162
163     If Self:oCache != Nil
164         Self:oCache:Put( cCacheID, oNewModel, 60 )
165     EndIf
166 Else
167     ConOut("Pegou do cache")
168 EndIf
169
170 Return oNewModel
171
172 //-----
173 /*{Protheus.doc} CreateMasterStruct
174 Cria a estrutura do model master
175 @param
176 @return oStruct
177 @obs
178 @sample
179 */
180 //-----
181 Method CreateMasterStruct() Class STDAOptionsPayments
182
183 Local oStruct := FWFormModelStruct():New() // Estrutura
184 Local cAlias := "SE4" // Alias que criara estrutura
185 Local cX2Unico := "E4_FILIAL+E4_CODIGO" //X2_UNICO da tabela SE4
186
187 //Carrega informacoes da Tabela
188 SX2->( DbSetOrder( 1 ) )
189 SX2->( DbSeek( cAlias ) )
190
191 If ExistFunc('FWX2Unico')
192     cX2Unico := FWX2Unico(cAlias)
193 EndIf
194
195 oStruct:AddTable(
196     FWX2CHAVE(), ; // [01] Alias da tabela
197     StrTokArr( cX2Unico, '+' ) , ; // [02] Array com os campos que correspondem a primary key
198     FWX2Nome(cAlias) ) // [03] Descrição da tabela
199
200 oStruct:AddField(
201     "FILIAL", ; // [01] Titulo do campo
202     "FILIAL", ; // [02] Desc do campo
203     "OPP_FILIAL", ; // [03] Id do Field
204     "C", ; // [04] Tipo do campo
205     FWSizeFilial(), ; // [05] Tamanho do campo
206     0, ; // [06] Decimal do campo
207     Nil, ; // [07] Code-block de validação do campo
208     Nil, ; // [08] Code-block de validação When do campo

```

```

209 Nil , ; // [09] Lista de valores permitido do campo
210 Nil , ; // [10] Indica se o campo tem preenchimento obrigatório
211 Nil , ; // [11] Code-block de inicializacao do campo
212 NIL , ; // [12] Indica se trata-se de um campo chave
213 NIL , ; // [13] Indica se o campo pode receber valor em uma operação de update.
214 ) // [14] Indica se o campo é virtual
215
216 oStruct:AddField( ;
217 "CODIGO" , ; // [01] Titulo do campo
218 "CODIGO" , ; // [02] Desc do campo
219 "OPP_CODIGO" , ; // [03] Id do Field
220 "C" , ; // [04] Tipo do campo
221 TamSX3("E4_CODIGO")[1], ; // [05] Tamanho do campo
222 0 , ; // [06] Decimal do campo
223 Nil , ; // [07] Code-block de validação do campo
224 Nil , ; // [08] Code-block de validação When do campo
225 Nil , ; // [09] Lista de valores permitido do campo
226 Nil , ; // [10] Indica se o campo tem preenchimento obrigatório
227 Nil , ; // [11] Code-block de inicializacao do campo
228 NIL , ; // [12] Indica se trata-se de um campo chave
229 NIL , ; // [13] Indica se o campo pode receber valor em uma operação de update.
230 ) // [14] Indica se o campo é virtual
231
232 Return oStruct
233
234 //-----
235 /*{Protheus.doc} CreateGridStruct
236 Cria a estrutura do model grid
237 @param
238 @return oStruct
239 @obs
240 @sample
241 */
242 //-----
243 Method CreateGridStruct() Class STDAOptionsPayments
244
245 Local oStruct := FWFormModelStruct():New() // Estrutura
246 Local nX := 0 // Contador
247 Local cValid := '' // Validacao para o bValid
248 Local bValid := {} // Validades ção X3
249 Local bWhen := {} // Em quanto do x3
250 Local bRelac := {} // Relacao do x3
251 Local cAlias := "SE4" // Alias que criara estrutura
252 Local cX2Unico := "E4_FILIAL+E4_CODIGO" //X2_UNICO da tabela SE4
253
254 //Carrega informacoes da Tabela
255 SX2->( DbSetOrder( 1 ) )
256 SX2->( DbSeek( cAlias ) )
257
258 If ExistFunc('FWX2Unico')
259 cX2Unico := FWX2Unico(cAlias)
260 EndIf

```

```

261
262 oStruct:AddTable(          ;
263 FWX2CHAVE()                , ; // [01] Alias da tabela
264 StrTokArr( cX2Unico, '+' ) , ; // [02] Array com os campos que correspondem a primary key
265 FWX2Nome(cAlias)           // [03] Descrição da tabela
266
267 //Carrega informacoes de campos
268 DbSelectArea("SX3")
269 SX3->(DbSetOrder(1))
270 SX3->(DbSeek("SE4"))
271
272
273 oStruct:AddField(          ;
274 "CODIGO"                   , ; // [01] Titulo do campo
275 "CODIGO"                   , ; // [02] Desc do campo
276 "OPP_CODIGO"               , ; // [03] Id do Field
277 "C"                        , ; // [04] Tipo do campo
278 TamSX3("E4_CODIGO")[1],   , ; // [05] Tamanho do campo
279 0                           , ; // [06] Decimal do campo
280 Nil                        , ; // [07] Code-block de validação do campo
281 Nil                        , ; // [08] Code-block de validação When do campo
282 Nil                        , ; // [09] Lista de valores permitido do campo
283 Nil                        , ; // [10] Indica se o campo tem preenchimento obrigatório
284 Nil                        , ; // [11] Code-block de inicializacao do campo
285 NIL                       , ; // [12] Indica se trata-se de um campo chave
286 NIL                       , ; // [13] Indica se o campo pode receber valor em uma operação de update.
287 )                          // [14] Indica se o campo é virtual
288
289 oStruct:AddField(          ;
290 "DESCRICAO"                , ; // [01] Titulo do campo
291 "DESCRICAO"                , ; // [02] Desc do campo
292 "OPP_DESCRI"               , ; // [03] Id do Field
293 "C"                        , ; // [04] Tipo do campo
294 TamSX3("E4_DESCRI")[1],   , ; // [05] Tamanho do campo
295 0                           , ; // [06] Decimal do campo
296 Nil                        , ; // [07] Code-block de validação do campo
297 Nil                        , ; // [08] Code-block de validação When do campo
298 Nil                        , ; // [09] Lista de valores permitido do campo
299 Nil                        , ; // [10] Indica se o campo tem preenchimento obrigatório
300 Nil                        , ; // [11] Code-block de inicializacao do campo
301 NIL                       , ; // [12] Indica se trata-se de um campo chave
302 NIL                       , ; // [13] Indica se o campo pode receber valor em uma operação de update.
303 )                          // [14] Indica se o campo é virtual
304
305
306 oStruct:AddField(          ;
307 "TIPO"                     , ; // [01] Titulo do campo
308 "TIPO"                     , ; // [02] Desc do campo
309 "OPP_TIPO"                 , ; // [03] Id do Field
310 "C"                        , ; // [04] Tipo do campo
311 2, ; // [05] Tamanho do campo
312 0                           , ; // [06] Decimal do campo

```

```
313         Nil          , ; // [07] Code-block de validação do campo
314         Nil          , ; // [08] Code-block de validação When do campo
315         Nil          , ; // [09] Lista de valores permitido do campo
316         Nil          , ; // [10] Indica se o campo tem preenchimento obrigatório
317         Nil          , ; // [11] Code-block de inicializacao do campo
318         NIL          , ; // [12] Indica se trata-se de um campo chave
319         NIL          , ; // [13] Indica se o campo pode receber valor em uma operação de update.
320         )            , // [14] Indica se o campo é virtual
321
322 Return oStruct
```

Componentes/Payment/STDPayCard.prw

108 linhas após remoção de metadados de autoria/data

[illegible]

```

53 Local lArredondar := SuperGetMV( "MV_LJINSAR",,.F. ) // Parâmetro para habilitar doação ao Inst. Arredondar
54 Local lAplicaTaxa := SuperGetMv('MV_LJTEFIA',,2)=1 // Aplica desconto da taxa administrativa na doação
55 Local nValDoacao := STBGetInsArr() // Valor da doação ao Ins. Arredondar
56 Local cFilSAE := xFilial("SAE") // Filial da tabela SAE
57 Local nTamAE_COD := TamSx3("AE_COD")[1] // Tamanho do campo AE_COD
58 Local nPctTaxAdm := 0 // % da Taxa da Administradora Financeira
59
60 Default cForma := ""
61 Default cAdmFin := ""
62 Default nValDoacao := 0
63
64 If lArredondar .AND. lAplicaTaxa .AND. nValDoacao > 0
65
66     DbSelectArea("SAE")
67     DbSetOrder(1)
68     If SAE->(MSeek(cFilSAE+Left(cAdmFin,nTamAE_COD))) .AND. AllTrim(cForma) $ "CC/CD"
69         nPctTaxAdm := SAE->AE_TAXA
70         STBSetDInsArr(nValDoacao-(nValDoacao*(nPctTaxAdm/100))) // Aplicar desconto
71     Else
72         STBSetDInsArr(nValDoacao)
73     EndIf
74
75 EndIf
76
77 RestArea(aArea)
78
79 Return nil
80
81 /*
82 {Protheus.doc} STDGetDias
83 Retorna os dias para pagamento da administradora selecionada
84
85 @param
86 @return Dias para pagamento da administradora AE_DIAS
87 @obs
88 @sample
89 */
90 Function STDGetDias(cChave)
91
92 Local aArea := GetArea() //Guardo Area atual
93 Local aAreaSAE := SAE->(GetArea()) //Guardo a Area da SAE
94 Local nRet := 0 //Retorno da função
95
96 Default cChave := "" //Chave de busca
97
98 DbSelectArea("SAE")
99 SAE->(DbSetOrder(1))//AE_FILIAL+AE_COD
100
101 If SAE->(DbSeek(xFilial("SAE") + cChave))
102     nRet := SAE->AE_DIAS
103 EndIf
104

```

```
105 RestArea(aAreaSAE)
106 RestArea(aArea)
107
108 Return nRet
```

Componentes/Payment/STDPayCdPg.prw

95 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protoheus.doc} STDCondPg
6 Consulta no banco as condicoes de pagamento disponiveis
7
8 @param
9
10 @param      cCodCliLoj, Caractere,Codigo + Loja do Cliente da venda
11 @return     aCondicoes, Array, Array de condicoes de pagamentos.
12
13 @obs
14 @sample
15 */
16 //-----
17 Function STDCondPg(cCodCliLoj)
18
19 Local aArea      := GetArea()          // Armazena alias corrente
20 Local aCondicoes := {}                // Array de condicoes de pagamentos
21 Local cFormPgNot := ""                // Formas de Pagamentos a serem desconsideradas
22
23 Default cCodCliLoj := ""              //Codigo + Loja do Cliente da venda
24
25 //Se for Cliente Padrão, não carrega a condição de pagamento do tipo (FI=Financiado)
26 If !Empty(cCodCliLoj) .And. (AllTrim(cCodCliLoj) == AllTrim(SuperGetMV("MV_CLIPAD"))+AllTrim(SuperGetMV("MV_LOJAPAD")))
27     cFormPgNot := "|" + PadR("FI",TamSX3("E4_FORMA")[1]) + "|"
28 EndIf
29
30 //Carrega com as condicoes gravadas no SE4
31 DbSelectArea("SE4")
32 DbSeek(xFilial("SE4"))
33 While !SE4->(Eof()) .And. SE4->E4_FILIAL == xFilial("SE4")
34     If ExistFunc("STIFpgtBlq") .And. STIFpgtBlq(AllTrim(SE4->E4_FORMA))
35         cFormPgNot += PadR(SE4->E4_FORMA,TamSX3("E4_FORMA")[1]) + "|"
36     EndIf
37     If !Empty(cFormPgNot) .And. SE4->E4_FORMA $ cFormPgNot
38         LjGrvLog( NIL, "Venda p/ Cliente padrão, não carrega a condição de pagamento do tipo (FI=Financiado). Foi desconsiderada a Cond. Pagto (SE4): " + SE4->E4_CODIGO)
39         SE4->(dbSkip())
40     Loop
41     EndIf
42     If SE4->E4_TIPO <> "9"
43         Add(aCondicoes, {SE4->E4_CODIGO, SubStr( SE4->E4_DESCRI, 1, 20 ), SE4->E4_FORMA, SE4->E4_TIPO, SE4->E4_DESCFIN, SE4->E4_ACRSFIN, SE4->E4_ACRES})
44     EndIf
45     SE4->(dbSkip())
46 End
47
48 RestArea(aArea)
49
50 Return aCondicoes
51
52 //-----

```

```

53 /*/{Protheus.doc} STDCondPg
54 Consulta no banco a condicao de pagamento padrao
55
56 @return      aCondicoes - Array de condicoes de pagamentos
57 */
58 //-----
59 Function STDCondPad(cCod)
60
61 Local aArea      := GetArea()    // Armazena alias corrente
62 Local aCondicoes := {}          // Array de condicoes de pagamentos
63 Local aDiasPgChck := {}         // Dias da condicao de pagamento
64
65 Default cCod      := ""
66
67 //Carrega com as condicoes gravadas no SE4
68 DbSelectArea("SE4")
69 DbSetOrder(1) //E4_FILIAL+E4_CODIGO
70 If DbSeek(xFilial("SE4")+cCod)
71     If SE4->E4_TIPO <> "9" .And. ExistFunc("STIFpgtBlq") .And. !STIFpgtBlq(AllTrim(SE4->E4_FORMA))
72         aDiasPgChck := StrTokArr( SE4->E4_COND, "," )
73         Add(aCondicoes,{SE4->E4_CODIGO,SubStr(E4_DESCRI,1,20),E4_FORMA,E4_TIPO,E4_DESCFIN,E4_ACRSFIN,E4_ACRES,aDiasPgChck })
74     EndIf
75 EndIf
76
77 RestArea(aArea)
78
79 Return aCondicoes
80
81 //-----
82 /*/{Protheus.doc} STDSCDPG
83 Persiste o campo L1_CONDPG
84
85 @param
86 @return      aCondicoes - Array de condicoes de pagamentos
87 @obs
88 @sample
89 */
90 //-----
91 Function STDSCondPG(cInfo)
92
93 STDSPBasket("SL1", "L1_CONDPG", cInfo)
94
95 Return Nil

```

Componentes/Payment/STDPayFinancial.prw

77 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

Componentes/Payment/STD PayMultiNeg.PRW

248 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

```

105 // Limpas as variáveis do Label
106 aLblPayForms := {"", "", "", "", "", "", ""}
107
108 DbSelectArea("MBT")
109 MBT->( DbSetOrder(1) )//MBT_FILIAL+MBT_CODIGO+MBT_FORMPG+MBT_ADM
110
111 aTypeDown := {}
112
113 If DbSeek(xFilial("MBT")+cMBTCodigo+cMBTFormPg)
114
115     DbSelectArea("SAE")
116     SAE->( dbSetOrder( 1 ) ) //AE_FILIAL+AE_COD
117
118     While !MBT->(EOF()) .AND. xFilial("MBT")+cMBTCodigo+cMBTFormPg == MBT->MBT_FILIAL+MBT->MBT_CODIGO+MBT->MBT_FORMPG
119
120         If Val(cMBTParcDe) == MBT->MBT_MINPAR .AND. Val(cMBTParcAt) == MBT->MBT_PARCEL
121
122             cBckAdmFin := ""
123             If !EMPTY(MBT->MBT_ADM)
124                 If SAE->( DbSeek(xFilial("SAE")+MBT->MBT_ADM) ) .AND. Upper(Substr(SAE->AE_DESC,1,nTamAEDesc)) == cDescAdmFin
125                     cBckAdmFin := Upper(Substr(SAE->AE_DESC,1,15))
126                 EndIf
127             EndIf
128
129             // Se achou a adm. Financeira equivalente ao conteudo do objeto oListPayForms clicado
130             // preencho o restante
131             If (!EMPTY(MBT->MBT_ADM) .AND. !EMPTY(cBckAdmFin)) .OR. (EMPTY(MBT->MBT_ADM))
132                 aLblPayForms[01] := Upper(MBT->MBT_FORMPG)
133                 aLblPayForms[03] := STRZERO(MBT->MBT_MINPAR,2) + " A " + STRZERO(MBT->MBT_PARCEL,2)
134                 aLblPayForms[04] := Alltrim(Transform(MBT->MBT_TXJURO, "@E 99.99"))
135
136                 If MBT->MBT_VALENT <> 0
137                     aLblPayForms[05] := Alltrim(Transform(MBT->MBT_VALENT, cPicDefault))
138                 ElseIf MBT->MBT_PERENT <> 0
139                     aLblPayForms[05] := Alltrim(Transform(nSaleVal*(MBT->MBT_PERENT/100), cPicDefault))
140                 Else
141                     aLblPayForms[05] := Alltrim(Transform(0, cPicDefault))
142                 EndIf
143
144                 If !EMPTY(MBT->MBT_FPENT) .AND. aLblPayForms[05] <> Alltrim(Transform(0, cPicDefault))
145                     aTypeDown := StrToKarr(MBT->MBT_FPENT, ";")
146                     DbSelectArea("SX5")
147                     DbSetOrder(1)
148                     For nFor := 1 To Len(aTypeDown)
149                         If DbSeek(xFilial("SX5")+ "24"+ aTypeDown[nFor])
150                             aTypeDown[nFor] += " - " + SX5->X5_DESCRI
151                         Endif
152                     Next nFor
153                 EndIf
154
155                 aLblPayForms[02] := cBckAdmFin
156                 aLblPayForms[06] := MBT->MBT_ADM

```

```

157         aLblPayForms[07]      := {MBT->MBT_JUR1PA, MBT->MBT_CAR1PA, MBT->MBT_LIM1PA}
158     EndIf
159
160 EndIf
161
162     MBT->(DbSkip())
163
164 EndDo
165 EndIf
166
167 RestArea(aArea)
168
169 Return NIL
170
171 //-----
172 /*{Protheus.doc} STDVldMn
173 Verifica se os itens de vendas estão na Multi Negociação.
174
175 @param      aItens - Array contendo os itens de venda
176 @return
177 @obs
178 @sample
179 */
180 //-----
181
182 Function STDVldMn(aItens)
183 Local nX      := 0 //Variavel de loop
184 Local aAreaMEK := {} //Guarda area da MEK
185 Local aAreaMBS := {} //Guarda area da MBS
186 Local cGrupoPrd := "" //Armazena o conteudo do B1_GRUPO
187 Local lVldProd  := .T. //Validacao das tabelas MEK e MBS
188
189 Default aItens := {}
190
191 If Len(aItens) > 0
192     aAreaMEK := MEK->(Getarea())
193     DbSelectArea("MEK")
194     DbSetorder(3) //MEK_FILIAL+MEK_PRODUT
195
196     aAreaMBS := MBS->(Getarea())
197     DbSelectArea("MBS")
198     DbSetorder(3) //MBS_FILIAL+MBS_GRPROD
199
200     SB1->(DbSetorder(1)) //B1_FILIAL+B1_COD
201
202     For nX := 1 to Len(aItens)
203         If !MEK->(DbSeek(xFilial("MEK") + aItens[nX][2]))
204             //Caso o item não esteja como produto procuro como grupo.
205
206             If SB1->(DbSeek(xFilial("SB1") + aItens[nX][2]))
207                 cGrupoPrd := SB1->B1_GRUPO
208             EndIf

```

```

209
210     lVldProd    := .F.
211     MBS->(DbgoTop())
212     While MBS->(!EOF())
213         If !Empty(cGrupoPrd) .And. xFilial("MBS") + cGrupoPrd == MBS->MBS_FILIAL + MBS->MBS_GRPROD //Verifico o Grupo do produto
214             lVldProd    := .T.
215             Exit
216         EndIf
217         If xFilial("MBS") + AllTrim(aItens[nX][2]) == MBS->MBS_FILIAL + AllTrim(MBS->MBS_PRODUT) //Verifico o produto
218             lVldProd    := .T.
219             Exit
220         EndIf
221         MBS->(DbSkip())
222     End
223     If !lVldProd // Se não achou na MBS busco na MEK por grupo de produto
224         If !Empty(cGrupoPrd)
225             MEK->(DbgoTop())
226             While MEK->(!Eof())
227                 If xFilial("MEK") + cGrupoPrd == MEK->MEK_FILIAL + MEK->MEK_GRPROD //Verifico o Grupo do produto
228                     lVldProd    := .T.
229                     Exit
230                 EndIf
231                 MEK->(DbSkip())
232             End
233         EndIf
234         If !lVldProd // Se nao estiver na MEK e na MBS deve zerar os itens
235             Exit
236         EndIf
237     EndIf
238 EndIf
239 Next nX
240 RestArea(aAreaMEK)
241 RestArea(aAreaMBS)
242
243 If !lVldProd // Se tiver itens nao cadastrados Limpo os Itens
244     aItens := {}
245 EndIf
246 EndIf
247
248 Return

```

Componentes/Payment/STDPayment.prw

72 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2
3 //-----
4 /*{Protheus.doc} STDGrvMHX
5 Função responsável por realizar a gravação e exclusão dos dados nas tabelas MHJ e MHK
6
7 @param      aMHJ - Dados para a gravação da tabela
8 @param      aMHK - Dados para a gravação da tabela
9 @param      cChvDel - Chave para deletar os registros
10 @return
11 @obs
12 @sample
13 */
14 //-----
15 Function STDGrvMHX(aMHJ,aMHK,cChvDel)
16 Local nI := 0
17 local nX := 0
18
19 If !Empty(cChvDel)
20     DbSelectArea("MHJ")
21     MHJ->(DbSetOrder(1))//MHJ_FILIAL+MHJ_PRXTIT+MHJ_NUMTIT+MHJ_PARTIT
22     If MHJ->( DbSeek( cChvDel ))
23         Begin Transaction
24
25             RecLock("MHJ",.F.)
26             MHJ->(DbDelete())
27             MHJ->(MsUnlock())
28             //Exclui MHK_LOTE atrelada a MHJ_LOTE (A exclusão sera chamada para cada titulo)
29             DbSelectArea("MHK")
30             MHK->(DbSetOrder(1)) //MHK_FILIAL+MHK_LOTE+MHK_BANCO+MHK_DATMOV+MHK_NUMMOV
31
32             If MHK->(DbSeek( xFilial("MHK") + MHJ->MHJ_LOTE))
33                 While MHK->(!Eof()) .AND. xFilial("MHK") + MHK->MHK_LOTE == xFilial("MHJ") + MHJ->MHJ_LOTE
34                     RecLock("MHK",.F.)
35                     MHK->(DbDelete())
36                     MHK->(MsUnlock())
37                     MHK->(DbSkip())
38                 End
39             Endif
40
41         End Transaction
42     EndIf
43 Else
44
45     Begin Transaction
46
47     If !Empty(aMHJ) .AND. !Empty(aMHK)
48         For nX := 1 To Len(aMHJ)
49             RecLock("MHJ",.T.)
50             For nI := 1 To Len(aMHJ[nX])
51                 REPLACE MHJ->&( aMHJ[nX][nI][1] ) WITH aMHJ[nX][nI][2]
52             Next nI

```

```
53         MHJ->(MsUnlock())
54     Next nX
55
56     For nX := 1 To Len(aMHK)
57         RecLock("MHK",.T.)
58         For nI := 1 To Len(aMHK[nX])
59             REPLACE MHK->&( aMHK[nX][nI][1] )    WITH aMHK[nX][nI][2]
60         Next nI
61         MHK->(MsUnlock())
62     Next nX
63
64     Else
65         LjGrvLog("STDGrvMHX" ,"Dados para a gravação das tabelas MHJ e MHK não foram informados.")
66     EndIf
67
68     End Transaction
69
70 Endif
71
72 Return
```

Componentes/Payment/STWChkTef.prw

70 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 //-----
6 /*{Protheus.doc} STWChkTef
7 Verifica em qual qual o tipo de transação deve ser executada
8
9 @param      cType - Tipo de transacao
10 @return     lRet - Retorna se o Tipo esta configurado
11 @obs
12 @sample
13 */
14 //-----
15 Function STWChkTef( cType )
16
17 Local lRet      := .F.           //Variavel de retorno
18 Local oTEF20    := STBGetTEF()   //Objeto do TEF já instanciado
19
20 Default cType := ""
21
22 LjGrvLog("STWChkTef"," Inicio - Verifica em qual qual o tipo de transação deve ser executada ", cType)
23
24 ParamType 0 Var      cType  As Character      Default      ""
25
26 If oTEF20 <> Nil
27
28     Do Case
29
30         Case cType $ "CC|CD"
31             If oTEF20:oConfig:ISCCCD()
32                 lRet := .T.
33             Else
34                 lRet := .F.
35             EndIf
36         Case cType == "CH"
37             If oTEF20:oConfig:ISCheque()
38                 lRet := .T.
39             Else
40                 lRet := .F.
41             EndIf
42         Case cType == "RC"
43             If oTEF20:oConfig:ISRecCel()
44                 lRet := .T.
45             Else
46                 lRet := .F.
47             EndIf
48         Case cType == "CB"
49             If oTEF20:oConfig:ISCB()
50                 lRet := .T.
51             Else
52                 lRet := .F.

```

```
53         EndIf
54     Case IsPDOrPix(cType) // PD e PIX
55         If oTEF20:oConfig:ISPgtoDig()
56             lRet := .T.
57         Else
58             lRet := .F.
59         EndIf
60     EndCase
61
62 Else
63     LjGrvLog("STWChkTef", "Variavel oTEF20 Nula!")
64
65 EndIf
66
67 LjGrvLog("STWChkTef"," Verifica em qual qual o tipo de transação deve ser executada - Fim", lRet)
68
69 Return lRet
70
```

Componentes/Payment/STWCurrency.prw

20 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STBQuantity
6 Seta quantidade do item
7 @param  nItemQuant      Quantidade do Item
8 @return Nil
9 @obs
10 @sample
11 */
12 //-----
13 Function STWCurrency()
14
15 /*/
16     Define a Moeda Padrão como Moeda Corrente
17 /*/
18 STBSetCurrency( STBDefCurrency() )
19
20 Return
```

Componentes/Payment/STWPayCard.prw

320 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWPAYCARD.CH"
4
5 static lIsPaf      := STBIsPAF()
6 Static aIdPgtoMfe  := {} //Id's de pagamento enviados ao Validador Fiscal MFe
7 Static lMFE        := IIF( ExistFunc("LjUsaMfe"), LjUsaMfe(), .F. ) //Se utiliza MFE
8 Static lLjEnvPgto  := ExistFunc("LjEnvPgto")
9 Static lIntegrador := IIF( ExistFunc("LjUsaIntgr"),LjUsaIntgr(), .T.) //Se utiliza MFe com Integrador
10
11 //-----
12 /*{Protheus.doc} STWTypeTran
13 Verifica em qual qual o tipo de transação deve ser executada
14
15 @param
16 @return      lRet - Se executou corretamente
17 @obs
18 @sample
19 */
20 //-----
21 Function STWTypeTran(oMdlMst, oTEF20, cTypeCard, nParc, lContTef)
22
23 Local oDados      := Nil //Dados para a transacao
24 Local oRetTran    := Nil //Retorno da transacao
25 Local lRet        := .F. //Retorno da funcao
26 Local cCupom      := "" //Armazena o numero do cupom
27 Local lUltTran    := IIF(ValType(oMdlMst) == '0', STBCalcSald("2", oMdlMst:GetValue("L4_VALOR")), .T.) //E ultima tansacao?
28 Local oTotal      := STFGetTot() //Objeto total
29 Local nTotal      := oTotal:GetValue("L1_VLRTOT") //Valor total da venda
30 Local nValor      := IIF(ValType(oMdlMst) == '0', oMdlMst:GetValue("L4_VALOR"), nTotal)+STBGetInsArr() //Valor da transacao
31 Local lTefManu    := SuperGetMv('MV_TEFMANU',,.F.) //Se utiliza POS no lugar do TEF quando ocorrer algum problema
32 Local lFinVendaAut := SuperGetMv('MV_LJFCVDA',,1) == 1 //Se finaliza venda automaticamente caso saldo esteja zerado
33 Local oPanPayment := STIGetPan() //Gera uma copia do oPanPayment original
34 Local aPayment    := {} //Armazena informações sobre pagamento, Ex.: Valor e parcela
35 Local aLtt        := {.T.,""}
36 Local aDadosCli   := {"","","","","","","",""} // {"cod","email","cpf/cnpj","nome","rua","cidade","cep","estado"}
37
38 Default oMdlMst    := Nil
39 Default oTEF20     := Nil
40 Default cTypeCard  := ""
41 Default nParc      := 1
42 Default lContTef   := .F.
43
44 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Inicio STWTypeTran. Chamada por: " + ProcName(1) )
45 //Parcela tem que ser 1 - mesmo se for a vista
46 If nParc < 1
47     nParc := 1
48 EndIf
49
50 If nValor > 0
51
52     If !lContTef .AND. ((STWChkTef("CC") .AND. cTypeCard == "CC") .OR. (STWChkTef("CD") .AND. cTypeCard == "CD")) .OR. ( STWChkTef("PD") .AND. IsPD0rPix(cTypeCard) )

```

```

53
54     If !Empty(cCupom := STBRetCup())
55
56         If ExistFunc("STGetLTT")
57             aLtt := STGetLTT()
58         EndIf
59
60     If aLtt[1]
61         nParc := IIF(ValType(oMdlMst) == '0', oMdlMst.GetValue("L4_PARC"), nParc)
62
63         If lMFE .And. lIntegrador .And. lLjEnvPgto
64             LjMsgRun("Enviando pagamento para o VFP-e..." ,,{ || aAdd(aIdPgtoMfe,LjEnvPgto(nValor,.T.)) })
65         EndIf
66
67     Do Case
68
69         // Cartao de Credito (A vista e Parcelado)
70         Case cTypeCard == "CC"
71             oDados      := LJCDadosTransacaoCredito():New(nValor, Val(cCupom), Date(), Time(), "CC", "", "", .F., Nil, lUltTran)
72
73             If nParc > 1
74                 oDados:nParcela := nParc
75                 oRetTran := oTEF20:Cartao():Credito(oDados)
76             else
77                 oRetTran := oTEF20:Cartao():Credito(oDados)
78                 nParc := oDados:nParcela
79             EndIf
80
81         // Cartao de debito (A vista, Parcelado, Pre-datado)
82         Case cTypeCard == "CD"
83             oDados := LJCDadosTransacaoDebito():New(nValor, Val(cCupom), Date(), Time(), "CD", "", "", .F., 1, lUltTran)
84             oRetTran := oTEF20:Cartao():Debito(oDados)
85
86         // Pagamentos digitais
87         Case cTypeCard == "PD"
88             If ExistFunc("GetDdClTrn")
89                 aDadosCli := GetDdClTrn(STDGPBasket('SL1','L1_CLIENTE'), STDGPBasket('SL1','L1_LOJA'))
90             EndIf
91             /* Inicia a classe LJDadosTransacaoPgtoDigitais para alimentar as propriedades de valor e
92                codigo da forma de pagamento que serao utilizadas posteriormente ao chamar a classe PagamentosDigitais */
93             oDados      := LJDadosTransacaoPgtoDigitais():New(nValor, "PD",,,aDadosCli)
94             oRetTran     := oTEF20:PgtoDigital():Digitais(oDados)
95         Case cTypeCard == "PX"
96             If ExistFunc("GetDdClTrn")
97                 aDadosCli := GetDdClTrn(STDGPBasket('SL1','L1_CLIENTE'), STDGPBasket('SL1','L1_LOJA'))
98             EndIf
99             oDados      := LJDadosTransacaoPgtoDigitais():New(nValor, "PX",,,aDadosCli)
100             oRetTran    := oTEF20:PgtoDigital():Digitais(oDados)
101     EndCase
102
103     If oRetTran:oRetorno:nParcs < 1
104         oRetTran:oRetorno:nParcs := nParc

```

```

105         EndIf
106
107         If (lRet := STBRetTef(oRetTran, oTEF20))
108             nParc := oRetTran:oRetorno:nParcs
109
110             //Parcela tem que ser 1 - mesmo se for a vista
111             If nParc < 1
112                 nParc := 1
113             EndIf
114             STISetCard(.T.)
115             STISetTef(oTEF20)
116             STBCalcJur()
117
118             STIAddPay(cTypeCard, oMdlMst, nParc, .T., /*cCodVp*/, nValor)
119
120         Else
121             LjGrvLog( "L1_NUM: "+ STDGPBasket('SL1','L1_NUM'), "TEF Retornou .F. | Operação não realizada." )
122             If AttIsMemberOf( oRetTran:oRetorno, "cErrorReason", .T.) .AND. !(Empty(oRetTran:oRetorno:cErrorReason))
123                 STFMessage("ErrorReason", "STOPPOPUP", STR0014 + oRetTran:oRetorno:cErrorReason) // "Não foi possível continuar, motivo: "
124                 STFShowMessage( "ErrorReason")
125             EndIf
126         EndIf
127     Else
128         STFMessage("TEF", "STOPPOPUP", aLtt[2])
129         STFShowMessage( "TEF")
130     EndIf
131
132     If !lRet .AND. lTefManu .AND. !IsPD0rPix(cTypeCard)
133
134         If SuperGetMv("MV_LJRGDES",,.F.) .AND. !STIGetRecTit() .AND. STIDescRegVar()
135             STFRestVlr()
136         Endif
137
138         If MsgNoYes(STR0007,STR0008) //"Não foi possível efetuar a transação com TEF, deseja continuar manualmente?" # "Atenção"
139             STISetContTef(.T.)
140             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "TEF MANUAL - SIM." )
141
142             aPayment := {nValor , IIF(nParc<1,1,nParc)}
143
144             STIAddNewPan( oPanPayment, ,cTypeCard,aPayment)
145
146         Else
147             STISetContTef(.F.)
148             LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "TEF MANUAL - NÃO." )
149         EndIf
150     Else
151         STISetContTef(.F.)
152     EndIf
153
154 Else
155
156     STFMessage("STWPayCard","STOP",STR0001) //"Falha para obter o número do cupom fiscal!"

```

```

157         STFS showMessage( "STWPayCard" )
158
159     EndIf
160
161 Else
162     // Quando for importação de orçamento o calculo dos juros é feito no momento da importação
163     If !STBIsImpOrc()
164         STBCalcJur()
165     EndIf
166
167     STIAddPay(cTypeCard, oMdMst, IIF(ValType(oMdMst) == 'O', oMdMst:GetValue("L4_PARC"),nParc),,,nValor)
168
169     If !STBIsImpOrc()
170         STISetContTef(.F.)
171     EndIf
172
173     lRet := .T.
174 EndIf
175
176 Else
177
178     STFMessage("STWPayCard","STOP", STR0002 + AllTrim(Str(nValor,10,2))) //"Valor inválido."
179     STFS showMessage("STWPayCard")
180     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Valor inválido." )
181 EndIf
182
183 If lFinVendaAut .AND. lRet .And. STBCalcSald("1") == 0
184     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Chama Finalização da venda após TEF" )
185     STIConfPay()
186 EndIf
187
188 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Retorno: " + IIF(lRet,".T.", ".F.") )
189
190 Return lRet
191
192
193 //-----
194 /*/{Protheus.doc} STWPrintComp
195 Imprissao do comprovante TEF
196
197 @param      oTEF20 - Objeto do TEF
198 @return      lRet - Retorna se imprimiu corretamente
199 @obs
200 @sample
201 */
202 //-----
203 Function STWPrintComp(oTEF20)
204
205 Local lRelGer      := .F.
206 Local cContGer      := ""
207 Local cContCDC      := ""
208 Local cContONF      := ""

```

```

209 Local aDados      := {}
210 Local aRet         := {}
211 Local lRet         := .T. //retorna se imprimiu comprovante
212
213 Default oTEF20 := Nil
214
215 ParamType 0 Var      oTEF20          As Object Default Nil
216
217 If ValType(oTEF20) == '0'
218
219     If oTEF20:Cupom():Imprimir(Nil)
220         /*
221             Se a impressao for OK, confirma a transacao
222         */
223         oTEF20:Confirmar(@lRelGer)
224
225         // Retorna o Contador Geral de Operação Não Fiscal
226         If lIsPAF
227             aDados := {"36", Space(6)}
228
229             aRet := STFFireEvent(ProcName(0), "STPrinterStatus", aDados)
230
231             If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) < 1
232                 STFMessage("STWPrintComp","STOP",STR0003) //"Falha na obtenção do Contador de Operação Nao Fiscal"
233                 STFShowMessage("STWPrintComp")
234             Else
235                 cContONF := aDados[2]
236             EndIf
237
238             // Retorna o Contador de Comprovante de Credito ou Debito
239             aDados := {"38", Space(6)}
240
241             aRet := STFFireEvent(ProcName(0), "STPrinterStatus", aDados)
242
243             If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) < 1
244                 STFMessage("STWPrintComp","STOP",STR0004) //"Falha na obtenção do Contador de CDC"
245                 STFShowMessage("STWPrintComp")
246             Else
247                 cContCDC := aDados[2]
248             EndIf
249
250             If lRelGer
251                 // Retorna o Contador Geral de Relatório Gerencial
252                 aDados := {"38", Space(6)}
253
254                 aRet := STFFireEvent(ProcName(0), "STPrinterStatus", aDados)
255
256                 If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) < 1
257                     STFMessage("STWPrintComp","STOP",STR0005) //"Falha na obtenção do Contador de Relatorio Gerencial"
258                     STFShowMessage("STWPrintComp")
259                 Else
260                     cContGer := aDados[2]

```

```

261         EndIf
262     EndIf
263
264
265     STDSPBasket("SL1", "L1_CONTONF", cContONF )
266     STDSPBasket("SL1", "L1_CONTCDC", cContCDC )
267     STDSPBasket("SL1", "L1_CONTRG", cContGer)
268 EndIf
269 Else
270     /*
271     Se falho a impressao, cancela a transacao
272     */
273     lRet := .F.
274
275     If !ValType(oTEF20:oConfig:opgdig) <> "0" .And. Len(oTEF20:oConfig:opgdig:oTrans:acolecao) > 0 .And. Len(oTEF20:oConfig:oCCcD:oTrans:acolecao) == 0
276         LjGrvLog(Nil, "Falha na impressao do comprovante de Pagamento Digital.",,,,T.)
277         oTEF20:Confirmar()
278         lRet := .T.
279         If !Isblind()
280             MsgInfo(STR0015 + CHR(10)+CHR(13) + STR0016,STR0017) //"Falha na impressão do comprovante do Pagamento Digital"#"Se necessário utilize a rotina de reimpressão
via TEF Gerenciais."#" "TOTVS Pagamentos Digitais"
281         EndIf
282     Else
283         LjGrvLog(Nil, "Falha na impressao do comprovante de Cartão.",,,,T.)
284         oTEF20:Desfazer()
285         STFMessage("TEF", "POPUP", STR0006) //"Transação não foi efetuada. Favor reter o cupom."
286         STFShowMessage( "TEF")
287     EndIf
288 EndIf
289 EndIf
290
291 /*
292 Seta a variavel lCard e lCheque para .F., além de zerar o ID dos cartões TEF
293 */
294 STISetCard()
295 STBSetCheck()
296 If FindFunction("STBSetIDTF")
297 STBSetIDTF()
298 EndIf
299
300 Return lRet
301
302 /*{Protheus.doc} STWGetIdPgto
303 Retorna os Id's dos pagamentos enviados ao validador fiscal MFe
304 @type Function
305 @param
306 @return
307 */
308 Function STWGetIdPgto()
309 Return aIdPgtoMfe
310
311 /*{Protheus.doc} STWZeraIdPgto

```

```
312     Seta os Id's dos pagamentos enviados ao validador fiscal MFe
313     @type Function
314     @param
315     @return
316 */
317 Function STWZeraIdPgto(aId)
318 Default aId := {}
319 aIdPgtoMfe := aId
320 Return
```

Componentes/Payment/STWPayCdPg.prw

45 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STWPayCdPg
6 Pagamento condicao de pagamento
7 @param      oMdlGrd - Grid do Model
8 @return      Se executou corretamente
9 @obs
10 @sample
11 */
12 //-----
13 Function STWPayCdPg(oMdlGrd)
14 Local cTypePay := "" //Tipo da forma de pagamento
15 Local lContinua := IIf(ExistFunc("STIVeCTef"),STIVeCTef(.F.),.T.)
16
17 Default oMdlGrd := Nil
18
19
20 If lContinua .And. ValType(oMdlGrd) == '0' .AND. STBChkIsVp(oMdlGrd) .AND. STBChkTpCP(oMdlGrd)
21
22     STIZeraPay()           //Zera os pagamentos antes de lancar a condicao de pagamento
23     STBApliDes(oMdlGrd)    //Aplica desconto da condicao de pagamento
24     STBApliAcr(oMdlGrd)    //Aplica acrescimo da condicao de pagamento
25     STBParcCp()           //Gera as parcelas da condicao de pagamento
26     STBTratPar()          //Tratamento das parcelas geradas
27
28     STBCDPGAtuBasket()     //Atualizo a condição de pagamento
29
30     cTypePay := STBGetTpPay()
31
32     Do Case
33         Case cTypePay $ 'CC|CD'
34             STBTrnsCard(cTypePay)
35         Case cTypePay == 'CH'
36             STBTrnsCheck(oMdlGrd:aDatAModel)
37         Case !Empty(cTypePay)
38             STBTrnsCp()
39         Case Empty(cTypePay) // Mv_CondPad = ""
40             STBTrnsCp()
41             STIZeraPay(.T.) // sem essa alteracao, trava a tela de pagamentos
42     EndCase
43 EndIf
44
45 Return .T.

```

Componentes/Payment/STWPayCheck.prw

156 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWPAYCHECK.CH"
4
5 Static aRetCheck      := {}           //Dados do cheque
6 Static lUsaConCh      := STWChkTef("CH") //Utiliza Consulta de Cheque
7
8
9 //-----
10 /*{Protheus.doc} STWPayCheck
11 Executa as funcionalidades de leitura, consulta e impressao de cheque
12
13 @param lCheck          Campo checado
14 @param oPanGetPaym     Objeto pegar pagamento
15 @param nPosHozPan      Posicao horizontal
16 @param nPosAltPan      Posicao Altura
17 @param nTamLagPan      Tamanho Largura
18 @param nTamAltPan      Tamanho Altura
19 @return lRet - Se executou corretamente
20 @obs
21 @sample
22 */
23 //-----
24 Function STWPayCheck( lCheck      , bCreatePan      )
25
26 Local lUsaCMC7      := STFUseCmc7()           //Utiliza CMC7
27 Local lUsaImpCh     := STFUsePrtCh()          //Utiliza Impressao de Cheque
28 Local lCallDataChk  := .F.                   // Indica se o STIDATACHECK foi chamado
29
30 Default lCheck      := .F.
31 Default bCreatePan  := Nil
32
33 ParamType 0 var      lCheck          As Logical Default .F.
34
35 If lUsaImpCh .AND. !lCheck .AND. Len(aRetCheck) > 0
36
37     STBPrintChk(aRetCheck)
38     aRetCheck := {}
39
40 ElseIf lCheck
41
42     /* Dados do cheque por CMC7 ou manual */
43     If lUsaCMC7
44
45         aRetCheck := {}
46         aRetCheck := STBLeCMC7()
47
48     /* Consulta do cheque */
49     If lUsaConCh
50         STBConCheck(aRetCheck)
51     Else
52         STBInsCheck()

```

```

53         EndIf
54
55     Else
56         lCallDataChk := .T.
57         STIDataCheck(bCreatePan)
58     EndIf
59
60 EndIf
61
62 If !lCallDataChk
63     // Caso o fonte de interface STIDataCheck nao seja chamado, habilito as opcoes do pagamento.
64     STIEnblPaymentOptions()
65 EndIf
66
67 Return .T.
68
69
70 //-----
71 /*/{Protheus.doc} STWGetCkRet
72 Retorna os dados do cheque
73
74 @param
75 @return      aRet - Dados do cheque
76 @obs
77 @sample
78 */
79 //-----
80 Function STWGetCkRet()
81
82 Local aRet := aRetCheck // Dados do cheque
83
84 Return aRet
85
86
87 //-----
88 /*/{Protheus.doc} STWSetCkRet
89 Seta os dados do cheque
90
91 @param      aRet
92 @return      lRet - Se executou corretamente
93 @obs
94 @sample
95 */
96 //-----
97 Function STWSetCkRet(aRet)
98
99 If !Empty(aRet)
100     aAdd(aRetCheck, aRet)
101 Else
102     aRetCheck := {}
103
104     If FindFunction("STBClearCk")

```

```

105         STBClearCk()
106     EndIf
107 EndIf
108
109 Return .T.
110
111
112 //-----
113 /*{Protheus.doc} STWConChk
114 Consulta cheque
115
116 @param
117 @return      lRet - Se executou corretamente
118 @obs
119 @sample
120 */
121 //-----
122 Function STWConChk()
123
124 // Consulta do cheque
125 If lUsaConCh
126     STBConCheck(aRetCheck)
127 Else
128     STBInsCheck()
129 EndIf
130
131 Return .T.
132
133
134 //-----
135 /*{Protheus.doc} STWValField
136 Valida campos
137
138 @param
139 @return      lRet - Retorna se o campo é vazio
140 @obs
141 @sample
142 */
143 //-----
144 Function STWValField(cCampo, cConteudo)
145
146 Local lRet := .T.
147
148 If Empty(cConteudo)
149     lRet := .F.
150     STFMessage(ProcName(),"STOP", STR0001 + cCampo + STR0002) //Campo '<campo>' deve ser preenchido.
151     STFShowMessage(ProcName())
152 Else
153     STFCleanInterfaceMessage()
154 EndIf
155
156 Return lRet

```

Componentes/Payment/STWPayGiftV.prw

44 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STWPayGiftV
6 Decisoões para vale presente
7
8 @param   cValPre - Vale presente
9 @param   oMdlDtl - Model detalhes
10 @return   cValor - Valor do vale presente
11 @obs
12 @sample
13 */
14 //-----
15 Function STWPayGiftV(cValPre, oMdlDtl)
16
17 Local cMsg := "" //Mensagem de inconsistência de vale presente
18 Local cValor := 0 //Valor do vale presente
19
20 Default cValPre := ""
21 Default oMdlDtl := Nil
22
23 ParamType 0 Var   cValPre   As Character   Default   ""
24 ParamType 1 Var   oMdlDtl   As Object     Default Nil
25
26 If !Empty(cValPre) .AND. STBVldVp(oMdlDtl, cValPre)
27     /*
28     Valida algumas validações do vale presente que o usuário informou
29     */
30     If Empty(cMsg := STBValidVP(cValPre))
31
32         /*
33         Consulta o valor do vale presente
34         */
35         cValor := STBValorVP(cValPre)
36
37     Else
38         STFMessage("STWPayGiftV", "STOP", cMsg)
39         STFShowMessage("STWPayGiftV")
40     EndIf
41
42 EndIf
43
44 Return cValor

```

Componentes/ProductBasket/STDProductBasket.prw

914 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "STDPRODUCTBASKET.CH"
7
8 Static oModelPBasket := Nil
9
10 //-----
11 /*{Protheus.doc} STDProductBasket
12 Definicao do Modelo
13 @param
14 @return Nil
15 @obs
16 @sample
17 */
18 //-----
19 Function STDProductBasket()
20
21 Return Nil
22
23 //-----
24 /*{Protheus.doc} ModelDef
25 Definicao do Modelo
26 @param
27 @return oModel - Model
28 @obs
29 @sample
30 */
31 //-----
32 Static Function ModelDef()
33
34 Local oStructSL1 := FWFormStruct(1,"SL1")
35 Local oStructSL2 := FWFormStruct(1,"SL2")
36 Local oStructSL4 := FWFormStruct(1,"SL4")
37 Local oStructMGC := FWFormStruct(1,"MGC")
38 Local oModel := Nil
39 Local lCRdesItTt := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMv("MV_LJC RDPT",,"0") == "1"
40
41 //-----
42 //Monta o modelo do formulário
43 //-----
44 oModel := MPFormModel():New("STDProductBasket",/*Pre-Validacao*/,/*Pos-Validacao*/,/*Commit*/,/*Cancel*/)
45
46 // Campo L1_TOTFISC - Esse campo nao existe no dicionário
47 oStructSL1:AddField(
48     STR0007      , ; // [01] Titulo do campo # "Tot It Fis"
49     STR0008      , ; // [02] Desc do campo #"Total do Item Fiscal"
50     "L1_TOTFISC" , ; // [03] Id do Field
51     "N"          , ; // [04] Tipo do campo
52     8            , ; // [05] Tamanho do campo

```

```

53         0           , ; // [06] Decimal do campo
54         Nil         , ; // [07] Code-block de validacao do campo
55         Nil         , ; // [08] Code-block de validacao When do campo
56         Nil         , ; // [09] Lista de valores permitido do campo
57         Nil         , ; // [10] Indica se o campo tem preenchimento obrigatorio
58         FwBuildFeature( STRUCT_FEATURE_INIPAD, "0" ) , ; // [11] Code-block de inicializacao do campo
59         NIL         , ; // [12] Indica se trata-se de um campo chave
60         NIL         , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
61         .T.         , ; // [14] Indica se o campo e virtual
62
63 // Campo L1_TOTNFIS - Esse campo nao existe no dicionário
64 oStructSL1:AddField( ;
65         STR0009      , ; // [01] Titulo do campo # "It Nao Fisc"
66         STR0010      , ; // [02] Desc do campo # "Total Item Nao Fiscal"
67         "L1_TOTNFIS" , ; // [03] Id do Field
68         "N"          , ; // [04] Tipo do campo
69         8            , ; // [05] Tamanho do campo
70         0            , ; // [06] Decimal do campo
71         Nil          , ; // [07] Code-block de validacao do campo
72         Nil          , ; // [08] Code-block de validacao When do campo
73         Nil          , ; // [09] Lista de valores permitido do campo
74         Nil          , ; // [10] Indica se o campo tem preenchimento obrigatorio
75         FwBuildFeature( STRUCT_FEATURE_INIPAD,"0" ) , ; // [11] Code-block de inicializacao do campo
76         NIL          , ; // [12] Indica se trata-se de um campo chave
77         NIL          , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
78         .T.          , ; // [14] Indica se o campo e virtual
79
80 // Campo L1_DESCTOT - Esse campo nao existe no dicionário
81 oStructSL1:AddField( ;
82         STR0021      , ; // [01] Titulo do campo # "Desc.Total"
83         STR0022      , ; // [02] Desc do campo # "Desconto concedido no total da venda"
84         "L1_DESCTOT" , ; // [03] Id do Field
85         "N"          , ; // [04] Tipo do campo
86         16           , ; // [05] Tamanho do campo
87         2            , ; // [06] Decimal do campo
88         Nil          , ; // [07] Code-block de validacao do campo
89         Nil          , ; // [08] Code-block de validacao When do campo
90         Nil          , ; // [09] Lista de valores permitido do campo
91         Nil          , ; // [10] Indica se o campo tem preenchimento obrigatorio
92         FwBuildFeature( STRUCT_FEATURE_INIPAD,"0" ) , ; // [11] Code-block de inicializacao do campo
93         NIL          , ; // [12] Indica se trata-se de um campo chave
94         NIL          , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
95         .T.          , ; // [14] Indica se o campo e virtual
96
97 // Campo L1_BONIF - Esse campo nao existe no dicionário
98 oStructSL1:AddField( ;
99         STR0023      , ; // [01] Titulo do campo # "Bonif."
100        STR0024      , ; // [02] Desc do campo # "Valor Tot. da Bonificação"
101        "L1_BONIF"    , ; // [03] Id do Field
102        "N"           , ; // [04] Tipo do campo
103        16            , ; // [05] Tamanho do campo
104        2             , ; // [06] Decimal do campo

```

```

105         Nil          ,      ; // [07] Code-block de validacao do campo
106         Nil          ,      ; // [08] Code-block de validacao When do campo
107         Nil          ,      ; // [09] Lista de valores permitido do campo
108         Nil          ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
109         FwBuildFeature( STRUCT_FEATURE_INIPAD,"0" ) ,      ; // [11] Code-block de inicializacao do campo
110         NIL           ,      ; // [12] Indica se trata-se de um campo chave
111         NIL           ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
112         .T.           )      // [14] Indica se o campo e virtual
113
114 // Campo L1_PFISICA - Esse campo nao existe no dicionário
115 oStructSL1:AddField(      ;
116         STR0025        ,      ; // [01] Titulo do campo # "Cli.Estrangeiro"
117         STR0026        ,      ; // [02] Desc do campo # "Doc.Cliente estrangeiro"
118         "L1_PFISICA"   ,      ; // [03] Id do Field
119         "C"            ,      ; // [04] Tipo do campo
120         14             ,      ; // [05] Tamanho do campo
121         0              ,      ; // [06] Decimal do campo
122         Nil            ,      ; // [07] Code-block de validacao do campo
123         Nil            ,      ; // [08] Code-block de validacao When do campo
124         Nil            ,      ; // [09] Lista de valores permitido do campo
125         Nil            ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
126         FwBuildFeature( STRUCT_FEATURE_INIPAD,"" ) ,      ; // [11] Code-block de inicializacao do campo
127         NIL            ,      ; // [12] Indica se trata-se de um campo chave
128         NIL            ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
129         .T.            )      // [14] Indica se o campo e virtual
130
131 oModel:AddFields("SL1MASTER", Nil/*cOwner*/, oStructSL1 ,/*Pre-Validacao*/,/*Pos-Validacao*/,/*Carga*/)
132
133 // Campo L2_FISCAL - Esse campo nao existe no dicionário
134 oStructSL2:AddField(      ;
135         STR0001        ,      ; // [01] Titulo do campo
136         STR0001        ,      ; // [02] Desc do campo
137         "L2_FISCAL"    ,      ; // [03] Id do Field
138         "L"            ,      ; // [04] Tipo do campo
139         1              ,      ; // [05] Tamanho do campo
140         0              ,      ; // [06] Decimal do campo
141         Nil            ,      ; // [07] Code-block de validacao do campo
142         Nil            ,      ; // [08] Code-block de validacao When do campo
143         Nil            ,      ; // [09] Lista de valores permitido do campo
144         Nil            ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
145         Nil            ,      ; // [11] Code-block de inicializacao do campo
146         NIL            ,      ; // [12] Indica se trata-se de um campo chave
147         NIL            ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
148         )              // [14] Indica se o campo e virtual
149
150 // Campo L2_BONIFICADOR - Esse campo nao existe no dicionário
151 oStructSL2:AddField(      ;
152         "Bonificador?" ,      ; // [01] Titulo do campo
153         "Bonificador?" ,      ; // [02] Desc do campo
154         "L2_BONIFICADOR" ,      ; // [03] Id do Field
155         "L"            ,      ; // [04] Tipo do campo
156         1              ,      ; // [05] Tamanho do campo

```

```

157         0 , ; // [06] Decimal do campo
158         Nil , ; // [07] Code-block de validacao do campo
159         Nil , ; // [08] Code-block de validacao When do campo
160         Nil , ; // [09] Lista de valores permitido do campo
161         Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
162         FwBuildFeature( STRUCT_FEATURE_INIPAD,".F." ) , ; // [11] Code-block de inicializacao do campo
163         NIL , ; // [12] Indica se trata-se de um campo chave
164         NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
165         .T. ) // [14] Indica se o campo e virtual
166
167 // Campo L2_ITFISC - Esse campo nao existe no dicionário
168 oStructSL2:AddField( ;
169     STR0011 , ; // [01] Titulo do campo#"It Fiscal"
170     STR0012 , ; // [02] Desc do campo #"Item Fical"
171     "L2_ITFISC" , ; // [03] Id do Field
172     "C" , ; // [04] Tipo do campo
173     TamSx3("L2_ITEM")[1] , ; // [05] Tamanho do campo
174     0 , ; // [06] Decimal do campo
175     Nil , ; // [07] Code-block de validacao do campo
176     Nil , ; // [08] Code-block de validacao When do campo
177     Nil , ; // [09] Lista de valores permitido do campo
178     Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
179     FwBuildFeature( STRUCT_FEATURE_INIPAD,"" ) , ; // [11] Code-block de inicializacao do campo
180     NIL , ; // [12] Indica se trata-se de um campo chave
181     NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
182     .T. ) // [14] Indica se o campo e virtual
183
184
185 // Campo L2_ITNFISC - Esse campo nao existe no dicionário
186 oStructSL2:AddField( ;
187     STR0013 , ; // [01] Titulo do campo "It Nao Fisc"
188     STR0014 , ; // [02] Desc do campo # "Item Nao Fiscal"
189     "L2_ITNFISC" , ; // [03] Id do Field
190     "C" , ; // [04] Tipo do campo
191     TamSx3("L2_ITEM")[1] , ; // [05] Tamanho do campo
192     0 , ; // [06] Decimal do campo
193     Nil , ; // [07] Code-block de validacao do campo
194     Nil , ; // [08] Code-block de validacao When do campo
195     Nil , ; // [09] Lista de valores permitido do campo
196     Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
197     FwBuildFeature( STRUCT_FEATURE_INIPAD,"" ) , ; // [11] Code-block de inicializacao do campo
198     NIL , ; // [12] Indica se trata-se de um campo chave
199     NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
200     .T. ) // [14] Indica se o campo e virtual
201
202 // Campo L2_ITEMREA - Esse campo nao existe no dicionário
203 oStructSL2:AddField( ;
204     "Item Real" , ; // [01] Titulo do campo "It Nao Fisc"
205     "Item Real" , ; // [02] Desc do campo # "Item Nao Fiscal"
206     "L2_ITEMREA" , ; // [03] Id do Field
207     "C" , ; // [04] Tipo do campo
208     TamSx3("L2_ITEM")[1] , ; // [05] Tamanho do campo

```

```

209         0 , ; // [06] Decimal do campo
210         Nil , ; // [07] Code-block de validacao do campo
211         Nil , ; // [08] Code-block de validacao When do campo
212         Nil , ; // [09] Lista de valores permitido do campo
213         Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
214         FwBuildFeature( STRUCT_FEATURE_INIPAD,"" ) , ; // [11] Code-block de inicializacao do campo
215         NIL , ; // [12] Indica se trata-se de um campo chave
216         NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
217         .T. ) // [14] Indica se o campo e virtual
218
219 // Campo L2_REGSLX - Esse campo nao existe no dicionário
220 oStructSL2:AddField( ;
221     "NumReg SLX?" , ; // [01] Titulo do campo
222     "NumReg SLX?" , ; // [02] Desc do campo
223     "L2_REGSLX" , ; // [03] Id do Field
224     "N" , ; // [04] Tipo do campo
225     10 , ; // [05] Tamanho do campo
226     0 , ; // [06] Decimal do campo
227     Nil , ; // [07] Code-block de validacao do campo
228     Nil , ; // [08] Code-block de validacao When do campo
229     Nil , ; // [09] Lista de valores permitido do campo
230     Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
231     FwBuildFeature( STRUCT_FEATURE_INIPAD,".F." ) , ; // [11] Code-block de inicializacao do campo
232     NIL , ; // [12] Indica se trata-se de um campo chave
233     NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
234     .T. ) // [14] Indica se o campo e virtual
235
236 // Campo L2_TABPAD - Esse campo nao existe no dicionário
237 oStructSL2:AddField( ;
238     "Tab Pad" , ; // [01] Titulo do campo
239     "Tab Pad" , ; // [02] Desc do campo
240     "L2_CODTAB" , ; // [03] Id do Field
241     "C" , ; // [04] Tipo do campo
242     TamSx3("DA0_CODTAB")[1] , ; // [05] Tamanho do campo
243     0 , ; // [06] Decimal do campo
244     Nil , ; // [07] Code-block de validacao do campo
245     Nil , ; // [08] Code-block de validacao When do campo
246     Nil , ; // [09] Lista de valores permitido do campo
247     Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
248     Nil , ; // [11] Code-block de inicializacao do campo
249     NIL , ; // [12] Indica se trata-se de um campo chave
250     NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
251     .T. ) // [14] Indica se o campo e virtual
252
253 If lCRdesItTt
254     // Campo L2_VLDRGDV - Esse campo nao existe no dicionário
255     oStructSL2:AddField( ;
256         "Desc. Regr." , ; // [01] Titulo do campo
257         "Desc. Regr." , ; // [02] Desc do campo
258         "L2_VLDRGDV" , ; // [03] Id do Field
259         "N" , ; // [04] Tipo do campo
260         TamSx3("L2_VALDESC")[1] , ; // [05] Tamanho do campo

```

```

261          TamSx3("L2_VALDESC")[2] ,      ; // [06] Decimal do campo
262          Nil ,                          ; // [07] Code-block de validacao do campo
263          Nil ,                          ; // [08] Code-block de validacao When do campo
264          Nil ,                          ; // [09] Lista de valores permitido do campo
265          Nil ,                          ; // [10] Indica se o campo tem preenchimento obrigatorio
266          Nil ,                          ; // [11] Code-block de inicializacao do campo
267          NIL ,                          ; // [12] Indica se trata-se de um campo chave
268          NIL ,                          ; // [13] Indica se o campo pode receber valor em uma operacao de update.
269          .T. )                          // [14] Indica se o campo e virtual
270 EndIf
271
272 oModel:AddGrid("SL2DETAIL","SL1MASTER"/oOwner*/oStructSL2/*bLinePre*//*bLinePost*//*bPre*//*bPost*//*Carga*/)
273 oModel:SetRelation("SL2DETAIL",{{"L2_FILIAL",'xFilial("SL2")'},{"L2_NUM","L1_NUM"}},SL2->(IndexKey(1)))
274 oModel:GetModel("SL2DETAIL"):SetOptional(.T.)
275 oModel:GetModel("SL2DETAIL"):SetOnlyView(.T.)
276
277 // Retira o obrigatorio de todos os campos da SL2 para nao sobrepor os itens smpre na primeira linha
278 oStructSL2:SetProperty("MODEL_FIELD_OBRIGAT,.F.)
279
280 oModel:AddGrid("SL4DETAIL","SL1MASTER"/oOwner*/oStructSL4/*bLinePre*//*bLinePost*//*bPre*//*bPost*//*Carga*/)
281 oModel:SetRelation("SL4DETAIL",{{"L4_FILIAL",'xFilial("SL4")'},{"L4_NUM","L1_NUM"}},SL4->(IndexKey(1)))
282 oModel:GetModel("SL4DETAIL"):SetOptional(.T.)
283
284 If AliasInDic("MGC")
285     oModel:AddGrid("MGCBRINDE","SL1MASTER"/oOwner*/oStructMGC/*bLinePre*//*bLinePost*//*bPre*//*bPost*//*Carga*/)
286     oModel:SetRelation("MGCBRINDE",{{"MGC_FILIAL",'xFilial("MGC")'},{"MGC_NUM","L1_NUM"}},MGC->(IndexKey(1)))
287     oModel:GetModel("MGCBRINDE"):SetOptional(.T.)
288 EndIf
289 /*
290 Seta a descricao de todos os modelos de dados
291 */
292 oModel:SetDescription("Cesta de Produtos")
293 oModel:GetModel("SL1MASTER"):SetDescription(STR0002) //"Orçamento"
294 oModel:GetModel("SL2DETAIL"):SetDescription(STR0003) //"Itens de Orçamento"
295 oModel:GetModel("SL4DETAIL"):SetDescription(STR0004) //"Condição Negociada"
296
297 If AliasInDic("MGC")
298     oModel:GetModel("MGCBRINDE"):SetDescription("BRINDE")//"Brinde"
299 EndIf
300
301 oStructSL1:SetProperty( "MODEL_FIELD_VALID,{|| .T.} )
302 oStructSL2:SetProperty( "MODEL_FIELD_VALID,{|| .T.} )
303 oStructSL4:SetProperty( "MODEL_FIELD_VALID,{|| .T.} )
304 If AliasInDic("MGC")
305     oStructMGC:SetProperty( "MODEL_FIELD_VALID,{|| .T.} )
306 EndIf
307
308 oModelPBasket:= oModel
309 oModelPBasket:SetOperation(3)
310 oModelPBasket:Activate()
311
312 Return oModel

```

```

313
314 //-----
315 /*{Protheus.doc} STDSPBasket
316 Funcao utilizada para setar valores ao model da cesta de produtos.
317 @param
318 @return lRet - Se setou valores corretamente
319 @obs
320 @sample
321 */
322 //-----
323 Function STDSPBasket( cAlias , cField , xValue , nLine )
324
325 Local cModelID      := ""           // ID do model de cAlias
326 Local cError        := ""           // Mensagem de erro contida no objeto de tratamento de erro.
327 Local lRet          := .F.          // Retorno da funcao
328 Local nField        := 0            // Retorno do aScan, que representa a posicao de cField dentro de aFields.
329 Local aFields       := {}           // Array com todos os campos da estrutura do cModelID selecionado
330 Local aSaveLines    := FWSaveRows() // array de linhas salvas
331 Local oViewGrid     := Nil           // Objeto view
332 Local oModelGrid    := Nil           // Objeto model
333 Local bOldError     := ErrorBlock( {|oErro|STBTratErro(oErro,@cError)} ) // Muda code-block de erro
334 Local cTipoxValue   := ""           // Tipo da variavel xValue
335 Local cMsg          := ""           // Mensagem auxiliar para log
336 Local cIdLog        := ""           // Mensagem auxiliar para log
337 Local lCancelSale   := .F.          // Cancela venda
338
339 Default cAlias      := ""
340 Default cField      := ""
341 Default xValue      := Nil
342 Default nLine       := 0
343
344 cTipoxValue := ValType(xValue)
345 cAlias      := AllTrim(cAlias)
346
347 If cTipoxValue != "U"
348     cMsg := "GravaCesta - Tabela:[" + cAlias + "] - Campo:[" + PadR(cField,10) + "] - Valor:[" + cValToChar(xValue) + "]"
349 Else
350     cMsg := "GravaCesta - Tabela:[" + cAlias + "] - Campo:[" + PadR(cField,10) + "] - Valor:[Indefinido]"
351 EndIf
352
353 Do Case
354     Case cAlias == "SL1"
355         nLine := 0
356         cIdLog := "L1_Num:" + STDGPBasket('SL1','L1_NUM')
357     Case cAlias == "SL2"
358         cIdLog := "L2_Num:" + STDGPBasket('SL2','L2_NUM')
359         cMsg += " - Item Produto:" + cValToChar(nLine)
360     Case cAlias == "SL4"
361         cIdLog := "L4_Num:" + STDGPBasket('SL4','L4_NUM')
362         cMsg += " - Item Forma:" + cValToChar(nLine)
363 EndCase
364

```

```

365 cModelID := STDGModelID(cAlias) // Funcao que pega o ID do model da tabela passada em cAlias
366
367 If !Empty(cModelID) // Verifico se o Model passado para o set existe
368     If cTipoxValue != "U"
369         Begin Sequence
370             If nLine > 0
371                 oModelPBasket:GetModel(cModelID):GoLine(nLine)
372                 lRet := oModelPBasket:LoadValue(cModelID,cField,xValue)
373             Else
374                 lRet := oModelPBasket:LoadValue(cModelID,cField,xValue)
375             EndIf
376         End Sequence
377     EndIf
378
379 // Caso nao haja erro, eh avaliado e exibido no console o motivo do valor nao ter sido setado
380 If !lRet
381
382     // Pego todos os campos da estrutura do cModelID
383     aFields := oModelPBasket:GetModel(cModelID):GetStruct():GetFields()
384
385     // Pego a posicao de cField em aFields, se existir
386     nField := aScan(aFields,{|aCampo| aCampo[MODEL_FIELD_IDFIELD] == cField})
387
388     If nField > 0
389
390         cTipoxValue := ValType(xValue)
391
392         Do Case
393             Case cTipoxValue == "C"
394                 cMsg := xValue
395             Case cTipoxValue == "D"
396                 cMsg := AllTrim( DTOS(xValue) )
397             Case cTipoxValue == "N"
398                 cMsg := AllTrim( STR(xValue) )
399         EndCase
400
401         If aFields[nField][MODEL_FIELD_TIPO] != ValType(xValue)
402             ConOut("0 valor: " + cMsg + " Tamanho: " + AllTrim(STR(Len(cMsg))) + " atribuido a variavel xValue Tipo: " + ValType(xValue) + " eh diferente do tipo do campo
"+cField+". Tipo: " + aFields[nField][MODEL_FIELD_TIPO] )
403             LjGrvLog( "STDProductBasket", "0 valor: " + cMsg + " Tamanho: " + AllTrim(STR(Len(cMsg))) + " atribuido a variavel xValue Tipo: " + ValType(xValue) + " eh
diferente do tipo do campo "+cField+". Tipo: " + aFields[nField][MODEL_FIELD_TIPO] )
404             ConOut("Em caso de importação de orcamentos Compare o campo " +cField+ " do PDV Local com o campo na base da retaguarda, os dois devem estar iguais." )
405             LjGrvLog( "STDProductBasket", "Em caso de importação de orcamentos Compare o campo " +cField+ " do PDV Local com o campo na base da retaguarda, os dois devem
estar iguais." )
406         Else
407             ConOut("A validacao do campo "+cField+" nao foi respeitada. Tipo: " + ValType(xValue) + " valor: " + cMsg + " Tamanho: " + AllTrim(STR(Len(cMsg))) )
408             LjGrvLog( "STDProductBasket", "A validacao do campo "+cField+" nao foi respeitada. Tipo: " + ValType(xValue) + " valor: " + cMsg + " Tamanho: " +
AllTrim(STR(Len(cMsg))) )
409             ConOut("Verifique se o valor esta respeitando o X3_VALID, X3_VLDUSER e consta no X3_CBOX caso preenchido.")
410             LjGrvLog( "STDProductBasket", "Verifique se o valor esta respeitando o X3_VALID, X3_VLDUSER e consta no X3_CBOX caso preenchido." )
411             ConOut("Em caso de importação de orcamentos Compare o campo " +cField+ " do PDV Local com o campo na base da retaguarda, os dois devem estar iguais." )
412             LjGrvLog( "STDProductBasket", "Em caso de importação de orcamentos Compare o campo " +cField+ " do PDV Local com o campo na base da retaguarda, os dois devem

```

```

    estar iguais." )
413         lCancelSale := .T.
414     EndIf
415
416
417     If lCancelSale
418         STFMessage(ProcName(),"POPUP", STR0005 + cField + " " + ;// "Atenção. Erro ao setar valor no campo "
419             STR0017 + ValType(xValue) + STR0018 + cMsg + STR0019 + AllTrim(STR(Len(cMsg))) + ;// "Tipo: "### " Valor: "### " Tamanho: "
420             STR0020 +; ;// " Para Importação de orçamento verifique o campo(SX3) na Base PDV x Retaguarda "
421             STR0006)//" . A venda em andamento será cancelada."
422         STFShowMessage(ProcName())
423         //Cancela Venda Atual
424         STWCancelSale( .T. )
425     EndIf
426
427     Else
428         ConOut("O campo "+cField+" nao faz parte do modelo de dados da tabela "+cAlias+".")
429         LjGrvLog( "STDProductBasket","O campo "+cField+" nao faz parte do modelo de dados da tabela "+cAlias+".")
430     EndIf
431 EndIf
432 Else
433     ConOut("O Alias passado para a função STDSPBasket não é válido.")
434     LjGrvLog( "STDProductBasket","O Alias passado para a função STDSPBasket não é válido.")
435 EndIf
436
437 ErrorBlock(bOldError)
438
439 FWRestRows( aSaveLines )
440
441 // Limpa os vetores para melhor gerenciamento de memoria (Desaloca Memória)
442 aSize( aSaveLines, 0 )
443 aSaveLines := Nil
444
445 Return lRet
446
447
448 //-----
449 /*{Protheus.doc} STDGPBasket
450 Funcao utilizada para pegar valores do model da cesta de produtos.
451 @param
452 @return xRet - Valor do model
453 @obs
454 @sample
455 */
456 //-----
457 Function STDGPBasket( cAlias , cField , nLine )
458
459 Local cModelID      := ""    // ID do model do Alias passado.
460 Local cError        := ""    // Mensagem de erro contida no objeto de tratamento de erro.
461 Local xRet          := Nil    // Retorno (Pode ser de qualquer tipo)
462 Local aSaveLines    := FWSaveRows() // Linhas salvas
463 Local bOldError     := ErrorBlock( { |oErro| STBTratErro(oErro,@cError)} ) // Muda code-block de erro

```

```

464
465 Default cAlias      := ""
466 Default cField      := ""
467 Default nLine       := 0
468
469 cAlias := AllTrim(cAlias)
470
471 cModelID := STDGModelID(cAlias) // Funcao que pega o ID do model da tabela passada em cAlias
472
473 If cAlias == "SL1"
474     nLine := 0
475 EndIf
476
477 If !Empty(cModelID) .AND. !Empty(cField) // Verifico se o Model passado para o get existe
478
479     Begin Sequence
480         If nLine > 0
481             oModelPBasket:GetModel(cModelID):GoLine(nLine)
482             xRet := oModelPBasket:GetValue(cModelID,cField)
483         Else
484             xRet := oModelPBasket:GetValue(cModelID,cField)
485         EndIf
486     End Sequence
487
488     // TO D0: Substituir o ConOut pelo componente de criacao de log
489     If !Empty(AllTrim(cError))
490         ConOut(cError)
491         LjGrvLog( Nil, cError)
492     EndIf
493 Else
494     // TO D0: Substituir o ConOut pelo componente de criacao de log
495     ConOut("0 alias passado para a função STDGPBasket não é válido.")
496     LjGrvLog( cAlias, "0 alias passado para a função STDGPBasket não é válido.")
497     LjGrvLog( cAlias, " Conteudo do cModelID", cModelID)
498     LjGrvLog( cAlias, " Conteudo do cField", cField)
499 EndIf
500
501 ErrorBlock(bOldError)
502
503 FWRestRows( aSaveLines )
504
505 // Limpa o vetor para melhor gerenciamento de memoria (Desaloca Memória)
506 aSize( aSaveLines,0 )
507 aSaveLines := Nil
508
509 Return xRet
510
511
512 //-----
513 /*{Protheus.doc} STDPBRestart
514 Funcao responsavel por reinicializar a cesta
515 @param

```

```

516 @return Nil
517 @obs
518 @sample
519 /*/
520 //-----
521 Function STDPBRestart()
522
523 Local oViewGrid := Nil      // Objeto View
524 Local oModelGrid := Nil    // Objeto Model
525 Local oModelPri := Nil      // Objeto Model principal
526
527 oModelPBasket := Nil
528 oModelPBasket := FwLoadModel("STDProductBasket")
529 oModelPri := STIGetMdlPri()
530 If oModelPri <> Nil
531     oModelGrid := oModelPri:GetModel("CUP_GRID")
532     oModelGrid:ClearData()
533     oModelGrid:InitLine()
534 EndIf
535
536 Return Nil
537
538
539 //-----
540 /*/{Protheus.doc} STDGPBModel
541 Funcao utilizada para todo o model da cesta de produtos.
542 @param
543 @return oModelPBasket - Model da cesta
544 @obs
545 @sample
546 /*/
547 //-----
548 Function STDGPBModel()
549
550 If oModelPBasket == Nil
551     oModelPBasket := FwLoadModel("STDProductBasket")
552 EndIf
553
554 Return oModelPBasket
555
556
557 //-----
558 /*/{Protheus.doc} STDPBAddLine
559 Funcao utilizada para todo o model da cesta de produtos.
560 @param
561 @return Nil
562 @obs
563 @sample
564 /*/
565 //-----
566 Function STDPBAddLine(cAlias)
567

```

```

568 Local cModelID := "" // ID do model
569 Local cCRLF := Chr(13) + Chr(10) //Controle de linha
570
571 Default cAlias := ""
572
573 If cAlias == "SL2" .OR. cAlias == "SL4" .OR. cAlias == "MGC"
574     cModelID := STDGModelID(cAlias)
575     oModelPBasket:GetModel(cModelID):AddLine(.T.)
576
577     If Len(oModelPBasket:GetErrorMessage()) > 0 .And. !Empty(oModelPBasket:GetErrorMessage()[6])
578         ConOut(cCRLF + STR0016 + oModelPBasket:GetErrorMessage()[6] + cCRLF)
579         LJGrvLog("L1_NUM: " + STDGPBasket("SL1", "L1_NUM"), STR0016 + oModelPBasket:GetErrorMessage()[6])
580         LJGrvLog("L1_NUM: " + STDGPBasket("SL1", "L1_NUM"), STR0016 + oModelPBasket:GetErrorMessage()[4])
581     EndIf
582 EndIf
583
584 Return Nil
585
586
587 //-----
588 /*{Protheus.doc} STDInitPBasket
589 Inicializa a cesta de produtos
590 @param
591 @return Nil
592 @obs
593 @sample
594 */
595 //-----
596 Function STDInitPBasket()
597
598 Local aArea := GetArea() // Guarda area
599 Local oModel := Nil // Model
600 Local cCliente := "" // Cliente
601 Local cLojaCli := "" // Loja
602 Local cTipoCli := "" // Tipo de cliente
603 Local cVendLoja := "" // Vendedor
604
605 If ValType(oModelPBasket) == "U" .OR. !oModelPBasket:IsActive()
606     oModel := FwLoadModel("STDProductBasket")
607 EndIf
608
609 //Confere se o Numero do Orcamento ja nao esta utilizado
610 STDPConfL1(STDGPBasket('SL1','L1_NUM'))
611
612 // Abertura do Cupom - Cria o SL1
613 // Se o cliente estiver vaziao, coloca o cliente padrão
614
615 nTamSxG := TamSxG("001")[1] // Grupo de Cliente
616 cCliente := Left(PadR(SuperGetMV("MV_CLIPAD"), nTamSxG), nTamSxG)
617
618 nTamSxG := TamSxG("002")[1] // Grupo de Loja
619 cLojaCli := Left(PadR(SuperGetMV("MV_LOJAPAD"), nTamSxG), nTamSxG)

```

```

620
621 nTamSXG := Len(SA3->A3_COD)
622 cVendLoja := Left(PadR(SuperGetMV("MV_VENDPAD"),nTamSXG),nTamSXG)
623
624 DbSelectArea("SA1")
625 SA1->(DbSetOrder())
626 If DbSeek(xFilial("SA1")+cCliente+cLojaCli)
627     cTipoCli := SA1->A1_TIPO
628 EndIf
629
630 /*
631     Inicializa o model do cliente com as informacoes do cliente padrao.
632 */
633 STWCustomerSelection(cCliente+cLojaCli)
634
635 STDSPBasket("SL1","L1_FILIAL",xFilial("SL1"))
636 STDSPBasket("SL1","L1_CLIENTE",cCliente)
637 STDSPBasket("SL1","L1_LOJA",cLojaCli)
638 STDSPBasket("SL1","L1_NOMCLI",Posicione("SA1",1,xFilial("SA1")+cCliente+cLojaCli,"A1_NOME"))
639 STDSPBasket("SL1","L1_TIPOCLI",cTipoCli)
640 STDSPBasket("SL1","L1_VEND",cVendLoja)
641 STDSPBasket("SL1","L1_COMIS",STDGComission( cVendLoja ))
642 STDSPBasket("SL1","L1_SERIE",STFGetStation("SERIE"))
643 STDSPBasket("SL1","L1_PDV",STFGetStation("PDV"))
644 STDSPBasket("SL1","L1_OPERADO",xNumCaixa())
645
646 RestArea(aArea)
647
648 Return Nil
649
650
651 //-----
652 /*{Protheus.doc} STBTratErro
653 Funcao de tratamento de erro.
654 @param      oErro      - Objeto de tratamento de erro
655 @param      cError      - Mensagem de erro. Como este parametro é passado por referencia, ele será retornado.
656 @return      Nil
657 @obs
658 @sample
659 */
660 //-----
661 Static Function STBTratErro( oErro , cError )
662
663 Default oErro      := Nil
664 Default cError      := ""
665
666 IF oErro:encode > 0 .AND. Empty(cError)
667     cError := oErro:DESCRIPTION + oErro:ErrorStack
668 EndIf
669
670 Return Nil
671

```

```

672
673 //-----
674 /*{Protheus.doc} STDGModelID
675 Funcao que recebe um Alias e retorna o ID do model que representa
676 esse alias. Apenas os alias SL1, SL2 e SL4 sao validos.
677 @param cAlias      - Alias que tera o ID do model retornado. Apenas os alias SL1, SL2 e SL4 sao validos.
678 @return      cModelID      - ID do model desejado.
679 @obs
680 @sample
681 */
682 //-----
683 Function STDGModelID(cAlias)
684
685 Local cModelID := ""          // ID do model
686
687 Default cAlias      := ""
688
689 If cAlias == "SL1"
690     cModelID := "SL1MASTER"
691 ElseIf cAlias == "SL2"
692     cModelID := "SL2DETAIL"
693 ElseIf cAlias == "SL4"
694     cModelID := "SL4DETAIL"
695 ElseIf cAlias == "MGC"
696     cModelID := "MGCBRINDE"
697 EndIf
698
699 Return cModelID
700
701
702 //-----
703 /*{Protheus.doc} STDPBLength
704 Funcao que recebe um Alias e retorna o numero de linhas do grid do alias informado.
705 Apenas os alias SL2,SL4 e MGC sao validos.
706 @param cAlias      - Alias que tera o tamanho do grid retornado. Apenas os alias SL2 e SL4 sao validos.
707 @param lItAtivos - lógico - Parametro para retornar todos os itens(.F. ou Default) ou somente itens NÃO CANCELADOS(.T.)
708 @return  nRet      - Numero de linhas do grid. Retorna 0 se o grid estiver vazio.
709 @obs
710 @sample
711 */
712 //-----
713 Function STDPBLength(cAliasTab, lItAtivos)
714
715 Local cModelID      := STDGModelID(cAliasTab)      // ID do Model
716 Local cField        := ""                          // Nome campo
717 Local nRet          := 0                          // Retorno
718
719 Default cAliasTab    := ""
720 Default lItAtivos    := .F.
721
722 If cAliasTab == "SL2" .OR. cAliasTab == "SL4" .OR. cAliasTab == "MGC"
723     If cAliasTab == "SL2"

```

```

724         cField := "L2_NUM"
725     Else
726         If cAliasTab == "SL4"
727             cField := "L4_NUM"
728         Else
729             cField := "MGC_NUM"
730         Endif
731     EndIf
732     If oModelPBasket == Nil
733         oModelPBasket := FWLoadModel("STDProductBasket")
734     EndIf
735
736     // Caso o grid nao esteja vazio, adiciona a linha
737     If oModelPBasket:GetModel(cModelID):Length() == 1 .AND. Empty(STDGPBasket(cAliasTab,cField,1))
738         nRet := 0
739     Else
740         nRet := oModelPBasket:GetModel(cModelID):Length(1ItAtivos)
741     EndIf
742 EndIf
743
744 Return nRet
745
746
747 //-----
748 /*{Protheus.doc} STDGetProperty
749 Funcao que retorna estrutura do campo
750 @param      cField      - Nome do campo
751 @return      aField      - Retorna campos do model
752 @obs
753 @sample
754 */
755 //-----
756 Function STDGetProperty( cField )
757
758 Local oModel      := Nil          // Model
759 Local aField      := {}           // Array de campos
760 Local aStruct     := {}           // Strutura
761 Local nPosition   := 0            // Numero Posicao
762
763 Default cField    := ""
764
765 If oModelPBasket == Nil
766     oModelPBasket := STDGPBModel()
767 EndIf
768
769 If !Empty(cField)
770
771     Do Case
772         Case SubStr( cField , 1 , 2 ) == "L1"
773
774             oModel      := oModelPBasket:GetModel("SLIMASTER")
775

```

```

776      Case SubStr( cField , 1 , 2 ) == "L2"
777
778          oModel      := oModelPBasket:GetModel("SL2DETAIL")
779
780      Case SubStr( cField , 1 , 2 ) == "L4"
781
782          oModel      := oModelPBasket:GetModel("SL4DETAIL")
783      Case SubStr( cField , 1 , 2 ) == "GC"
784          If AliasInDic("MGC")
785              oModel      := oModelPBasket:GetModel("MGCBRINDE")
786          EndIf
787      EndCase
788
789      If oModel <> Nil
790
791          aStruct      := oModel:GetStruct():aFields
792          nPosition    := aScan(aStruct,{|x| x[3] == cField } )
793          If nPosition > 0
794              aField      := aStruct[nPosition]
795          EndIf
796
797      EndIf
798
799      EndIf
800
801      Return aField
802
803      //-----
804      /*{Protheus.doc} STDPBIsDeleted
805      Funcao que retorna estrutura do campo
806
807      @param      cField      - Nome do campo
808      @return      aField      - Retorna campos do model
809      @obs
810      @sample
811      */
812      //-----
813      Function STDPBIsDeleted( cAliasTable, nLine )
814      Local lRet      := .F.
815      Local aSaveLines := FwSaveRows()
816      Local oModelTable := Nil
817
818      DEFAULT cAliasTable := ""
819      DEFAULT nLine := 0
820
821      If cAliasTable == "SL2" .OR. cAliasTable == "SL4"
822          If STDPBLength( cAliasTable ) >= nLine
823              oModelTable := oModelPBasket:GetModel( STDGModelID(cAliasTable) )
824              oModelTable:GoLine( nLine )
825              lRet := oModelTable:IsDeleted()
826          Else
827              lRet := .T.

```

```

828     EndIf
829 EndIf
830
831 FwRestRows( aSaveLines )
832
833 // Limpa o vetor para melhor gerenciamento de memoria (Desaloca Memória)
834 aSize( aSaveLines,0 )
835 aSaveLines := Nil
836
837 Return lRet
838
839 //-----
840 /*{Protheus.doc} STDLoadImg
841 Retorna o codigo da imagem do produto contido no repositorio de imagens
842
843 @param      cField      - Nome do campo
844 @return     aField      - Retorna campos do model
845 @obs
846 @sample
847 */
848 //-----
849 Function STDLoadImg( cCodProd )
850 Local aArea := GetArea()
851 Local cRet := ""
852
853 DbSelectArea("SB1")
854 SB1->(DbSetOrder(1)) // B1_FILIAL+B1_COD
855 If DbSeek(xFilial("SB1")+cCodProd)
856     cRet := SB1->B1_BITMAP
857 EndIf
858
859 If Empty(cRet)
860     cRet := "SEM FOTO"
861 EndIf
862
863 RestArea(aArea)
864
865 Return cRet
866
867 //-----
868 /*{Protheus.doc} STDPConfL1
869 Confere se o numero gerado na inicializacao da cesta esta correto , pois pode haver casos do XE/XF ficar corrompidos e
870 retornavam a mesma numeracao , alterando o orçamento.
871 @param      cNumOrc      - Numero do Orcamento
872 */
873 //-----
874 Function STDPConfL1( cNumOrc )
875
876 Local cMay      := ""           // Conteudo da filial e orcamento para nao haver duplicacao
877 Local nTent     := 0           // Tentativas para gravacao do SL1 com a numeracao do orcamento
878 Local lRet      := .T.
879 Local lUpdate   := .F.

```

```

880 Local aSL1Area := {}
881
882 Default cNumOrc      := ""
883
884 If !Empty(cNumOrc)
885     aSL1Area := SL1->(GetArea())
886
887     cMay := Alltrim(xFilial("SL1")) + cNumOrc
888     FreeUsedCode()
889     SL1->( DbSetOrder( 1 ) )
890
891     // Verifica se o numero da cesta nao ja esta sendo utilizado
892     While SL1->(DbSeek(xFilial("SL1")+cNumOrc)) .OR. !MayIUseCode( cMay )
893         SL1->(MsUnlock())      // Caso encontrou algum que ja existe
894         SL1->(ConfirmSX8())    // Confirma e libera
895         If ++nTent > 20
896             LjGrvLog( cNumOrc, "Impossivel gerar num seq de orc correto(STDPConfL1)" )
897             lRet := .F.
898             LjGrvLog( cNumOrc,STR0027+STR0028 )
899             Final(STR0027, STR0028)//Impossivel gerar número sequencial correto da venda, Informe o Suporte de TI para ajustar o número Sequencial da Venda (L1_NUM) na SXE e
            SXF."
900         EndIf
901         LjGrvLog( cNumOrc, "Ajustou o numero do orcamento L1_NUM" )
902         cNumOrc := CriaVar( "L1_NUM" )
903         FreeUsedCode()
904         cMay := Alltrim(xFilial("SL1"))+cNumOrc
905         lUpdate := .T.
906     End
907
908     If lRet .AND. lUpdate
909         STDSPBasket("SL1","L1_NUM"      ,cNumOrc)
910     EndIf
911     RestArea(aSL1Area)
912 EndIf
913
914 Return Nil

```